Tweed: Adaptively Secure Lattice-Based Two-Round Threshold Signatures

Kaijie Jiang[1], Stefano Tessaro[2], Hoeteck Wee[3], and Chenzhi Zhu[3]

[1] Institute for Advanced Study, Tsinghua University, Beijing, China
[2] Paul G. Allen School of Computer Science & Engineering, University of Washington, Seattle, USA
[3] NTT Research, Sunnyvale, USA

Abstract. This paper gives the first lattice-based two-round threshold signature scheme that tolerates the adaptive corruption of up to $T-1$ out of N signers. Our construction is based on the MLWE and MSIS assumptions. We substantially improve upon the only existing adaptively secure lattice-based construction, recently given by Katsumata, Reichle, and Takemure (CRYPTO '24), which requires five rounds.

1 Introduction

A T-out-of-N threshold signature [Des88, DF90] enables a group of N parties to jointly generate digital signatures such that any subset of at least T out of N parties can produce a valid signature, while any adversary corrupting fewer than T of these parties cannot produce such a signature. Crucially, to achieve this, the secret signing key is secret-shared among these N parties, e.g., as the result of running a distributed key generation (DKG) protocol. Threshold signatures play a crucial role in securing decentralized systems, distributed key management, and fault-tolerant cryptographic protocols. They have found widespread practical adoption in securing digital wallets. Ongoing IETF [CKGW22] and NIST [Natnt] standardization efforts further highlight the practical significance of threshold signatures.

Adaptive security. A central question is *how* the adversary decides which parties to corrupt. As a threshold signature could be repeatedly executed over a long time span, e.g., in order to allow signing of multiple messages using the same secret-shared secret key, the most conservative assumption is that adversaries corrupt parties *adaptively*—i.e., they corrupt parties during the protocol execution, and choose the next party to corrupt based on the execution so far. Designing schemes that are *provably* adaptively secure presents a number of technical challenges. For this reason, many schemes are only proved secure against *static* adversaries corrupting all parties ahead of time, while their adaptive security is a conjecture. It has been a folklore heuristic to assume that, as long as a protocol which is proved statistically secure is not obviously adaptively insecure, then it is also adaptively secure. The existing separations between adaptive and static security typically rely on some form of committee sampling process, where one expects the fraction of corrupted parties to be preserved when sampling a small committee of parties running a protocol (e.g. [CFGN96]).

This state of affairs, however, is rather unsatisfactory. This has become even more true as NIST has explicitly stated its interest in adaptive security as part of their potential standardization of threshold signatures. This has motivated a number of recent works that have resulted in a growing number of *practical* adaptively secure schemes based on traditional assumptions in pairing-friendly [BL22, DR24] and pairing-free groups [Mak22, BLT+24, KRT24, BDLR25b, Che25a, CKK+25, BDLR25a, GCRS25, Che25b]. A recent work by Crites and Stewart [CS25], while falling short of giving an efficient attack, shows for the first time that adaptive attacks can leverage very different structures than static ones, hence reducing our confidence in the folklore conjecture that most statically secure schemes are also adaptively secure.

Lattice-based threshold signatures. The prospect of practical quantum computing motivates the urgent need to develop threshold signatures from quantum-resistant assumptions, but all of the aforementioned adaptively secure threshold signatures are not post-quantum secure. The most promising avenue towards practical post-quantum secure signatures relies on *lattices*, and indeed, NIST has recently selected DILITHIUM [LDK+22] and FALCON [PFH+22], two lattice-based signature schemes, for standardization. This has motivated the design of a number of lattice-based threshold signatures over the last few years, such as e.g. [BGGK17, BGG+18, ASY22, GKS24, DKM+24, EKT24, KRT24, CATZ24, ZT25, BKL+25, dPENP25, PKN+25, BCdP+25, BdCE+25]. However, among these, only the scheme of [KRT24] is proved to achieve adaptive security. Their protocol is fairly expensive in that it requires five rounds of interaction. In contrast, the most efficient lattice-based threshold

Scheme	Rounds	Corruptions
[BGGK17, BGG+18, ASY22]	1	static
[EY24]	1	part. ad.
[GKS24, GTJ+26]	0 + 2	static
[DKM+24]	0 + 3	static
[EKT24, CATZ24, ZT25, BKL+25]	1 + 1	static
[KRT24]	2 + 3	adaptive
This work	0 + 2	adaptive

Scheme	Rounds	Assumption
Twinkle [BLT+24]	2 + 1	DDH
KRT [KRT24]	2 + 3	M-LWE / DL
Glacius [BDLR25b]	2 + 3	DDH
Gargos [BDLR25a]	1 + 2	DDH
FROST-Mask [Che25b]	2 + 1	AOMDL
Dazzle [Che25a]	0 + 2	DDH
HBTS-Mask [Che25b]	0 + 2	DL
This work	0 + 2	M-LWE

Fig. 1. Summary of T-out-of-N threshold signatures. The notation $X + Y$ denotes the number of offline (i.e., message independent) and online (i.e., message dependent) rounds, respectively. On the left, we list all existing constructions from lattices. On the right, we give threshold signatures proved to be adaptively secure from pairing-free groups, without the AGM, and lattices. We only consider schemes that are stateless and without erasures, and for which optimal corruptions (up $T - 1$ parties) are tolerated.

signatures only require two rounds (e.g., [EKT24, CATZ24, BKL+25]), but are only proved statically secure. Furthermore, less computationally efficient *round-optimal* solutions exist [BGGK17, BGG+18, ASY22], relying on FHE, and these have been (at best) proved [EY24] to satisfy "partially adaptive" security, where the adversary can defer choosing corruptions to a later point in time after the execution of the protocol has begun, but these corruptions need to be specified *all at once*.

We also point out that the lattice setting offers other examples where the heuristic that a static security proof suffices to ensure adaptive security fails. For example, a recent attack [AMR25] on the "adaptive LWE" assumption [QWW18] as well as the conjectured adaptive security of the attribute-based encryption scheme in [BGG+14] demonstrates that the heuristic is false in general for lattice-based schemes.

Our results. In this work, we ask the following question:

> *Can we build more efficient lattice-based threshold signatures which provably withstand adaptive corruptions?*

We answer this question affirmatively by presenting Tweed,[4] the first construction of a *two-round* threshold signature scheme based on the hardness of the Module Learning With Errors (MLWE) and Module Short Integer Solution (MSIS) problems, achieving adaptive security in the random oracle model. This is a significant improvement over the only existing adaptively secure lattice-based threshold signature [KRT24], which requires five rounds. In Figure 1, we position our scheme in the broader context of the most closely related schemes, both from lattices but also from pairing-free groups. We note that latter serve as natural starting points for lattice-based schemes, and it is unlikely that we can design simple lattice-based schemes that achieve goals not achievable in the pairing-free setting.

Our parameter estimation indicates that Tweed produces signatures of size roughly 170KB. These are arguably larger than those achieved for example by EKT [EKT24], which are in the ballpark of 30-50KB when using the LWE/SIS security proof from [ZT25], or 10KB when using the cryptanalysis driven parameter estimation adopted in [EKT24]. However, we see our main contribution in introducing new techniques for adaptive security, and our ability to achieve two rounds. We leave the improvement of parameters to future works.

Technical barriers. We only mention a few high-level ideas behind our scheme, and refer the reader to the technical overview for further details. To start with, it is helpful to point out that a major barrier in proving adaptive security in many prior schemes (such as EKT [EKT24] and CATZ [CATZ24]) is the use of rewinding in conjunction with the simulation of corruptions. In particular, rewinding forces the reduction to consider two (correlated) executions of the experiment, and the simulation of corruptions would fail if more than $T - 1$ corruptions occur *overall* across both executions. However, with adaptive corruptions we cannot give this guarantee, as all corruptions may occur *after* the forking point. (The concrete way in which this problem surfaces differs across various schemes and proofs, but the essence remains the same.)

[4] Tweeds are a staple of traditional Scottish, Irish, Welsh, and English clothing, typically woven with a lattice-like structure. They are made to withstand harsh climate, and known for durability and warmth. The first letter T refers to both Threshold and Two-round.

We also point out that a similar barrier has been encountered in the pairing-free setting, e.g., in the analysis of FROST [BCK+22, CKK+25]. Still, for group based schemes, this issue can be overcome in several ways, which we can hope to adapt to the lattice setting. One approach that is specific to the group setting is the use of ideal models (such as the Algebraic Group Model [FKL18]) to obtain security proofs that avoid rewinding [CKM23, CKK+25]. Alternatively, some works [CKM23, CKK+25] weaken the model to only allow corruption of fewer than $T/2$ parties, but here we target tolerating up to $T-1$ corruptions. Another route is to reduce the security of the threshold signature to the security of the non-threshold version of this signature, and relegate rewinding to proof of security of the latter, which does not deal with corruptions. This is the approach behind [EKT24], leading to a five-round scheme. Chen [GCRS25] very recently used a similar approach and obtained two-round protocols, but only in the group setting. Chen's ideas are promising, but we go in a different direction by considering the final option, namely to develop schemes where the security reduction *knows* the secret key (and thus can simulate corruptions for free) and (typically) does not rewind. Two examples of such schemes are Dazzle [Che25a] and Twinkle [BLT+24].

The threshold signature presented in this paper is the result of an attempt to transform Dazzle into a lattice-based scheme. This requires overcoming a number of technical challenges. For example, a Dazzle signature includes a NIZK proof (obtained from a simple Σ-protocol) that ensures, with high probability, that part of the signature satisfies a particular relation which can, alternatively, be efficiently checked given the secret key. In the lattice setting, however, we will need to deal with the well-known issue that the natural Σ-protocol analogue only offers a relaxed soundness condition, and that checking for this condition is no longer efficient, even given the secret key. This requires introducing a very different technique, which we explain below. We will also show that the folklore intuition that one cannot rewind in these proofs is not fully correct—part of our proof will indeed use rewinding.

What we do not achieve. First off, we note that the first round, in our scheme is message-dependent, unlike in some of the other schemes only proved to be statically secure [EKT24, CATZ24]. A message-independent first round can be preprocessed, and signing can then be completed later, in a single round, once the message becomes known. Such schemes promise to be nearly as good as round-optimal ones. In practice, however, this feature is likely less beneficial than one may think, as ensuring that parts of the preprocessed first-round communication are not reused adds significant complexity from an engineering standpoint, and such reuse typically leads to a complete loss of security. Figure 1 also highlights that our scheme matches the state-of-the art for pairing-free schemes, and hence we are unlikely to overcome this barrier using our techniques.

Another limitation of our scheme (also inherited from Dazzle) is that we do not achieve strong unforgeability. Furthermore, we do not achieve identifiable abort—some recent works [dPENP25, PKN+25] reiterate that this can be quite complex in the lattice setting, and defer investigating achieving it to future work.

2 Technical Overview

Most of the technical novelty in this work can be understood in the context of building a standard (one-party) signature scheme. We fix a ring $\mathcal{R}$ (either $\mathbb{Z}$ or $\mathbb{Z}[X]/(X^d+1)$) and let $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$.

Warm-up. The starting point of our construction is an adaption of the Dazzle signature scheme to the lattice setting, following [Lyu12].

- The secret key is a low-norm matrix $\mathbf{S} \in \mathcal{R}_q^{m\times k}$, and the public key consists of the matrices $\mathbf{A} \in \mathcal{R}_q^{\ell\times m}$ and $\mathbf{P} = \mathbf{AS}$.
- To sign a message μ, the signer first samples a low-norm vector $\mathbf{r} \in \mathcal{R}_q^m$, computes

$$\mathbf{U} = H_1(\mu) \in \mathcal{R}_q^{t\times m},\ \mathbf{Y} = \mathbf{US},$$
$$\mathbf{w} = \begin{bmatrix}\mathbf{A}\\ \mathbf{U}\end{bmatrix}\mathbf{r},\ \mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w}) \in \{0,1\}^k,\ \mathbf{z} = \mathbf{r} + \mathbf{Sc}$$

  and outputs $(\mathbf{Y}, \mathbf{w}, \mathbf{z})$ as the signature.
- The verifier checks that $\|\mathbf{z}\|$ is small and that $\begin{bmatrix}\mathbf{A}\\ \mathbf{U}\end{bmatrix}\mathbf{z} = \mathbf{w} + \begin{bmatrix}\mathbf{P}\\ \mathbf{Y}\end{bmatrix}\mathbf{c}$.

Namely, $(\mathbf{w}, \mathbf{z})$ is a non-interactive zero-knowledge proof of knowledge (zkPoK) of a low-norm $\mathbf{S}$ such that $\begin{bmatrix}\mathbf{A}\\ \mathbf{U}\end{bmatrix}\mathbf{S} = \begin{bmatrix}\mathbf{P}\\ \mathbf{Y}\end{bmatrix}$. In particular, if we omit $(\mathbf{U}, \mathbf{Y})$, this is the Lyubashevsky signature, with noise flooding in place of aborts and rejection sampling.

Proof strategy. Our high-level proof strategy is the same as that in Dazzle; we later explain how to extend this proof strategy to the threshold setting with adaptive corruptions. We would like to prove security in two steps:

– Step 1: show that any valid forgery $(\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*)$ on a message μ^* must satisfy $\mathbf{U}^*\mathbf{S} = \mathbf{Y}^*$, where $\mathbf{U}^* = H_1(\mu^*)$.
– Step 2: switch the distributions of $\mathbf{A}, \mathbf{U}$ in the public key and signing queries to be "lossy" so that $\mathbf{AS}, \mathbf{US}$ "lose" information about $\mathbf{S}$; this step relies on a computational assumption. Combined with the zero-knowledge property, we can argue that $\mathbf{U}^*\mathbf{S}$ is statistically hidden from the adversary.

At the end of Step 2, security follows from a straight-forward statistical argument, since the probability that $\mathbf{Y}^*$ is equal the random matrix $\mathbf{U}^*\mathbf{S}$ is exponentially small. Throughout the security proof, the reduction knows the secret key $\mathbf{S}$. This allows us to handle adaptive corruptions in the threshold setting.

Adapting Step 1. The main technical challenge lies in adapting Step 1 to the lattice setting. In Dazzle, Step 1 relies on a statistical argument, and uses basic linear algebra to prove a statement of the following form: if $\mathbf{A}$ is full-rank and $\mathbf{Y} \neq \mathbf{U}^*\mathbf{S}$, then for any $\mathbf{w}$, there exists a unique challenge $\mathbf{c}$ for which the verifier can pass the verification check. We take a completely different approach. Fix $\mathbf{A}, \mathbf{Y}, \mathbf{w}$ and suppose we have a cheating adversary $\mathcal{A}$ such that

$$\Pr_{\mathbf{c} \leftarrow \{0,1\}^k}[\text{verifier accepts } \mathbf{z}^* : \mathbf{z}^* \leftarrow \mathcal{A}(\mathbf{c})] \geq \epsilon$$

This means that upon "rewinding" $\mathcal{A}$ with two random challenges $\mathbf{c}_1, \mathbf{c}_2$ and subtracting to cancel out $\mathbf{w}$, we have

$$\Pr_{\mathbf{c}_1,\mathbf{c}_2 \leftarrow \{0,1\}^k}\Big[\begin{bmatrix} \mathbf{A} \\ \mathbf{U}^* \end{bmatrix} (\mathbf{z}_1^* - \mathbf{z}_2^*) = \begin{bmatrix} \mathbf{P} \\ \mathbf{Y}^* \end{bmatrix} (\mathbf{c}_1 - \mathbf{c}_2) \\ \wedge \ \|\mathbf{z}_1^* - \mathbf{z}_2^*\| \text{ is small} : \ \mathbf{z}_b^* \leftarrow \mathcal{A}(\mathbf{c}_b), \ b \in \{1,2\}\Big] \ \geq \ \epsilon^2$$

Adapting the Dazzle argument to our setting, we will set $\ell \gg m$ (i.e., $\mathbf{A}$ is tall) so that

$$\mathbf{A}(\mathbf{z}_1^* - \mathbf{z}_2^*) = \overbrace{\mathbf{AS}}^{\mathbf{P}}(\mathbf{c}_1 - \mathbf{c}_2) \wedge \|\mathbf{z}_1^* - \mathbf{z}_2^*\| \text{ is small} \Longrightarrow \mathbf{z}_1^* - \mathbf{z}_2^* = \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2) \tag{1}$$

Combining the latter with $\mathbf{U}^*(\mathbf{z}_1^* - \mathbf{z}_2^*) = \mathbf{Y}^*(\mathbf{c}_1 - \mathbf{c}_2)$ yields $(\mathbf{Y}^* - \mathbf{U}^*\mathbf{S})(\mathbf{c}_1 - \mathbf{c}_2) = \mathbf{0}$, which means we want to argue that if ϵ is non-negligible, then

$$\Pr_{\mathbf{c}_1,\mathbf{c}_2 \leftarrow \{0,1\}^k}[(\mathbf{Y}^* - \mathbf{U}^*\mathbf{S})(\mathbf{c}_1 - \mathbf{c}_2) = \mathbf{0}] \geq \epsilon^2 \Longrightarrow \mathbf{Y}^* - \mathbf{U}^*\mathbf{S} = \mathbf{0} \tag{2}$$

Unfortunately, (2) is false in general, even if we relax the RHS to be $\approx \mathbf{0}$. The problem lies with wrap-arounds modulo q: for instance, if q is a multiple of 4 and $\mathbf{Y}^* - \mathbf{U}^*\mathbf{S}$ is the matrix where every entry is $q/4$, then the LHS holds with probability roughly $1/4$. The only fix we know for proving a statement in (2) is to replace $\mathbf{c}$ with matrices of width much larger than k; however, that would significantly blow up the size of the signatures.

We now describe the two modifications to Step 1:

– Our first key insight is that the property on the LHS of (2) is efficiently checkable in time $\mathsf{poly}(1/\epsilon)$, even without rewinding the adversary! This means that applying a computational assumption in Step 2 will (approximately) preserve the property, which must then still hold after we replace $\mathbf{U}^*\mathbf{S}$ with a random matrix. At this point, we use the fact that as long as $\mathbf{c}_1 - \mathbf{c}_2 \neq 0$ (which happens with probability $1 - 2^{-k}$), the vector $(\mathbf{Y}^* - \mathbf{U}^*\mathbf{S})(\mathbf{c}_1 - \mathbf{c}_2)$ is uniformly distributed over $\mathcal{R}_q^t$, and equals $\mathbf{0}$ with probability at most q^{-dt}.
– In order to keep the signature sizes small, we use $\ell \ll m$ (i.e. $\mathbf{A}$ is wide, following [Lyu12]) and instead, rely on SIS in (1) to argue that $\mathbf{z}_1^* - \mathbf{z}_2^* = \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2)$. Indeed, our security reduction for SIS will rewind the adversary $\mathcal{A}$ to obtain $\mathbf{z}_1^*, \mathbf{z}_2^*$. Note that the reduction, unlike the ones in [Lyu12, CATZ24, ZT25], does not require that $\mathbf{S}$ has entropy from $\mathcal{A}$'s view-point. We believe that the same idea can be used in the Dazzle protocol to save one group element in the signing protocol.

Completing the proof. We proceed to Step 2. Another advantage of our second modification is that it simplifies Step 2. Now that $\mathbf{A}$ is wide (and not tall or full-rank), we can immediately deduce that $\mathbf{AS}$ loses information about $\mathbf{S}$, without having to modify the distribution of $\mathbf{A}$. We can then rely on LWE to replace $\mathbf{U}$ with $\mathbf{TA} + \mathbf{E}''$ so that $\mathbf{US} \approx \mathbf{TAS}$ only leaks $\mathbf{AS}$. Here, we need to introduce additional error terms to $(\mathbf{US}, \mathbf{Ur})$ to mask the leakage in $\mathbf{E}''\mathbf{S}, \mathbf{E}''\mathbf{r}$. At this point, $\mathbf{U}^* = H_1(\mu^*)$ remains uniformly random, so $(\mathbf{AS}, \mathbf{U}^*\mathbf{S})$ is statistically close to the uniform distribution.

Towards the threshold scheme. We describe one more modification we make to the security proof, which is necessary for the threshold scheme. Roughly speaking, in the threshold variant, signing is divided into two steps $\mathsf{Sign}_1, \mathsf{Sign}_2$ that take place over two rounds. Namely,

$$\begin{aligned}
&\underline{\mathsf{Sign}_1(\mathbf{S}, \mu)}: && \mathbf{U} = H_1(\mu),\ \mathbf{Y} = \mathbf{U}\mathbf{S} + \mathbf{E}',\ \mathbf{w} = \begin{bmatrix} \mathbf{A}\mathbf{r} \\ \mathbf{U}\mathbf{r} + \mathbf{e} \end{bmatrix} \\
&\underline{\mathsf{Sign}_2(\mathbf{S}, \mu, \mathbf{r}, (\mathbf{Y}, \mathbf{w}))}: && \mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w}),\ \mathbf{z} = \mathbf{r} + \mathbf{S}\mathbf{c}
\end{aligned}$$

The input $\mathbf{r}$ to Sign_2 corresponds to $\mathbf{r}$ computed in Sign_1, but the input $(\mathbf{Y}, \mathbf{w})$ to Sign_2 incorporates values contributed by the other signers, and may not correspond to the value computed in Sign_1. This requires modifying the zero-knowledge argument for Step 2, since we may no longer program $\mathbf{c}$ while simulating $\mathbf{w}$. Ignoring the noise terms $\mathbf{E}''$, $\mathbf{e}$ for simplicity, we want to show that

$$\mathbf{w} = \begin{bmatrix} \mathbf{A} \\ \mathbf{T}\mathbf{A} \end{bmatrix} \mathbf{r},\ \mathbf{z} = \mathbf{r} + \mathbf{S}\mathbf{c}$$

doesn't leak information about $\mathbf{S}$ beyond $\mathbf{A}\mathbf{S}$, for any $\mathbf{c} \in \{0,1\}^k$ that may depend adversarially on $\mathbf{w}$. First, we sample $\mathbf{A}$ together with a low-norm matrix $\mathbf{D}$ such that $\mathbf{A}\mathbf{D} = \mathbf{0}$. As an aside, to keep the signature sizes small, we derive $\mathbf{A} = [\mathbf{B}_0 \mid \mathbf{B}_0\mathbf{R} + \mathbf{E} \mid \mathbf{I}_\ell]$ from an LWE sample following [Lyu12].

Next, we sample gaussian $\mathbf{S}^{(0)}, \mathbf{S}^{(1)}, \mathbf{r}^{(0)}, \mathbf{r}^{(1)}$ from the appropriate distributions (using gaussian convolution) so that

$$(\mathbf{S}, \mathbf{r}) \approx_s (\mathbf{S}^{(0)} + \mathbf{D}\mathbf{S}^{(1)}, \mathbf{r}^{(0)} + \mathbf{D}\mathbf{r}^{(1)})$$

Then, $\mathbf{A}\mathbf{S} = \mathbf{A}\mathbf{S}^{(0)}$ perfectly hides $\mathbf{S}^{(1)}$, and for statistical argument at the end of Step 2, it suffices to show that $(\mathbf{w}, \mathbf{z})$ also hides $\mathbf{S}^{(1)}$. Observe that $\mathbf{A}\mathbf{r} = \mathbf{A}\mathbf{r}^{(0)}$ and thus $\mathbf{w}$ also perfectly hides $\mathbf{r}^{(1)}$. We have $\mathbf{z} = (\mathbf{r}^{(0)} + \mathbf{S}^{(0)}\mathbf{c}) + \mathbf{D}(\mathbf{r}^{(1)} + \mathbf{S}^{(1)}\mathbf{c})$, upon which we can use $\mathbf{r}^{(1)}$ to mask $\mathbf{S}^{(1)}\mathbf{c}$.

Overview of threshold scheme. In the threshold scheme, signing is distributed across a subset SS of T out of N signers. We follow the same high-level approach in Dazzle and prior lattice-based schemes:

- Key generation assigns each party a Shamir T-out-of-N share $\mathbf{S}_i$ of the signing key $\mathbf{S}$ (so that any subset of $T-1$ shares are uniformly random matrices over $\mathcal{R}_q$);
- To sign, each party $i \in \mathrm{SS}$ computes a partial signature $(\mathbf{Y}_i, \mathbf{w}_i, \mathbf{z}_i)$ on μ, so that the sum

$$(\mathbf{Y}, \mathbf{w}, \mathbf{z}) = \left(\sum_{i \in \mathrm{SS}} \mathbf{Y}_i, \sum_{i \in \mathrm{SS}} \mathbf{w}_i, \sum_{i \in \mathrm{SS}} \mathbf{z}_i \right)$$

  constitutes a valid signature on μ.

Concretely, during signing, party i samples its own $\mathbf{E}'_i, \mathbf{r}_i, \mathbf{e}_i$, and computes $\mathbf{Y}_i, \mathbf{w}_i, \mathbf{z}_i$ as before, except with $L_{\mathrm{SS},i} \cdot \mathbf{S}_i$ in place of $\mathbf{S}$, where $L_{\mathrm{SS},i}$ are the standard Lagrangian coefficients. Note that we still compute $\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$, which means signing takes two rounds, in order to first compute $\mathbf{Y}, \mathbf{w}$ before computing $\mathbf{z}_i$. Correctness follows readily from $\mathbf{S} = \sum_{i \in \mathrm{SS}} L_{\mathrm{SS},i}\mathbf{S}_i$. For security, introduce additional masking following [EKT24] in order to accommodate the large $L_{\mathrm{SS},i}$'s.

Our threshold scheme. The scheme proceeds as follows:

- During key generation, party i additionally receives a masking key mk_i that allows the party given SS and a tag $\mathsf{tag} \in \{0,1\}^*$, derive a mask $\mathsf{mask}_{i,\mathrm{SS},\mathsf{tag}}$, which is a (pseudo)random matrix/vector over $\mathcal{R}_q$ subject to the constraint $\sum_{i \in \mathrm{SS}} \mathsf{mask}_{i,\mathrm{SS},\mathsf{tag}} = \mathbf{0}$.
- During signing, party i additionally masks $\mathbf{Y}_i, \mathbf{z}_i$ (but not $\mathbf{w}_i$). Namely,

$$\begin{aligned}
\underline{\mathsf{Sign}_1}: \mathbf{Y}_i &= L_{\mathrm{SS},i} \cdot \mathbf{U}\mathbf{S}_i + \mathbf{E}'_i + \mathsf{mask}_{i,\mathrm{SS},(\mathsf{pk},\mu)} \\
\mathbf{w}_i &= \begin{bmatrix} \mathbf{A}\mathbf{r}_i \\ \mathbf{U}\mathbf{r}_i + \mathbf{e}_i \end{bmatrix} \\
\underline{\mathsf{Sign}_2}: \quad \mathbf{c} &= H_2(\mu, \mathbf{Y}, \mathbf{w}) \\
\mathbf{z}_i &= \mathbf{r}_i + L_{\mathrm{SS},i} \cdot \mathbf{S}_i\mathbf{c} + \mathsf{mask}_{i,\mathrm{SS},(\mathsf{pk},\mu,\{(j,\mathbf{Y}_j,\mathbf{w}_j)\}_{j\in\mathrm{SS}})}
\end{aligned}$$

  where $\mathsf{Sign}_1, \mathsf{Sign}_2$ denote the first and second round respectively. One difference from [EKT24] is that we mask the first-round message $\mathbf{Y}_i$, in addition to the second-round message $\mathbf{z}_i$.

Proof overview. In the security proof, we use CS to denote the set of corrupted parties. Thanks to the masks, the collection of partial signatures $\{\mathbf{Y}_i, \mathbf{w}_i, \mathbf{z}_i\}_{i\in\mathsf{SS}\setminus\mathsf{CS}}$ only leak $\{\mathbf{w}_i\}_{i\in\mathsf{SS}\setminus\mathsf{CS}}, \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{Y}_i, \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{z}_i$. It's not hard to see that we can simulate the latter two partial sums given

$$\mathbf{Y} = \mathbf{US} + \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}} \mathbf{E}'_i,\ \mathbf{z} = \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}} \mathbf{r}_i + \mathbf{Sc}$$

along with $\{\mathbf{S}_i, \mathsf{mk}_i\}_{i\in\mathsf{CS}}$. The proof then proceeds in two steps:

- First, we introduce an intermediate security game IUF (cf. Fig 7) where-in the adversary only sees

$$\{\mathbf{w}_i\}_{i\in\mathsf{SS}\setminus\mathsf{CS}}, \mathbf{Y} = \mathbf{US} + \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}} \mathbf{E}'_i, \mathbf{z} = \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}} \mathbf{r}_i + \mathbf{Sc} \tag{3}$$

  instead of all of the partial signatures. Moreover, the adversary is allowed to obtain *adaptively* all but one of the values in $\{(\mathbf{E}'_i, \mathbf{r}_i)\}_{i\in\mathsf{SS}\setminus\mathsf{CS}}$. Thanks to our proof strategy, which largely avoids rewinding, we can prove the hardness of IUF under the standard lattice assumptions similarly to the single-signer case.
- Next, we prove hardness of IUF implies threshold security. The reduction from IUF to the threshold security is similar to the one in [ZT25] for the static setting, with additional care needed to program the masks in the presence of adaptive corruptions. Here, we treat the pseudorandom functions used to derive the masks as programmable random oracles (cf. [Nie02]).

We defer the detailed proof to Section 5.

3 Preliminaries

3.1 Notation

For any positive integers $k < n$, $[n]$ denotes $\{1, \dots, n\}$, and $[k..n]$ denotes $\{k, \dots, n\}$. We use κ to denote the security parameter. For a sequence of variables $x_1, \dots, x_\ell$, we use $x_{[i]}$ to denote $(x_1, \dots, x_i)$ and $x_{[i..j]}$ to denote $(x_i, \dots, x_j)$. For a matrix $\mathbf{A}$, we define the norm $\|\mathbf{A}\|_{1,2} := \max_{1\le j\le k} \|\mathbf{A}_{:j}\|$, $\mathbf{A}_{:j}$ is the j'th column of $\mathbf{A}$.

For a finite set S, $|S|$ denotes the size of S, and $x \leftarrow S$ denotes sampling an element uniformly from S and assigning it to x. For a distribution $\mathcal{D}$, denote $\mathsf{Supp}(\mathcal{D})$ as the support of $\mathcal{D}$, and $x \leftarrow \mathcal{D}$ denotes sampling x according to $\mathcal{D}$. For any $y \in \mathsf{Supp}(\mathcal{D})$, denote $\mathcal{D}(y) := \Pr_{x\leftarrow\mathcal{D}}[x = y]$, and for $S \subseteq \mathsf{Supp}(\mathcal{D})$, denote $\mathcal{D}(S) := \Pr_{x\leftarrow\mathcal{D}}[x \in S]$. For any set T and any function $F : \mathsf{Supp}(\mathcal{D}) \to T$, denote $F(\mathcal{D})$ as the distribution of $F(x)$ for x sampled from $\mathcal{D}$.

For a distribution $\mathcal{D}$, we denote $\mathcal{D}^k$ as a matrix $\mathbf{M} = (\mathbf{m}_1, \dots, \mathbf{m}_k)$ with k columns, and each column is drawn i.i.d. from distribution $\mathcal{D}$, where $\mathsf{Supp}(\mathcal{D})$ is a column vector space.

3.2 Polynomial Rings

Let q be an odd prime and d be a power of 2. We denote the ring $\mathcal{R} := \mathbb{Z}[X]/(X^d+1)$, contained in the cyclotomic field $K := \mathbb{Q}[X]/(X^d+1)$, and let $\mathcal{R}_q := \mathcal{R}/q\mathcal{R} \cong \mathbb{Z}_q[X]/(X^d+1)$. Denote $K_\mathbb{R} := \mathbb{R}\otimes K \cong \mathbb{R}[X]/(X^d+1)$. For an element $v \in K_\mathbb{R}$, where $v = \sum_{i=0}^{d-1} v_i X^i$, we denote its conjugate as $v^* = \sum_{i=0}^{d-1} -v_i X^{d-i}$. We use ϕ to denote the coefficient embedding that embeds $K_\mathbb{R}$ in $\mathbb{R}^d$, and ϕ maps v to vector $(v_0, \dots, v_{d-1}) \in \mathbb{R}^d$. When applying ϕ to a vector $\mathbf{v} \in K_\mathbb{R}^m$, ϕ maps $\mathbf{v}$ to a vector in $\mathbb{R}^{md}$ by applying ϕ to each entry of $\mathbf{v}$. The map ϕ is a bijection, and we denote its inverse by ϕ^{-1}. An ℓ_p-norm of $\mathbf{v} \in K_\mathbb{R}^m$ is given by

$$\|\mathbf{v}\|_p := \|\phi(\mathbf{v})\|_p = \left(\sum_{i=1}^{m}\sum_{j=0}^{d-1} |v_{i,j}|^p\right)^{\frac{1}{p}},$$

where $v_{i,j}$ denotes the coefficient of X^j of the i-th entry of $\mathbf{v}$. Additionally, the ℓ_∞-norm of $\mathbf{v}$ is defined as $\|\mathbf{v}\|_\infty := \max_{i\in[m],j\in[0..d-1]} |v_{i,j}|$. For the ℓ_2-norm, we omit the subscript and denote $\|\mathbf{v}\|$ as the ℓ_2-norm of $\mathbf{v}$. Denote the conjugate transpose of $\mathbf{v} \in K_\mathbb{R}^m$ as $\mathbf{v}^\dagger := (\mathbf{v}^*)^T$. We define the inner product of two vectors $\mathbf{v}, \mathbf{v}' \in K_\mathbb{R}^m$ as $\langle \mathbf{v}, \mathbf{v}'\rangle := \phi(\mathbf{v})^T\phi(\mathbf{v}') = \langle\phi(\mathbf{v}), \phi(\mathbf{v}')\rangle$. We have $\|\mathbf{v}\| = \langle \mathbf{v}, \mathbf{v}\rangle$. We say $\mathbf{v}$ is a unit vector if $\|\mathbf{v}\| = 1$.

Also, we define a map ϕ_M that maps each element in $K_\mathbb{R}$ to a matrix in $\mathbb{R}^{d\times d}$ as follows. Let $M_X := \begin{pmatrix} \mathbf{0} & -1 \\ \mathbf{I}_{d-1} & \mathbf{0} \end{pmatrix} \in \mathbb{R}^d$, where I_{d-1} is the identity matrix in $\mathbb{R}^{d-1}$. For each $v \in K_\mathbb{R}$, $\phi_M(v) := \sum_{i=0}^{d-1} v_i M_X^i$, which can be viewed as the matrix representation of v. In particular, for ϕ and ϕ_M, the following properties hold: for any $v, v' \in K_\mathbb{R}$, $\phi_M(v^*) = \phi_M(v)^T$, $\phi_M(vv') = \phi_M(v)\phi_M(v')$ and $\phi_M(v)\phi(v') = \phi(vv')$. We extend the above definitions to $\mathcal{R}_q$ by representing each $v \in \mathcal{R}_q$ as $v = \sum_{i=0}^{d-1} v_i X^i$, where $v_i \in \{-(q-1)/2, \ldots, (q-1)/2\}$.

For a matrix $\mathbf{M} \in K_\mathbb{R}^{m\times m}$, we denote its conjugate transpose as $\mathbf{M}^\dagger = (\mathbf{M}^*)^T$, and we say $\mathbf{M}$ is *hermitian* if $\mathbf{M} = \mathbf{M}^\dagger$. We say $\mathbf{M}$ is *positive definite* if and only if $\mathbf{M}$ is hermitian and for all $\mathbf{x} \in K_\mathbb{R}^m \setminus \{\mathbf{0}\}$, $\langle \mathbf{x}, \mathbf{M}\mathbf{x}\rangle > 0$, or equivalently, $\phi_M(\mathbf{M})$ is positive definite. Also, denote $\sigma_{\min}(\mathbf{M}) := \inf_{\mathbf{x}\in K_\mathbb{R}^m, \|\mathbf{x}\|=1} \langle \mathbf{x}, \mathbf{M}\mathbf{x}\rangle$ as the smallest singular value of $\mathbf{M}$ and $\sigma_{\max}(\mathbf{M}) := \sup_{\mathbf{x}\in K_\mathbb{R}^m, \|\mathbf{x}\|=1} \langle \mathbf{x}, \mathbf{M}\mathbf{x}\rangle$ as the largest singular value of $\mathbf{M}$.

3.3 Lattices and Discrete Gaussian Distributions

In this subsection, we give definitions for lattices and discrete Gaussian distributions over $\mathbb{R}$ and $K_\mathbb{R}$. An m-dimensional lattice Λ over $\mathbb{Z}$ (resp. $\mathcal{R}$) is a discrete additive subgroup of $\mathbb{Z}$ (resp. $\mathcal{R}$). Equivalently,

$$\Lambda = \mathcal{L}(\{\mathbf{b}_1, \ldots, \mathbf{b}_k\}) := \left\{ \sum_{i\in[k]} x_i \mathbf{b}_i : x_i \in \mathbb{Z} \right\}$$

for a set of linearly independent vectors $\mathbf{b}_1, \ldots, \mathbf{b}_k \in \mathbb{Z}^m$ (resp. $\mathcal{R}^m$), which is referred to as a basis of Λ. The size k is the *rank* of the lattice Λ. We say Λ is a *full rank* lattice if $k = m$ (resp. $k = md$ for Λ over $\mathcal{R}$). For any $\mathbf{a} \in \mathbb{Z}^m$ (resp. $\mathcal{R}^m$), $\Lambda + \mathbf{a}$ is a *coset* of Λ. The *dual lattice* of Λ is denoted as $\Lambda^* = \{\mathbf{x} \in \mathsf{Span}(\Lambda) : \forall\, \mathbf{y} \in \Lambda, \langle \mathbf{x}, \mathbf{y}\rangle \in \mathbb{Z}\}$. A Λ-subspace is the linear span of some subset of Λ, i.e., a subspace S such that $S = \mathsf{Span}(S \cap \Lambda)$.

For a matrix $\mathbf{A} \in \mathcal{R}_q^{k\times m}$, we define the $\mathcal{R}$-lattice $\Lambda_q^\perp(\mathbf{A}) \subseteq \mathcal{R}^m$ as

$$\Lambda_q^\perp(\mathbf{A}) := \{\mathbf{x} \in \mathcal{R}^m : \mathbf{A}\mathbf{x} = \mathbf{0} \mod q\} .$$

We know $\Lambda_q^\perp(\mathbf{A})$ has full-rank since $q\mathcal{R}^m \subseteq \Lambda_q^\perp(\mathbf{A})$.

For a positive definite matrix $\Sigma \in \mathbb{R}^{m\times m}$ (resp. a positive definite matrix $\Sigma \in K_\mathbb{R}^{m\times m}$) and a vector $\mathbf{c} \in \mathbb{R}^m$ (resp. $K_\mathbb{R}^m$), we define the function $\rho_{\Sigma,\mathbf{c}}$ over $\mathbb{R}^m$ (resp. $K_\mathbb{R}^m$) as

$$\rho_{\Sigma,\mathbf{c}}(\mathbf{x}) := \exp\left(-\pi\langle \mathbf{x} - \mathbf{c}, \Sigma^{-1}(\mathbf{x} - \mathbf{c})\rangle\right) .$$

Then, we denote $\mathcal{D}_{\Lambda+\mathbf{a},\Sigma,\mathbf{c}}$ as the discrete Gaussian distribution over a lattice coset $\Lambda + \mathbf{a} \subseteq \mathbb{Z}^m$ (resp. $\mathcal{R}^m$) with covariance matrix Σ, centered at $\mathbf{c} \in \mathbb{R}^m$, where for $\mathbf{x} \in \Lambda + \mathbf{a}$, we define

$$\mathcal{D}_{\Lambda+\mathbf{a},\Sigma,\mathbf{c}}(\mathbf{x}) := \Pr[\mathbf{x} \leftarrow \mathcal{D}_{\Lambda+\mathbf{a},\Sigma,\mathbf{c}}] = \frac{\rho_{\Sigma,\mathbf{c}}(\mathbf{x})}{\rho_{\Sigma,\mathbf{c}}(\Lambda + \mathbf{a})}$$

where $\rho_{\Sigma,\mathbf{c}}(\Lambda + \mathbf{a}) = \sum_{\mathbf{x}\in\Lambda+\mathbf{a}} \rho_{\Sigma,\mathbf{c}}(\mathbf{x})$.

We collect below several useful lemmas on discrete Gaussian distributions, which will be used in the analysis of the scheme.

Lemma 1 ([AGHS13, Lemma 3]). *For any $\epsilon \in (0, 1)$, a lattice $\Lambda \subseteq \mathcal{R}^m$, $\mathbf{c} \in K_\mathbb{R}^m$, and $\sigma \geq \eta_\epsilon(\Lambda)$, then*

$$\Pr[\|\mathbf{x} - \mathbf{c}\| \geq \sigma\sqrt{md} : \mathbf{x} \leftarrow \mathcal{D}_{\Lambda,\sigma,\mathbf{c}}] \leq \frac{1+\epsilon}{1-\epsilon} \cdot 2^{-md}.$$

Lemma 2 ([GPV08, Lemma 2.6]). *For any full-rank lattice Λ in $\mathbb{R}^m$ and $\varepsilon > 0$,*

$$\eta_\varepsilon(\Lambda) \leq \frac{\sqrt{\log(2m(1 + 1/\varepsilon))/\pi}}{\lambda_1^\infty(\Lambda^*)},$$

where $\lambda_1^\infty(\Lambda^*)$ denotes the ℓ_2 norm of the shortest non-zero vector in the ℓ_∞ norm in the dual lattice Λ^*.

Lemma 3 ([GPV08, Corollary 2.8]). *Let Λ, Λ' be n-dimensional lattices, with $\Lambda' \subseteq \Lambda$. Then for any $\varepsilon \in (0, \frac{1}{2})$, any $s \geq \eta_\varepsilon(\Lambda')$, and any $\mathbf{c} \in \mathbb{R}^n$, the distribution of $\mathcal{D}_{\Lambda,s,\mathbf{c}} \mod \Lambda'$ is within statistical distance at most 2ε of uniform over $\Lambda \mod \Lambda'$.*

Lemma 4 **([DPS23, Lemma 7]).** *Let $m \geq n, \varepsilon \in (0,1), \mathbf{D} \in \mathbb{Z}^{md\times nd}$, $\sigma_{S_1} \geq \eta_\varepsilon(\mathbb{Z}^{nd})$ and $\sigma_S \geq \sqrt{2}\sigma_{\max}(\mathbf{D})\cdot \sigma_{S_1}$. Define*

$$\mathbf{\Sigma}(\mathbf{D}) := \sigma_S^2 \mathbf{I}_{md} - \sigma_{S_1}^2 \mathbf{D}\mathbf{D}^\top,$$

and let $\mathbf{s}_0 \leftarrow \mathcal{D}_{\mathbb{Z}^{md},\mathbf{\Sigma}(\mathbf{D})}$ and $\mathbf{s}_1 \leftarrow \mathcal{D}_{\mathbb{Z}^{nd},\sigma_{S_1}}$, then $\mathbf{\Sigma}(\mathbf{D})$ is positive definite and the distribution $P_{\mathbf{s}}$ of $\mathbf{s} = \mathbf{s}_0 + \mathbf{D}\mathbf{s}_1$ satisfies

$$R_\infty(P_{\mathbf{s}} \| \mathcal{D}_{\mathbb{Z}^{md},\sigma_S}) \leq \frac{1+\varepsilon}{1-\varepsilon}, \quad \Delta(P_{\mathbf{s}}, \mathcal{D}_{\mathbb{Z}^{md},\sigma_S}) \leq \frac{2\varepsilon}{1-\varepsilon}.$$

Lemma 5 **([TT15, Lemma 5]).** *For any m-dimensional lattice $\Lambda \subseteq \mathcal{R}^m$, $\sigma \in \mathbb{R}$ and $\mathbf{c}, \mathbf{c}' \in \Lambda$, then*

$$R_\alpha(\mathcal{D}_{\Lambda,\sigma,\mathbf{c}} \| \mathcal{D}_{\Lambda,\sigma,\mathbf{c}'}) \leq \exp\Big(\frac{\alpha\pi\|\mathbf{c}-\mathbf{c}'\|^2}{\sigma^2}\Big)$$

If $\mathbf{c}, \mathbf{c}' \in K_{\mathbb{R}}^m$, $\sigma \geq \eta_\varepsilon(\Lambda)$, then

$$R_\alpha(\mathcal{D}_{\Lambda,\sigma,\mathbf{c}} \| \mathcal{D}_{\Lambda,\sigma,\mathbf{c}'}) \leq \Big(\frac{1+\varepsilon}{1-\varepsilon}\Big)^{\frac{\alpha}{\alpha-1}} \exp\Big(\frac{\alpha\pi\|\mathbf{c}-\mathbf{c}'\|^2}{\sigma^2}\Big)$$

The following lemma states that if a vector $\mathbf{v} \in \mathcal{R}_q$ is short enough, it must have an inverse.

Lemma 6 **([LS18, Corollary 1.2]).** *Let $d \geq h > 1$ be powers of 2 and $q = 2h + 1 \bmod 4h$ be a prime. Then any $y \in \mathbb{Z}_q[X]/(X^d+1)$ that satisfies either $0 < \|y\|_\infty < \frac{1}{\sqrt{h}} \cdot q^{1/h}$ or $0 < \|y\| < q^{1/h}$ has an inverse in $\mathbb{Z}_q[X]/(X^d+1)$.*

Lemma 7 **([BTT22, Lemma 2.5]).** *Let $\mathbf{B}$ be a uniformly random matrix in $\mathcal{R}_q^{n\times m}$, $n < m$. Then for any $\epsilon > 0$, except with probability at most 2^{-d} on the choice of $\mathbf{B}$, we have $\eta_\epsilon(\Lambda_q^\perp(\mathbf{B})) \leq \frac{8}{\sqrt{\pi}} q^{\frac{n}{m}} \sqrt{d\log(2md + 2md/\epsilon)}$.*

We adopt the notation $P \overset{\varepsilon}{\approx} Q$ from [GMPW20]: for any two distributions P, Q with the same support and $\varepsilon > 0$, we say that $P \overset{\varepsilon}{\approx} Q$ if and only if $\max_{x\in\mathrm{Supp}(P)} |\log P(x) - \log Q(x)| \leq \log(1+\varepsilon)$, or equivalently, $\max\big(R_\infty(P\|Q), R_\infty(Q\|P)\big) \leq 1+\varepsilon$. Note that if $P \overset{\varepsilon}{\approx} Q$, then the statistical distance between P and Q is bounded by $\varepsilon/2$, i.e., $\frac{1}{2}\sum_{x\in\mathrm{Supp}(P)} |P(x) - Q(x)| \leq \frac{\varepsilon}{2}$.

Lemma 8 **(Adapted from [GMPW20, Lemma 2.6]).** *For any $\varepsilon \in (0,1)$ defining $\varepsilon' = 2\varepsilon/(1-\varepsilon)$, $\sigma > 0$, lattice coset $\Lambda + \mathbf{a} \subseteq K_{\mathbb{R}}^m$, and matrix $\mathbf{T} \in K^{k\times m}$ such that $\phi_{\mathbf{M}}(\mathbf{T})$ has full-row rank, $\ker(\mathbf{T})$ is a Λ-subspace and $\eta_\varepsilon(\Lambda \cap \ker(\mathbf{T})) \leq \sigma$, we have*

$$\mathbf{T}\cdot\mathcal{D}_{\Lambda+\mathbf{a},\sigma} \overset{\varepsilon'}{\approx} \mathcal{D}_{\mathbf{T}\Lambda+\mathbf{T}\mathbf{a},\sigma^2\mathbf{T}\mathbf{T}^\dagger}.$$

Lemma 9 **(Adapted from [CATZ24, Lemma 11]).** *For any constant $\varepsilon \in (0,1)$, $\sigma_r > 0$, $1 \leq k < t$, full-rank $\mathcal{R}$-lattice $\Lambda \subseteq \mathcal{R}^m$ with $2\eta_\varepsilon(\Lambda) \leq \sigma_r$, let $\mathbf{r}_1, \ldots \mathbf{r}_t$ be independent samples with $\mathbf{r}_i \leftarrow \mathcal{D}_{\Lambda,\sigma_r}$ and $\mathbf{P} = \begin{pmatrix} \mathbf{I}_k & \mathbf{0} \\ \multicolumn{2}{c}{1,1,\ldots,1} \end{pmatrix} \in \mathcal{R}^{(k+1)\times t}$ and $\begin{pmatrix}\mathbf{y}_1\\ \mathbf{x}\end{pmatrix} := \begin{pmatrix}\mathbf{y}_1\\ \vdots \\ \mathbf{y}_{k+1}\end{pmatrix} = (\mathbf{P}\otimes\mathbf{I}_m)\begin{pmatrix}\mathbf{r}_1\\ \vdots\\ \mathbf{r}_t\end{pmatrix}$. Denote the joint distribution of $\begin{pmatrix}\mathbf{y}_1\\ \mathbf{x}\end{pmatrix}$ as $\mathcal{D}$. Then,*

$$\mathcal{D} \overset{\varepsilon''}{\approx} \mathcal{D}_{(\mathbf{P}\otimes\mathbf{I}_m)\Lambda^t, \Sigma\otimes\mathbf{I}_m},$$

where $\varepsilon'' = \frac{2((1+\varepsilon)^{t-k-1}-1)}{2-(1+\varepsilon)^{t-k-1}}$, Λ^t is the t-th direct sum of Λ, which is a tm-dimensional lattice over $\mathcal{R}$, and $\Sigma = \sigma_r^2\mathbf{P}\mathbf{P}^\dagger \in K_{\mathbb{R}}^{(k+1)\times(k+1)}$ is invertible and positive definite.

Moreover, denote $\mathcal{D}_1$ as the marginal distribution of $\mathbf{x}$ and $\mathcal{D}_{2|\mathbf{x}_0}$ as the distribution of $\mathbf{y}_1$ conditioning on $\mathbf{x} = \mathbf{x}_0$ for any $\mathbf{x}_0 \in \Lambda^k$, and we have

$$\mathcal{D}_1 \overset{\varepsilon''}{\approx} \mathcal{D}_{\Lambda^k,\sigma_r^2\Sigma_{22}}, \quad \mathcal{D}_{2|\mathbf{x}_0} \overset{\varepsilon''}{\approx} \mathcal{D}_{\Lambda, V_{11}^{-1}\cdot\mathbf{I}_m, -V_{11}^{-1}\cdot(\mathbf{V}_{21}\otimes\mathbf{I}_m)\cdot\mathbf{x}_0},$$

where $\Sigma = \begin{pmatrix}\sigma_r^2 & \sigma_r^2\mathbf{e}_k^\top\\ \sigma_r^2\mathbf{e}_k & \sigma_r^2\Sigma_{22}\end{pmatrix}$, $\Sigma^{-1} := \begin{pmatrix}V_{11} & \mathbf{V}_{21}\\ \mathbf{V}_{12} & \mathbf{V}_{22}\end{pmatrix}$, $V_{11} = \frac{1}{\sigma_r^2}\cdot(1+\frac{1}{t-k})$, $\mathbf{V}_{21} = \frac{1}{\sigma_r^2}\cdot(\frac{1}{t-k},\ldots,\frac{1}{t-k},-\frac{1}{t-k})$.

Game $\mathrm{MSIS}^{\mathcal{A}}_{q,d,k,m,\beta}$:	Game $\mathrm{HNF\text{-}MSIS}^{\mathcal{A}}_{q,d,k,m,\beta}$:
$\mathbf{A} \leftarrow \mathcal{R}_q^{k\times m}$ $\mathbf{x} \leftarrow \mathcal{A}(\mathbf{A})$ Return $(\|\mathbf{x}\| \leq \beta \wedge \mathbf{Ax} = \mathbf{0})$	$\mathbf{A}' \leftarrow \mathcal{R}_q^{k\times(m-k)}, \mathbf{A} := [\mathbf{A}' \mid \mathbf{I}_k]$ $\mathbf{x} \leftarrow \mathcal{A}(\mathbf{A})$ Return $(\|\mathbf{x}\| \leq \beta \wedge \mathbf{Ax} = \mathbf{0})$
Game $\mathrm{MLWE}^{\mathcal{A}}_{q,d,k,m,\beta}$:	Game $\mathrm{HNF\text{-}MLWE}^{\mathcal{A}}_{q,d,n+k,n+2k,\beta}$:
$\mathbf{A}' \leftarrow \mathcal{R}_q^{k\times(m-k)}, \mathbf{A} := [\mathbf{A}' \mid \mathbf{I}_k]$ $\mathbf{s} \leftarrow \mathscr{B}_\beta^m; \mathbf{t}_0 \leftarrow \mathbf{As}; \mathbf{t}_1 \leftarrow \mathcal{R}_q^k$ $b \leftarrow \{0,1\}$ $\bar{b} \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t}_b)$ Return $(\bar{b} = b)$	$\mathbf{A}' \leftarrow \mathcal{R}_q^{n\times k}, \mathbf{A} := \begin{bmatrix}\mathbf{A}' \\ \mathbf{I}_k\end{bmatrix}$ $\mathbf{s} \leftarrow \mathcal{R}_q^k, \mathbf{e} \leftarrow \mathscr{B}_\beta^{n+k}$ $\mathbf{t}_0 \leftarrow \mathbf{As} + \mathbf{e}; \mathbf{t}_1 \leftarrow \mathcal{R}_q^{n+k}$ $b \leftarrow \{0,1\}$ $\bar{b} \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t}_b)$ Return $(\bar{b} = b)$

Fig. 2. The module (HNF-)SIS, (HNF-)LWE problem, where $\mathcal{R} := \mathbb{Z}[X]/(X^d + 1)$, $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$ and $\mathscr{B}_\beta := \{x \in \mathcal{R} : \|x\|_\infty \leq \beta\}$.

3.4 Assumptions

We recall the module short integer solution (MSIS) problem, the module learning with error (MLWE) problem, and their Hermite normal form versions (defined in Figure 2). Note that the MSIS and MLWE problems can be reduced to their Hermite normal form versions, respectively [Pei15, ACPS09].

Note that the HNF-MLWE problem considered here is not the standard HNF-LWE variant in the literature, where the secret is typically sampled from a small-norm distribution. In our setting, the "HNF" qualifier instead refers to the fact that the matrix $\mathbf{A}$ is given in Hermite normal form, mirroring the corresponding HNF-MSIS assumption.

The advantage of $\mathcal{A}$ for the (HNF-)MSIS problem is defined as

$$\mathrm{Adv}^{\text{(hnf-)msis}}_{q,d,k,m,\beta}(\mathcal{A}) := \Pr\left[\text{(HNF-)MSIS}^{\mathcal{A}}_{q,d,k,m,\beta} = 1\right].$$

The advantage of $\mathcal{A}$ for the (HNF-)MLWE problem are defined as

$$\mathrm{Adv}^{\text{(hnf-)mlwe}}_{q,d,k,m,\beta}(\mathcal{A}) := |\Pr\left[\text{(HNF-)MLWE}^{\mathcal{A}}_{q,d,k,m,\beta} = 1\right] - \frac{1}{2}|.$$

3.5 Rényi Divergence

We define the notion of Rényi Divergence between two distributions P, Q which we will use in the analysis of the scheme. For a discrete distribution P, we denote the support of P as $\mathrm{Supp}(P) := \{x : P(x) > 0\}$.

Definition 1 (Rényi Divergence). *Let P, Q be two discrete probability distributions such that $\mathrm{Supp}(P) \subseteq \mathrm{Supp}(Q)$ and $\alpha \in [1, +\infty]$. We define the Rényi Divergence of order α, for $\alpha \in (1, \infty)$, as $R_\alpha\big(P\|Q\big) := \left(\sum_{x\in\mathrm{Supp}(P)} \frac{P(x)^\alpha}{Q(x)^{\alpha-1}}\right)^{\frac{1}{\alpha-1}}$.*

For $\alpha = 1$ and $\alpha = \infty$, we define $R_1\big(P\|Q\big) := \exp\left(\sum_{x\in\mathrm{Supp}(P)} P(x)\log\frac{P(x)}{Q(x)}\right)$ and $R_\infty\big(P\|Q\big) := \max_{x\in\mathrm{Supp}(P)} \frac{P(x)}{Q(x)}$.

Lemma 10 ([ASY22, Lemma 2.27]). *Let $\alpha \in [1, \infty]$ and P, Q be discrete probability distributions with $\mathrm{Supp}(P) \subseteq \mathrm{Supp}(Q)$. Then, the following properties hold:*

- ***Log Positivity:*** *$R_\alpha(P\|Q) \geq R_\alpha(P\|P) = 1$.*
- ***Data Processing Inequality:*** *$R_\alpha(P^f\|Q^f) \leq R_\alpha(P\|Q)$ for any function f, where P^f (and Q^f) denotes the distribution which samples $x \leftarrow P$ ($x \leftarrow Q$) and outputs $f(x)$.*

– ***Probability Preservation:*** *Let $E \subseteq \mathrm{Supp}(Q)$ be an arbitrary event. Then, for $\alpha \in (1, \infty)$,*

$$\Pr_{x \leftarrow P}[E]^{\alpha/(\alpha-1)} / R_\alpha(P \| Q) \le \Pr_{x \leftarrow Q}[E] \,.$$

– ***Weak Triangle Inequality:*** *Let P_1, P_2, P_3 be three probability distributions where $\mathrm{Supp}(P_1) \subseteq \mathrm{Supp}(P_2) \subseteq \mathrm{Supp}(P_3)$. Then, we have*

$$R_\alpha(P_1 \| P_3) \le \begin{cases} R_\alpha(P_1 \| P_2) \cdot R_\infty(P_2 \| P_3) \\ R_\infty(P_1 \| P_2)^{\frac{\alpha}{\alpha-1}} \cdot R_\alpha(P_2 \| P_3) & \text{if } \alpha \in (1, \infty) \end{cases} .$$

Lemma 11 ([Ros20, Proposition 2]). *Let P and Q denote two distributions of a sequence of random variables $(X_1, \ldots, X_n)$. For $1 \le i \le n$, denote $P_{i|x_{[i-1]}}$ (resp. $Q_{i|x_{[i-1]}}$) as the conditional distribution of X_i given $(X_1, \ldots, X_{i-1}) = (x_1, \ldots, x_{i-1})$. Then, for any $\alpha > 1$,*

$$R_\alpha(P \| Q) \le \prod_{i \in [n]} \max_{x_{[i-1]}} R_\alpha(P_{i|x_{[i-1]}} \| Q_{i|x_{[i-1]}}) \,.$$

3.6 Threshold signatures

We define *two-round threshold signature schemes*. In our syntax, the key generation phase is assumed to be trusted, though in practice it can be implemented via a distributed protocol. The signing protocol consists of three algorithms that each signer executes locally.

In the first stage, each signer produces a secret state along with a protocol message. After exchanging these protocol messages with the other signers, each signer proceeds to the second stage, generating a second protocol message. The final stage is a combining algorithm that takes all previously generated protocol messages as input and outputs the final threshold signature. Notably, this combining algorithm does not require any secret state and can be executed by any designated party.

Definition 2 (two-round threshold signature schemes). *A two-round threshold signature scheme TS consists of the following algorithms:*

– $\mathsf{KeyGen}(1^\kappa) \to (\mathsf{pk}, \{\mathsf{sk}_i\}_{i \in [N]})$*: On input the security parameter 1^κ, the key-generation algorithm KeyGen outputs a public key pk and N secret-key shares $\{\mathsf{sk}_i\}_{i \in [N]}$. Each secret-key share implicitly specifies the number of parties N and the threshold T. All algorithms associated with TS implicitly take pk as input.*
– *The signing protocol consists of three algorithms:*
  - $\mathsf{Sign}_1(i, \mathsf{sk}_i, \mathrm{SS}, \mu) \to (\bar{\mathsf{st}}, psig_i^{(1)})$*: On inputs a signer index i, a set of signer indices SS, a secret key share sk_i, and a message μ, the first-round signing algorithm Sign_1 outputs a state $\bar{\mathsf{st}}$ and a protocol message $psig_i^{(1)}$.*
  - $\mathsf{Sign}_2(i, \mathsf{sk}_i, \bar{\mathsf{st}}, \mathrm{SS}, \mu, M) \to psig_i^{(2)}$*: On inputs a signer index i, a secret key share sk_i, a state $\bar{\mathsf{st}}$, a set of signer indices SS, a message μ, and a set of protocol messages M, the second-round signing algorithm Sign_2 outputs a protocol message $psig_i^{(2)}$.*
  - $\mathsf{Combine}(\mathrm{SS}, \mu, M = \{psig_j^{(1)}\}_{j \in \mathrm{SS}}, M' = \{psig_j^{(2)}\}_{j \in \mathrm{SS}}) \to sig$*: On inputs a set of signer indices SS, a message μ, and two sets of protocol messages M and M', the combining algorithm outputs a signature sig.*
– $\mathsf{Ver}(\mathsf{pk}, \mu, sig) \to 0/1$*: On inputs a public key pk, a message μ, and a signature sig, the verification algorithm Ver outputs 0 or 1.*

An honest execution of the signing protocol is formalized by the procedure TS-COR (defined in Fig 3). We say that TS is correct with correctness error $\varepsilon_{\mathsf{cor}}$ if for $\mu \in \{0,1\}^*$ and $\mathrm{SS} \subseteq [N]$ with $|\mathrm{SS}| \ge T$, we have $\Pr[\text{TS-COR}_{\mathsf{TS}[N,T]}(\kappa, \mu, \mathrm{SS}) = 0] \le \varepsilon_{\mathsf{cor}}(\kappa)$.

Security. We formalize the notion of fully adaptive security for two-round threshold signature schemes. In this setting, the adversary is provided with the public parameters, a public key, and access to oracles that enable it to corrupt signers and obtain signatures. When a signer is corrupted, the adversary learns not only the corresponding secret key share but also the secret state generated during the first round of every ongoing signing session. For each signer, we maintain a counter that is used to generate unique session identifiers, ensuring that all signing sessions can be accurately tracked. The adversary's objective is to produce a valid signature on a message that has never been submitted to the signing oracle. Our security definition models an adversary with fully control over the communication channels. In particular, we do not assume *authenticated channels* and *secure erasure*.

Game $\mathrm{TS\text{-}COR}_{\mathsf{TS}[N,T]}(\kappa, \mu, \mathrm{SS})$:

$(\mathsf{pk}, \{\mathsf{sk}_i\}_{i\in[N]}) \leftarrow \mathsf{KeyGen}(1^\kappa)$
For $i \in \mathrm{SS}$ do $\mathsf{Sign}_1(i, \mathsf{sk}_i, \mathrm{SS}, \mu) \to (\bar{\mathsf{st}}_i, psig_i^{(1)})$
For $i \in \mathrm{SS}$ do $\mathsf{Sign}_2(i, \mathsf{sk}_i, \bar{\mathsf{st}}_i, \mathrm{SS}, \mu, \{psig_i^{(1)}\}_{i\in\mathrm{SS}}) \to psig_i^{(2)}$
$\mathsf{Combine}(\mathrm{SS}, \mu, \{psig_i^{(1)}\}_{i\in\mathrm{SS}}, \{psig_i^{(2)}\}_{i\in\mathrm{SS}}) \to sig$
Return $\mathsf{Ver}(\mathsf{pk}, \mu, sig)$

Fig. 3. The correctness game $\mathsf{TS\text{-}COR}$ for a threshold signature scheme TS for N signers and threshold T.

Game $\mathsf{adp\text{-}TSUF}^{\mathcal{A}}_{\mathsf{TS}[N,T]}(\kappa)$:

$S = \emptyset$; $\mathsf{CS} = \emptyset$; $H \leftarrow \mathsf{TS.HF}$
$(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N) \leftarrow \mathsf{KeyGen}(1^\kappa)$
For $i \in [N]$ do
  $\mathsf{st}_i.\mathsf{cnt} = 0$; $\mathsf{st}_i.\mathsf{P} = ()$
  $\mathsf{st}_i.\mathsf{sk} \leftarrow \mathsf{sk}_i$
$(\mu, sig) \leftarrow \mathcal{A}^{\textsc{Cor}, \textsc{oSign}_1, \textsc{oSign}_2, \mathrm{RO}}(\mathsf{pk})$
If $|\mathsf{CS}| \geq T$ then return 0
Return $(\mu \notin S \ \wedge \ \mathsf{Ver}(\mathsf{pk}, \mu, sig) = 1)$

Oracle $\textsc{Cor}(i)$:

Require: $|\mathsf{CS}| < T - 1$
$\mathsf{CS} \leftarrow \mathsf{CS} \cup \{i\}$
Return st_i

Oracle $\mathrm{RO}(x)$:

Return $H(x)$

Oracle $\textsc{oSign}_1(i, \mathrm{SS}, \mu)$:

Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$
$\mathsf{st}_i.\mathsf{cnt} \leftarrow \mathsf{st}_i.\mathsf{cnt} + 1$; $S \leftarrow S \cup \{\mu\}$
$(psig^{(1)}, \bar{\mathsf{st}}) \leftarrow \mathsf{Sign}_1(i, \mathsf{sk}_i, \mathrm{SS}, \mu)$
$\mathsf{st}_i.\mathsf{P}(\mathsf{st}_i.\mathsf{cnt}) \leftarrow (0, \bar{\mathsf{st}})$
Return $psig^{(1)}$

Oracle $\textsc{oSign}_2(i, \mathrm{sid}, \mathrm{SS}, \mu, M)$:

Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{sid} \leq \mathsf{st}_i.\mathsf{cnt}$, $\mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$
$(\mathsf{isUsed}, \bar{\mathsf{st}}) = \mathsf{st}_i.\mathsf{P}(\mathrm{sid})$
If isUsed then return $\perp$
$psig^{(2)} \leftarrow \mathsf{Sign}_2(i, \mathsf{sk}_i, \bar{\mathsf{st}}, \mathrm{SS}, \mu, M)$
$\mathsf{st}_i.\mathsf{P}(\mathrm{sid}) = (1, \bar{\mathsf{st}})$
Return $psig^{(2)}$

Fig. 4. The adaptive unforgeability game $\mathsf{adp\text{-}TSUF}$ for a threshold signature scheme TS with N signers and threshold T. Here $\mathsf{TS.HF}$ denotes the family of the hash functions from which the random oracle is sampled.

Definition 3 (Adaptive Security of Threshold Signature Schemes). *A threshold signature scheme* TS *with random oracle is* $(Q_s, Q_h, \tau, \epsilon)$*-adaptively secure if for all adversaries* $\mathcal{A}$ *that makes at most* $Q_s = Q_s(\kappa)$ *signing queries and* $Q_h = Q_h(\kappa)$ *random oracle queries is of running time at most* $\tau = \tau(\kappa)$*, for all* N *and* T *that the scheme supports,* $\Pr\left[\mathsf{adp\text{-}TSUF}^{\mathcal{A}}_{\mathsf{TS}[N,T]}(\kappa) = 1\right] \leq \epsilon(\kappa)$*, where the security game* $\mathsf{adp\text{-}TSUF}_{\mathsf{TS}[N,T]}$ *is defined in Fig. 4.*

4 Single-Signer Scheme

In this section, we present the single-signer version of our threshold signature scheme, i.e. $N = T = 1$. The protocol is shown in Figure 5. We focus on proving the correctness and security of this *Single-Signer Scheme*. More specifically, we reduce the unforgeability of the scheme to MSIS and MLWE (see the proof strategy in Section 2). This scheme can be viewed as a two-round interactive single-signer signature scheme. The primary motivation for adopting an interactive setting is to simplify the security proof of the threshold scheme presented in the next section (as outlined in Section 2).

The following theorem establishes the correctness of the scheme in Figure 5.

Theorem 1 (Correctness of single signer scheme). *If* $\sigma_1, \sigma_2 \geq \eta_\varepsilon(\mathcal{R}_q^t)$*,* $\sigma_r, \sigma_S \geq \eta_\varepsilon(\mathcal{R}_q^m)$*, and* $L_1 \geq \sqrt{md}(\sigma_r + \beta_c k \sigma_S)$*,* $L_2 \geq \sqrt{td}(\sigma_2 + \beta_c k \sigma_1)$*, then the scheme in Fig. 5 is correct with correctness error* $\varepsilon_{\mathsf{cor}} \leq 4(k+1) \cdot 2^{-2\kappa}$*.*

Proof. Firstly, $\mathbf{AS} = \mathbf{P}$ holds for any matrix pair output by KeyGen.

For an honestly generated transcript $(\mathbf{Y}, \mathbf{w}, \mathbf{z})$ and $\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$, we have:

– $\mathbf{Az} = \mathbf{A}(\mathbf{r} + \mathbf{Sc}) = \overline{\mathbf{w}} + \mathbf{Pc}$

$\underline{\mathsf{KeyGen}(1^\kappa):}$

$\mathbf{A}' \leftarrow \mathcal{R}_q^{\ell\times(m-\ell)}$, $\mathbf{A} := [\mathbf{A}' \mid \mathbf{I}_\ell]$
$\mathbf{S} \leftarrow \mathcal{D}_{\mathcal{R}^{m\times k},\sigma_S}$, $\mathbf{P} = \mathbf{AS}$
Return $(\mathsf{pk} = (\mathbf{A}, \mathbf{P}), \mathsf{sk} = \mathbf{S})$

$\underline{\mathsf{Sign}_1(1, \mathsf{sk}, \{1\}, \mu):}$

$\mathbf{S} = \mathsf{sk}$
$\mathbf{E}' \leftarrow \mathcal{D}_{\mathcal{R}^{t\times k},\sigma_1}$; $\mathbf{r} \leftarrow \mathcal{D}_{\mathcal{R}^m,\sigma_r}$
$\mathbf{e} \leftarrow \mathcal{D}_{\mathcal{R}^t,\sigma_2}$
$\mathbf{U} = H_1(\mu) \in \mathcal{R}_q^{t\times m}$
$\mathbf{Y} = \mathbf{US} + \mathbf{E}'$
$\mathbf{w} = \begin{bmatrix}\overline{\mathbf{w}}\\ \underline{\mathbf{w}}\end{bmatrix} = \begin{bmatrix}\mathbf{Ar}\\ \mathbf{Ur}+\mathbf{e}\end{bmatrix}$
$\mathsf{st} = \mathbf{r}$
Return $(\mathbf{Y}, \mathbf{w}), \mathsf{st}$

$\underline{\mathsf{Sign}_2(1, \mathsf{sk}, \mathsf{st}, \{1\}, \mu, M):}$

$(\mathbf{Y}, \mathbf{w}) = M$, $\mathbf{S} = \mathsf{sk}$, $\mathbf{r} = \mathsf{st}$
$\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w}) \in \mathcal{C}^k \subset \mathcal{R}^k$
$\mathbf{z} = \mathbf{r} + \mathbf{Sc}$
Return $\mathbf{z}$

$\underline{\mathsf{Ver}(\mathsf{pk}, \mu, (\mathbf{Y}, \mathbf{w}, \mathbf{z})))}$

$\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$
Return $(\|\mathbf{z}\| \leq L_1)$
$\quad \wedge (\mathbf{Az} = \overline{\mathbf{w}} + \mathbf{Pc})$
$\quad \wedge (\|\mathbf{Uz} - \underline{\mathbf{w}} - \mathbf{Yc}\| \leq L_2)$.

Fig. 5. Lattice-based 1-out-of-1 threshold signatures $\mathsf{Tweed}[1,1]$. Here, $H_i(\cdot)$ denotes $H(i,\cdot)$. Also, $H_1 : \{0,1\}^* \to \mathcal{R}_q^{t\times m}$, and $H_2 : \{0,1\}^* \to \mathcal{C}^k$. The public key pk is implicitly given to all algorithms except KeyGen.

- $\|\mathbf{Uz} - \underline{\mathbf{w}} - \mathbf{Yc}\| = \|-(\mathbf{e} + \mathbf{E}'\mathbf{c})\| \leq \|\mathbf{e}\| + \|\mathbf{E}'\|_{1,2}\|\mathbf{c}\|_1$
- $\|\mathbf{z}\| = \|\mathbf{r} + \mathbf{Sc}\| \leq \|\mathbf{r}\| + \|\mathbf{S}\|_{1,2}\|\mathbf{c}\|_1$

where $\|\mathbf{c}\|_1 = \beta_c \cdot k$ (cf. Section 5.3) and $\|\mathbf{S}\|_{1,2} := \max_{1\leq j\leq k} \|\mathbf{S}_{:j}\|$, $\mathbf{S}_{:j}$ is the j-th column of $\mathbf{S}$.

By Lemma 2, we set

- $\eta_\varepsilon(\mathcal{R}^t) \leq \sqrt{\log(2td(1+1/\varepsilon))/\pi} \leq \sigma_a, a \in \{1,2\}$.
- $\eta_\varepsilon(\mathcal{R}^m) \leq \sqrt{\log(2md(1+1/\varepsilon))/\pi} \leq \sigma_b, b \in \{r, S\}$.

Therefore, by Lemma 1 and union bound, we have

- $\|\mathbf{e}\| \geq \sigma_2\sqrt{td}$ with probability at most $\frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-td}$.
- $\|\mathbf{E}'\|_{1,2} \geq \sigma_1\sqrt{td}$ with probability at most $k \cdot \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-td}$.
- $\|\mathbf{r}\| \geq \sigma_r\sqrt{md}$ with probability at most $\frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md}$.
- $\|\mathbf{S}\|_{1,2} \geq \sigma_S\sqrt{md}$ with probability at most $k \cdot \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md}$.

Therefore, we have that

- $\|\mathbf{Uz} - \underline{\mathbf{w}} - \mathbf{Yc}\| \leq \sqrt{td}(\sigma_2 + \beta_c k\sigma_1) \leq L_2$
- $\|\mathbf{z}\| \leq \sqrt{md}(\sigma_r + \beta_c k\sigma_S) \leq L_1$

except with error probability $\varepsilon_{\mathsf{cor}} = (k+1) \cdot \frac{1+\varepsilon}{1-\varepsilon} \cdot (2^{-td} + 2^{-md}) \leq 4(k+1) \cdot 2^{-2\kappa}$, where we used $d \geq 2\kappa$, $\varepsilon \leq 2^{-2\kappa}$, and $\frac{1+\varepsilon}{1-\varepsilon} \leq 1 + 4 \cdot 2^{-2\kappa} \leq 2$ □

4.1 Security

The following theorem establishes the unforgeability of the single-signer signature scheme Fig. 5 under the MLWE and MSIS assumptions in the ROM.

Theorem 2 (Security of single signer scheme). *For any integers $q = q(\kappa), m = m(\kappa), t = t(\kappa), n = n(\kappa), k = k(\kappa), \ell = \ell(\kappa)$ and any adversary $\mathcal{A}$ making at most $Q_s = Q_s(\kappa)$ queries to the* Sign_1 *oracle and $Q_h = Q_h(\kappa)$ queries to the Random Oracle, then either there exists an* HNF-MSIS *adversary $\mathcal{A}'$ running in time roughly two times that of $\mathcal{A}$ such that,*

$$\mathsf{Adv}^{\mathsf{adp\text{-}tsuf}}_{\mathsf{Tweed}[1,1]}(\mathcal{A},\kappa) \leq 32^{\frac{1}{3}} Q_h^2 \cdot (\mathsf{Adv}^{\mathsf{hnf\text{-}msis}}_{q,d,\ell,m,4L_1}(\mathcal{A}',\kappa) + 4(k+1)\cdot 2^{-6\kappa})^{\frac{1}{3}} ,$$

or there exist an HNF-MLWE *adversary $\mathcal{B}$ and an* MLWE *adversary $\mathcal{C}$ running in time roughly that of $\mathcal{A}$ plus* $\mathsf{poly}(\kappa, 1/\epsilon)$ *such that*

$$\mathsf{Adv}^{\mathsf{adp\text{-}tsuf}}_{\mathsf{Tweed}[1,1]}(\mathcal{A},\kappa) \le \max\{2^{-\kappa}\ ,8Q_h^2\Big(\delta_2\cdot 2^{-2\kappa}+\delta_3\cdot 2^{-3\kappa}+\delta_4\cdot 2^{-4\kappa} \\ +t\cdot Q_h\cdot \mathsf{Adv}^{\mathsf{hnf\text{-}mlwe}}_{q,d,m,\ell+m,\beta_{\mathsf{hlwe}}}(\mathcal{B},\kappa) \\ +(m-\ell-n)\cdot \mathsf{Adv}^{\mathsf{mlwe}}_{q,d,\ell,n+\ell,\beta_{\mathsf{lwe}}}(\mathcal{C},\kappa)\Big)\}\ ,$$

where $\delta_2 = e^{6\pi}\sqrt{k}+1$, $\delta_3 = e^{9\pi}\sqrt{48\kappa}\cdot 2Q_h^2$, $\delta_4 = e^{9\pi}\sqrt{2k}+k+Q_s$.

Proof. We make the following simplifying assumptions about $\mathcal{A}$ WLOG:

- whenever $\mathcal{A}$ makes a query to oSign$_1$ or oSign$_2$, it first makes queries H_1 or H_2 on the value that would be queried in the respective algorithm (and whenever the challenger makes a query during oSign$_1$, oSign$_2$, we answer consistently);
- $\mathcal{A}$ never repeats a query to H_1 or to H_2;
- $\mathcal{A}$ always queries H_1 on μ^* and H_2 on $(\mu^*,\mathbf{Y}^*,\mathbf{w}^*)$ corresponding to the output signature $(\mu^*,\mathbf{Y}^*,\mathbf{w}^*,\mathbf{z}^*)$ (if it doesn't, just make an extra query).

Game sequence. We begin with the game sequence. We write Adv_x to denote the advantage in G_x. Every game starting with G_2 takes as input an additional parameter $\epsilon \le \mathsf{Adv}_1 \le 2\epsilon$ which we specify later.

$\mathbf{G}_0$: Same as the original game in Fig 4. We use $\mathbf{z}_j$ to denote the j-th query to oSign$_2$, and $(\mu^*, sig^* = (\mathbf{Y}^*,\mathbf{w}^*,\mathbf{z}^*))$ to denote the message-signature pair output by the adversary.

$\mathbf{G}_1$: Same as $\mathbf{G}_0$, except we sample $j_1, j_2 \leftarrow [Q_h]$ at the beginning of the game, abort at the end if the j_1-th query to H_1 is not μ^*, or the j_2-th query to H_2 is not $(\mu^*,\mathbf{Y}^*,\mathbf{w}^*)$. We have $\mathsf{Adv}_1 = \frac{1}{Q_h^2}\cdot\mathsf{Adv}_0$.

$\mathbf{G}_2$: Same as $\mathbf{G}_1$, except we add one more verification condition for the signature sig^* on μ^*: sample $\lceil 12\kappa/\epsilon^2\rceil$ random pairs $(\mathbf{c}_1,\mathbf{c}_2)$, and reject if none of them satisfies $\|(\mathbf{Y}^*-\mathbf{U}^*\mathbf{S})(\mathbf{c}_1-\mathbf{c}_2) \bmod q\| \le 2L_2$, where $\mathbf{U}^* = H_1(\mu^*)$. We relate Adv_1 to Adv_2 in Lemma 12.

$\mathbf{G}_3$: For $j \ne j_1$, replace $\mathbf{U}_j \leftarrow \mathcal{R}_q^{t\times m}$ in the response to the j-th query to H_1 with $\mathbf{T}_j\mathbf{A}+\mathbf{E}''_j$, where $\mathbf{T}_j \leftarrow \mathcal{R}_q^{t\times\ell}, \mathbf{E}''_j \leftarrow \mathscr{B}^{t\times m}_{\beta_{\mathsf{hlwe}}}$. By the HNF-MLWE assumption (the corresponding game is defined in Figure 2), for $\mathbf{t}\leftarrow\mathcal{R}_q^{\ell}$ and $\mathbf{e}\leftarrow\mathscr{B}^m$, we know $\mathbf{A}^\top\mathbf{t}+\mathbf{e}$ is computationally indistinguishable from a uniformly random vector sampled from $\mathcal{R}_q^m$. Therefore, $|\mathsf{Adv}_2-\mathsf{Adv}_3| \le t\cdot Q_h\cdot\mathsf{Adv}^{\mathsf{hnf\text{-}mlwe}}_{q,d,m,\ell+m,\beta_{\mathsf{hlwe}}}(\mathcal{B},\kappa)$.

$\mathbf{G}_4$: Same as $\mathbf{G}_3$, except $\mathbf{A} = [\mathbf{B}_0 \mid \mathbf{B}_0\mathbf{R}+\mathbf{E} \mid \mathbf{I}_\ell]$ where $\mathbf{B}_0 \leftarrow \mathcal{R}_q^{\ell\times n}, \mathbf{R}\leftarrow\mathscr{B}^{n\times(m-\ell-n)}_{\beta_{\mathsf{lwe}}}, \mathbf{E}\leftarrow\mathscr{B}^{\ell\times(m-\ell-n)}_{\beta_{\mathsf{lwe}}}$.

In addition, we write $\mathbf{D} = \begin{bmatrix} -\mathbf{R} \\ \mathbf{I}_{m-\ell-n} \\ -\mathbf{E}\end{bmatrix}$ so that $\mathbf{A}\cdot\mathbf{D} = \mathbf{0}$.

From MLWE, we have $|\mathsf{Adv}_3-\mathsf{Adv}_4| \le (m-\ell-n)\cdot\mathsf{Adv}^{\mathsf{mlwe}}_{q,d,\ell,n+\ell,\beta_{\mathsf{lwe}}}(\mathcal{C},\kappa)$.

$\mathbf{G}_5$: Same as $\mathbf{G}_4$, except we replace

$$\mathbf{S} \approx_s \mathbf{S}^{(0)}+\mathbf{D}\cdot\mathbf{S}^{(1)},\ \mathbf{r}_j \approx_s \mathbf{r}_j^{(0)}+\mathbf{D}\cdot\mathbf{r}_j^{(1)}$$

where $\mathbf{S}^{(0)} \leftarrow (\mathcal{D}_{\mathcal{R}^m,\mathbf{\Sigma}_S})^k, \mathbf{S}^{(1)}\leftarrow(\mathcal{D}_{\mathcal{R}^{m-\ell-n},\sigma_{S_1}})^k, \mathbf{r}_j^{(0)}\leftarrow\mathcal{D}_{\mathcal{R}^m,\mathbf{\Sigma}_r}, \mathbf{r}_j^{(1)}\leftarrow\mathcal{D}_{\mathcal{R}^{m-\ell-n},\sigma_{r_1}}, j\in[Q_s]$, here $\mathbf{r}_j$ is the st used in the j-th query to oSign$_1$, and $\mathbf{\Sigma}_S, \mathbf{\Sigma}_r$ is defined in Lemma 13.

We relate Adv_4 to Adv_5 in Lemma 13.

$\mathbf{G}_6$: Same as $\mathbf{G}_5$, except that we modify the way in which the outputs of oSign$_1$ are generated. For the j-th query to oSign$_1$, which takes μ as input and for which $H_1(\mu)$ is the j'-th query to H_1, we replace $(\mathbf{Y}_j, \underline{\mathbf{w}}_j)$ with

$$\big((\mathbf{T}_{j'}\mathbf{A}+\mathbf{E}''_{j'})\mathbf{S}^{(0)}+\mathbf{E}'_j,\ (\mathbf{T}_{j'}\mathbf{A}+\mathbf{E}''_{j'})\mathbf{r}_j^{(0)}+\mathbf{e}_j\big),$$

where $\mathbf{T}_{j'}\mathbf{A}+\mathbf{E}''_{j'} = H_1(\mu)$. We relate Adv_5 to Adv_6 in Lemma 14.

$\mathbf{G}_7$: Same as $\mathbf{G}_6$, except that we modify the way in which the outputs of oSign$_2$ are generated. For the j-th query to oSign$_2$, which takes $(\mu,\mathbf{Y},\mathbf{w})$ as input, replace $\mathbf{z}_j$ with $\mathbf{r}+\mathbf{S}^{(0)}\mathbf{c}$ here $\mathbf{r}$ is the st used by Sign_1 to generate the $(\mathbf{Y},\mathbf{w})$, and $\mathbf{c} = H_2(\mu,\mathbf{Y},\mathbf{w})$, we relate Adv_6 to Adv_7 in Lemma 14.

$\mathbf{G}_8$: Same as $\mathbf{G}_7$, except we replace $\mathbf{U}^*\mathbf{S}$ in the extra verification check (added in $\mathbf{G}_2$) with a uniformly random matrix $\mathbf{V}\leftarrow\mathcal{R}_q^{t\times k}$.

At this point, the public key and the signatures only leak $\mathbf{S}^{(0)}$.

By Lemma 7, except with probability at most 2^{-d}, we have

$$8q^{\frac{t}{m-\ell-n}} \cdot \sqrt{\frac{d\log(2(m-\ell-n)d(1+1/\varepsilon'))}{\pi}} \geq \eta_{\varepsilon'}\left(\Lambda_q^{\perp}(\mathbf{U}^*\mathbf{D})\right).$$

Note that $\mathbf{U}^*\mathbf{D} \in \mathcal{R}_q^{t\times(m-\ell-n)}$ is uniformly distributed, and we set $\sigma_{S_1} \geq \eta_{\varepsilon'}\left(\Lambda_q^{\perp}(\mathbf{U}^*\mathbf{D})\right)$, $\varepsilon' = 2^{-8\kappa}$. Thus, $\mathbf{U}^*(\mathbf{S}^{(0)} + \mathbf{D}\mathbf{S}^{(1)}) = \mathbf{U}^*\mathbf{S}^{(0)} + (\mathbf{U}^*\mathbf{D})\mathbf{S}^{(1)}$, which is statistically close to a uniformly random matrix by Lemma 3. We have $|\mathsf{Adv}_7 - \mathsf{Adv}_8| \leq 2k \cdot 2^{-8\kappa}$.

Finally: We bound Adv_8 in Lemma 15.

Now we derive the final bounds in Theorem 2. From $\mathbf{G}_1$ to $\mathbf{G}_5$, we have

$$\mathsf{Adv}_0 \leq 32^{\frac{1}{3}} Q_h^2 \cdot \left(\mathsf{Adv}^{\mathsf{hnf\text{-}msis}}_{q,d,\ell,m,4L_1}(\mathcal{A}',\kappa) + 4(k+1)\cdot 2^{-6\kappa}\right)^{\frac{1}{3}},$$

or,

$$\mathsf{Adv}_0 \leq 8Q_h^2\Big(\mathsf{Adv}_5 + t\cdot Q_h \cdot \mathsf{Adv}^{\mathsf{hnf\text{-}mlwe}}_{q,d,m,\ell+m,\beta_{\mathsf{hlwe}}}(\mathcal{B},\kappa) + (m-\ell-n)\cdot \mathsf{Adv}^{\mathsf{mlwe}}_{q,d,\ell,n+\ell,\beta_{\mathsf{lwe}}}(\mathcal{C},\kappa) + 2^{-2\kappa}\Big).$$

Moreover, from $\mathbf{G}_5$ to $\mathbf{G}_8$ we obtain

$$\begin{aligned}
\mathsf{Adv}_5 &\leq e^{2(\alpha-1)\pi}\mathsf{Adv}_6^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa},\\
\mathsf{Adv}_6 &\leq e^{(\alpha-1)\pi}\mathsf{Adv}_7^{\frac{\alpha-1}{\alpha}} + k\cdot 2^{-4\kappa},\\
\mathsf{Adv}_7 &\leq \mathsf{Adv}_8 + 2k\cdot 2^{-8\kappa},\\
\mathsf{Adv}_8 &\leq \frac{48\kappa}{\epsilon^2}\cdot 2^{-8\kappa},
\end{aligned}$$

where $\epsilon \leq Q_h^{-2}\cdot \mathsf{Adv}_0 \leq 2\epsilon$. Setting $\alpha = 4$, we have

$$\mathsf{Adv}_5 \leq e^{9\pi}\sqrt{2k}\cdot 2^{-4\kappa} + e^{9\pi}\sqrt{48\kappa}\cdot \frac{2Q_h^2}{\mathsf{Adv}_0}\cdot 2^{-4\kappa} + e^{6\pi}\sqrt{k}\cdot 2^{-2\kappa} + (k+Q_s)\cdot 2^{-4\kappa}.$$

In the above derivation, we use the following elementary facts: for any $0 < x \leq 1$ and $0 < a_1 < a_2$, we have $x^{a_2} \leq x^{a_1}$; moreover, for any $a_1, a_2 \geq 0$ and $0 < x \leq 1$, we have $(a_1+a_2)^x \leq a_1^x + a_2^x$.
Finally, if $\mathsf{Adv}_0 > 2^{-k}$, then $\frac{1}{\mathsf{Adv}_0} \leq 2^{\kappa}$. Substituting this bound into the expression for Adv_5 gives the claimed bound. ☐

Lemma 12. *Suppose there exists an adversary $\mathcal{A}$ that wins $\mathbf{G}_1$ with probability ϵ. Then, either there exists an* HNF-MSIS *adversary $\mathcal{A}'$ running in time roughly two times that of $\mathcal{A}$ plus* $\mathsf{poly}(\kappa, 1/\epsilon)$ *such that*

$$\mathsf{Adv}_1(\mathcal{A},\kappa) \leq \sqrt[3]{32\cdot \mathsf{Adv}^{\mathsf{hnf\text{-}msis}}_{q,d,\ell,m,4L_1}(\mathcal{A}',\kappa) + (k+1)\cdot 2^{-6\kappa+7}},$$

or $\mathsf{Adv}_1(\mathcal{A},\kappa) \leq 4\cdot \mathsf{Adv}_2(\mathcal{A},\kappa) + 2^{-2\kappa+2}$.

Step 0. First, we write the randomness in $\mathbf{G}_1$ explicitly as $(j_2, \tau^{(1)}, \mathbf{c}, \tau^{(2)})$ where

- $\tau^{(1)}$ corresponds to (i) the adversary's random tape, (ii) $\mathbf{A}, \mathbf{S}$ as sampled during KeyGen, (iii) the random index j_1, (iv) all the randomness used for answering queries to H_1, H_2 and for signing queries up to the j_2'th query to H_2;
- $\mathbf{c}$ denotes $H_2(\mu^*, \mathbf{Y}^*, \mathbf{w}^*)$;
- $\tau^{(2)}$ corresponds to the remaining randomness.

In particular,

- Given $j_2, \tau^{(1)}$, we can compute $\mu^*, \mathbf{Y}^*, \mathbf{w}^*$;
- Given $j_2, \tau^{(1)}, \mathbf{c}, \tau^{(2)}$, we can additionally compute $\mathbf{z}^*$.

We write $(\mu^*, \mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*) \leftarrow \mathbf{G}_1^{\mathcal{A}}(j_2, \tau^{(1)}, \mathbf{c}, \tau^{(2)})$ to denote the message-signature pair output by $\mathcal{A}$ in $\mathbf{G}_1$ with randomness $(j_2, \tau^{(1)}, \mathbf{c}, \tau^{(2)})$.

By our assumption on $\mathcal{A}$, we have:

$$\Pr_{j_2,\tau^{(1)},\mathbf{c},\tau^{(2)}}\left[\begin{array}{c}\mathsf{Ver}(\mathsf{pk}, \mu^*, (\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*)) = 1 : \\ (\mu^*, \mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*) \leftarrow \mathbf{G}_1^{\mathcal{A}}(j_2, \tau^{(1)}, \mathbf{c}, \tau^{(2)})\end{array}\right] \geq \epsilon.$$

By an averaging argument, we have:

$$\Pr_{j_2,\tau^{(1)}}\left[\Pr_{\mathbf{c},\tau^{(2)}}\left[\begin{array}{c}\mathsf{Ver}(\mathsf{pk}, \mu^*, (\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*)) = 1 : \\ (\mu^*, \mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*) \leftarrow \mathbf{G}_1^{\mathcal{A}}(j_2, \tau^{(1)}, \mathbf{c}, \tau^{(2)})\end{array}\right] \geq \epsilon/2\right] \geq \epsilon/2.$$

Step 1. Analyzing "good" $(j_2, \tau^{(1)})$. Fix a "good" $(j_2, \tau^{(1)})$, namely one for which

$$\Pr_{\mathbf{c},\tau^{(2)}}\left[\begin{array}{c}\mathsf{Ver}(\mathsf{pk}, \mu^*, (\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*)) = 1 : \\ (\mu^*, \mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*) \leftarrow \mathbf{G}_1^{\mathcal{A}}(j_2, \tau^{(1)}, \mathbf{c}, \tau^{(2)})\end{array}\right] \geq \epsilon/2 \; .$$

This means that upon "rewinding", we have:

$$\Pr_{\mathbf{c}_1,\tau_1^{(2)},\mathbf{c}_2,\tau_2^{(2)}}\left[\begin{array}{c}\mathsf{Ver}(\mathsf{pk}, \mu^*, (\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}_1^*)) = 1 \\ \wedge\ \mathsf{Ver}(\mathsf{pk}, \mu^*, (\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}_2^*)) = 1 : \\ (\mu^*, \mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}_b^*) \leftarrow \mathbf{G}_1^{\mathcal{A}}(j_2, \tau^{(1)}, \mathbf{c}_b, \tau_b^{(2)}), b \in \{1,2\}\end{array}\right] \geq \epsilon^2/4 \; .$$

Expanding $\mathsf{Ver}(\mathsf{pk}, \mu^*, (\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}_b^*)) = 1$ yields:

$$\|\mathbf{z}_b^*\| \leq L_1 \wedge\ \mathbf{A}\mathbf{z}_b^* = \overline{\mathbf{w}}^* + \mathbf{P}\mathbf{c}_b \wedge\ \|\mathbf{U}^*\mathbf{z}_b^* - \underline{\mathbf{w}}^* - \mathbf{Y}^*\mathbf{c}_b\| \leq L_2, b \in \{1,2\}$$

where $\mathbf{U}^* = H_1(\mu^*)$. Subtracting the two and substituting $\mathbf{P} = \mathbf{A}\mathbf{S}$ yields:

$$\begin{aligned}\|\mathbf{z}_1^* - \mathbf{z}_2^*\| \ \leq\ 2L_1 \ \wedge\ \mathbf{A} \cdot \big((\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2)\big) = \mathbf{0} \\ \wedge\ \|\mathbf{U}^*(\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{Y}^*(\mathbf{c}_1 - \mathbf{c}_2)\| \ \leq\ 2L_2\end{aligned} \tag{4}$$

Now, we divide (4) into two cases depending on whether $(\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2) \stackrel{?}{=} \mathbf{0}$. Observe that

$$(\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2) = \mathbf{0} \Longrightarrow \mathbf{U}^*(\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{Y}^*(\mathbf{c}_1 - \mathbf{c}_2) = (\mathbf{U}^*\mathbf{S} - \mathbf{Y}^*)(\mathbf{c}_1 - \mathbf{c}_2).$$

Therefore, (4) implies $E_1 \vee E_2$, where:

$$\begin{aligned}&E_1 : \ \Big(\mathbf{v}^* := (\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2) \neq \mathbf{0} \ \wedge\ \|\mathbf{z}_1^* - \mathbf{z}_2^*\| \leq 2L_1 \wedge\ \mathbf{A}\mathbf{v}^* = \mathbf{0}\Big) \\ &E_2 : \ \|(\mathbf{U}^*\mathbf{S} - \mathbf{Y}^*)(\mathbf{c}_1 - \mathbf{c}_2)\| \leq 2L_2\end{aligned}$$

In particular, for every "good" $(j_2, \tau^{(1)})$, either E_1 happens with probability $\epsilon^2/8$, or E_2 happens with probability $\epsilon^2/8$. This means that one of the following must hold:

- Case 1:

$$\Pr_{j_2,\tau^{(1)}}\left[\Pr_{\mathbf{c}_1,\tau_1^{(2)},\mathbf{c}_2,\tau_2^{(2)}}\left[\begin{array}{c}(\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2) \neq \mathbf{0} \\ \wedge\ \|\mathbf{z}_1^* - \mathbf{z}_2^*\| \leq 2L_1 \\ \wedge\ \mathbf{A} \cdot \big((\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2)\big) = \mathbf{0}\end{array}\right] \geq \epsilon^2/8\right] \geq \epsilon/4$$

- Case 2:

$$\Pr_{j_2,\tau^{(1)}}\left[\Pr_{\mathbf{c}_1,\mathbf{c}_2}\big[\|(\mathbf{U}^*\mathbf{S} - \mathbf{Y}^*)(\mathbf{c}_1 - \mathbf{c}_2)\| \leq 2L_2\big] \geq \epsilon^2/8\right] \geq \epsilon/4$$

  Note here that $(j_2, \tau^{(1)})$ specifies $\mathbf{Y}^*, \mathbf{U}^*$, which allows us to omit $\tau_1^{(2)}, \tau_2^{(2)}$.

We proceed to analyze the two cases.

Step 2: breaking SIS in case 1. Here, we show how to break HNF-MSIS with probability $\epsilon^3/32 - 4(k+1) \cdot 2^{-6\kappa}$ (cf. Theorem 1). Write $\tau^{(1)} = (\mathbf{A}, \mathbf{S}, \tau_0^{(1)})$. The HNF-MSIS adversary $\mathcal{A}'$ takes as input $\mathbf{A}$ and proceeds as follows:

- Sample uniformly at random $j_2, \mathbf{S}, \tau_0^{(1)}$.
- Sample uniformly at random $\mathbf{c}_1, \tau_1^{(2)}, \mathbf{c}_2, \tau_2^{(2)}$.
- Compute $\mathbf{z}_1^*$ and $\mathbf{z}_2^*$ by invoking $\mathcal{A}$ in $\mathbf{G}_1$ twice, on inputs $(j_2, \tau^{(1)}, \mathbf{c}_1, \tau_1^{(2)})$ and $(j_2, \tau^{(1)}, \mathbf{c}_2, \tau_2^{(2)})$, respectively.
- Output $\mathbf{x} = (\mathbf{z}_1^* - \mathbf{z}_2^*) - \mathbf{S}(\mathbf{c}_1 - \mathbf{c}_2)$.

Then, by the same analysis as in correctness (cf. Theorem 1), we have $\|\mathbf{S}\mathbf{c}_1\|, \|\mathbf{S}\mathbf{c}_2\| \leq L_1$, except with probability $\varepsilon_{\mathsf{cor}} \leq 4(k+1) \cdot 2^{-6\kappa}$, here we used $\varepsilon \leq 2^{-6\kappa}$, and therefore

$$\Pr[\mathbf{x} \neq \mathbf{0} \wedge \mathbf{A}\mathbf{x} = \mathbf{0} \wedge \|\mathbf{x}\| \leq 4L_1 : \mathbf{x} \leftarrow \mathcal{A}'(\mathbf{A})] \geq \epsilon^3/32 - 4(k+1) \cdot 2^{-6\kappa}$$

Step 3: winning $\mathbf{G}_2$ in case 2. Here, if $\Pr_{\mathbf{c}_1,\mathbf{c}_2}[\|(\mathbf{U}^*\mathbf{S}-\mathbf{Y}^*)(\mathbf{c}_1-\mathbf{c}_2)\| \le 2L_2] \ge \epsilon^2/8$, then the probability that none of $\lceil 12\kappa/\epsilon^2 \rceil$ random $\mathbf{c}_1, \mathbf{c}_2$ satisfies $\|(\mathbf{U}^*\mathbf{S}-\mathbf{Y}^*)(\mathbf{c}_1-\mathbf{c}_2)\| \le 2L_2$ is at most $(1-\epsilon^2/8)^{12\kappa/\epsilon^2} \le 2^{-2\kappa}$. Therefore, the probability that the adversary $\mathcal{A}$ wins in $\mathbf{G}_2$ is at least $\epsilon/4 - 2^{-2\kappa}$. □

Lemma 13. *Let $m \ge \ell + n$, $\varepsilon \in (0,1)$, and $\mathbf{D} \in \mathbb{Z}^{md\times(m-\ell-n)d}$ [5], where $\mathbf{D}$ is defined in $\mathbf{G}_4$. And if we set $\sigma_{S_1}, \sigma_{r_1} \ge \eta_\varepsilon(\mathbb{Z}^{(m-\ell-n)d})$ and $\sigma_S \ge \sqrt{2}\sigma_{\max}(\mathbf{D})\cdot\sigma_{S_1}$, $\sigma_r \ge \sqrt{2}\sigma_{\max}(\mathbf{D})\cdot\sigma_{r_1}$. Define*

$$\mathbf{\Sigma}_S := \sigma_S^2\mathbf{I}_{md} - \sigma_{S_1}^2\mathbf{D}\mathbf{D}^\top, \mathbf{\Sigma}_r := \sigma_r^2\mathbf{I}_{md} - \sigma_{r_1}^2\mathbf{D}\mathbf{D}^\top$$

and let $\mathbf{S}^{(0)} \leftarrow (\mathcal{D}_{\mathcal{R}^m,\mathbf{\Sigma}_S})^k$, $\mathbf{S}^{(1)} \leftarrow (\mathcal{D}_{\mathcal{R}^{m-\ell-n},\sigma_{S_1}})^k$, $\mathbf{r}_j^{(0)} \leftarrow \mathcal{D}_{\mathcal{R}^m,\mathbf{\Sigma}_r}$, $\mathbf{r}_j^{(1)} \leftarrow \mathcal{D}_{\mathcal{R}^{m-\ell-n},\sigma_{r_1}}$, $j \in [Q_s]$.

Then, $\mathsf{Adv}_4 \le 2\cdot\mathsf{Adv}_5$, $|\mathsf{Adv}_4 - \mathsf{Adv}_5| \le 4(k+Q_s)\cdot 2^{-2\kappa}$. Also, $\sigma_{\max}(\mathbf{D}) \le \beta_{\mathsf{lwe}} d\sqrt{(m-\ell-n)m}$.

Proof. Applying Lemma 4 to each column of $\mathbf{S}, \mathbf{r}_j$ and taking a union bound, we have: $R_\infty(\mathbf{S},(\mathbf{r}_j)_{j\in[Q_s]} \| \mathbf{S}^{(0)} + \mathbf{D}\cdot\mathbf{S}^{(1)}, (\mathbf{r}_j^{(0)} + \mathbf{D}\cdot\mathbf{r}_j^{(1)})_{j\in[Q_s]}) \le (\frac{1+\varepsilon}{1-\varepsilon})^{Q_s+k}$ and $\Delta(\mathbf{S},(\mathbf{r}_j)_{j\in[Q_s]}; \mathbf{S}^{(0)} + \mathbf{D}\cdot\mathbf{S}^{(1)}, (\mathbf{r}_j^{(0)} + \mathbf{D}\cdot\mathbf{r}_j^{(1)})_{j\in[Q_s]}) \le (k+Q_s)\cdot\frac{2\varepsilon}{1-\varepsilon}$. Because we set $\varepsilon \le 2^{-2\kappa}$ and $4\varepsilon\cdot(Q_s+k)^2 < 1$, then we have $(\frac{1+\varepsilon}{1-\varepsilon})^{Q_s+k} \le 1 + (Q_s+k+1)\cdot\frac{2\varepsilon}{1-\varepsilon} \le 1 + 4(Q_s+k+1)2^{-2\kappa} \le 2$, and $(k+Q_s)\cdot\frac{2\varepsilon}{1-\varepsilon} \le 4(Q_s+k)\cdot 2^{-2\kappa}$. □

Lemma 14. *Assume that$\sigma_{S_1} \ge \eta_\varepsilon(\mathcal{R}^{m-\ell-n})$, $\sigma_{r_1} \ge \max(a_3\sqrt{Q_s}, \eta_\varepsilon(\mathcal{R}^{m-\ell-n}))$, $\sigma_1 \ge a_1\sqrt{kQ_s}$, and $\sigma_2 \ge a_2\sqrt{Q_s}$. Then*

$$\mathsf{Adv}_5 \le e^{2(\alpha-1)\pi}\mathsf{Adv}_6^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa} \quad \textit{and} \quad \mathsf{Adv}_6 \le e^{(\alpha-1)\pi}\mathsf{Adv}_7^{\frac{\alpha-1}{\alpha}} + k\cdot 2^{-4\kappa},$$

where $a_1 \ge \|\mathbf{E}''_{j'}\mathbf{D}\mathbf{S}^{(1)}\|_{1,2}$, $a_2 \ge \|\mathbf{E}''_{j'}\mathbf{D}\mathbf{r}_j^{(1)}\|$, $a_3 \ge \|\mathbf{S}^{(1)}\mathbf{c}\|$ are as defined in the proof.

Proof. In $\mathbf{G}_5$, using $\mathbf{A}\mathbf{S} = \mathbf{A}\mathbf{S}^{(0)}$, $\mathbf{A}\mathbf{r}_j = \mathbf{A}\mathbf{r}_j^{(0)}$, we have

$$\begin{aligned}
\mathbf{Y}_j &= (\mathbf{T}_{j'}\mathbf{A} + \mathbf{E}''_{j'})\mathbf{S}^{(0)} + \boxed{\mathbf{E}'_j + \mathbf{E}''_{j'}\mathbf{D}\mathbf{S}^{(1)}},\\
\underline{\mathbf{w}}_j &= (\mathbf{T}_{j'}\mathbf{A} + \mathbf{E}''_{j'})\mathbf{r}_j^{(0)} + \boxed{\mathbf{e}_j + \mathbf{E}''_{j'}\mathbf{D}\mathbf{r}_j^{(1)}},\\
\mathbf{z}_j &= \mathbf{r}^{(0)} + \mathbf{S}^{(0)}\mathbf{c} + \mathbf{D}\cdot\left(\boxed{\mathbf{r}^{(1)} + \mathbf{S}^{(1)}\mathbf{c}}\right)
\end{aligned}$$

Here $\mathbf{z}_j$ is the j-th query to oSIGN$_2$, which takes $(\mathbf{Y},\mathbf{w})$ as input and $\mathbf{r} = \mathbf{r}^{(0)} + \mathbf{D}\mathbf{r}^{(1)}$ is the st used by Sign_1 to generate the $(\mathbf{Y},\mathbf{w})$, $\mathbf{U}_{j'} = \mathbf{T}_{j'}\mathbf{A} + \mathbf{E}''_{j'} = H_1(\mu)$ is the j'-th query to H_1, and $\mathbf{c}_j = H_2(\mu,\mathbf{Y},\mathbf{w})$. In $\mathbf{G}_7$, the boxed terms are replaced by $\mathbf{E}'_j, \mathbf{e}_j, \mathbf{r}^{(1)}$ respectively. We will use noise flooding in two steps:

- $\mathbf{G}_6$: First replace $(\mathbf{E}'_j + \mathbf{E}''_{j'}\mathbf{D}\mathbf{S}^{(1)}, \mathbf{e}_j + \mathbf{E}''_{j'}\mathbf{D}\mathbf{r}_j^{(1)})$ by $(\mathbf{E}'_j, \mathbf{e}_j)$ so that $\mathbf{r}_j^{(1)}$ is perfectly hidden;
- $\mathbf{G}_7$: Then, replace $\mathbf{r}^{(1)} + \mathbf{S}^{(1)}\mathbf{c}$ by $\mathbf{r}^{(1)}$.

If $\eta_\varepsilon(\mathcal{R}^{m-\ell-n}) \le \sigma_b$, $b \in \{r_1, S_1\}$, by Lemma 1 and union bound, we have

- $\|\mathbf{r}_j^{(1)}\| \ge \sigma_{r_1}\sqrt{(m-\ell-n)d}$ with probability at most $\frac{1+\varepsilon}{1-\varepsilon}\cdot 2^{-(m-\ell-n)d} = \delta$.
- $\|\mathbf{S}^{(1)}\|_{1,2} \ge \sigma_{S_1}\sqrt{(m-\ell-n)d}$ with probability at most $k\cdot\delta$.

Thus, except for a probability $(k+Q_s)\cdot\delta \le (k+Q_s)\cdot 2^{-4\kappa}$, because $\varepsilon \le 2^{-4\kappa}$ and $d \ge 2\kappa$. We have

- $\|\mathbf{E}''_{j'}\mathbf{D}\mathbf{S}^{(1)}\|_{1,2} \le \sigma_{S_1}\beta_{\mathsf{lwe}}\beta_{\mathsf{hlwe}}m(m-\ell-n)d^2\sqrt{td} = a_1$
- $\|\mathbf{E}''_{j'}\mathbf{D}\mathbf{r}_j^{(1)}\| \le \sigma_{r_1}\beta_{\mathsf{lwe}}\beta_{\mathsf{hlwe}}m(m-\ell-n)d^2\sqrt{td} = a_2$
- $\|\mathbf{S}^{(1)}\mathbf{c}\| \le \sigma_{S_1}\sqrt{(m-\ell-n)d}\cdot\beta_c\cdot k = a_3$

Using Lemma 5 twice and Lemma 10, we have:

- $\mathsf{Adv}_5 \le \exp\left(\alpha\pi Q_s\left(\frac{ka_1^2}{\sigma_1^2} + \frac{a_2^2}{\sigma_2^2}\right)\right)^{\frac{\alpha-1}{\alpha}}\cdot\mathsf{Adv}_6^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa}$

[5] Here we abuse the notation of $\mathbf{D} = \phi_M(\mathbf{D})$

$\underline{\mathsf{KeyGen}(1^\kappa):}$

$\mathbf{A}' \leftarrow \mathcal{R}_q^{\ell \times (m-\ell)}, \mathbf{A} := [\mathbf{A}' \mid \mathbf{I}_\ell]$
$\mathbf{S} \leftarrow \mathcal{D}_{\mathcal{R}^{m \times k}, \sigma_S}, \mathbf{P} = \mathbf{AS}$
For $(i, j) \in [N] \times [N]$ do
  $\mathsf{seed}_{i,j} \leftarrow \{0,1\}^{2\kappa}$
$\tilde{\mathbf{S}}_1, \ldots, \tilde{\mathbf{S}}_{T-1} \leftarrow \mathcal{R}_q^{m \times k}$
For $i \in [N]$ do
  $\mathbf{S}_i = \mathbf{S} + \sum_{j=1}^{T-1} \tilde{\mathbf{S}}_j i^j$
  $\mathsf{sk}_i = (\mathbf{S}_i, (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N]})$
Return $(\mathsf{pk} = (\mathbf{A}, \mathbf{P}), (\mathsf{sk}_i)_{i \in [N]})$

$\underline{\mathsf{Sign}_1(i, \mathsf{sk}_i, \mathrm{SS}, \mu):}$

Require: $i \in \mathrm{SS} \subseteq [N]$
$(\mathbf{S}_i, (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N]}) = \mathsf{sk}_i$
$\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}$; $\mathbf{e}_i \leftarrow \mathcal{D}_{\mathcal{R}^t, \sigma_2}$
$\mathbf{U} = H_1(\mu) \in \mathcal{R}_q^{t \times m}$
$\mathbf{E}_i' = H_5(\mathsf{seed}_{i,i}, \mathsf{pk}, \mathrm{SS}, \mu)$
$\mathsf{mask}_i \leftarrow \sum_{j \in \mathrm{SS}} H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu)$
$\mathsf{mask}_i' \leftarrow \sum_{j \in \mathrm{SS}} H_3(\mathsf{seed}_{j,i}, \mathsf{pk}, \mathrm{SS}, \mu)$
$\mathbf{Y}_i = L_{\mathrm{SS},i} \cdot \mathbf{US}_i + \mathbf{E}_i' + \mathsf{mask}_i - \mathsf{mask}_i'$
$\mathbf{w}_i = \begin{bmatrix} \overline{\mathbf{w}}_i \\ \underline{\mathbf{w}}_i \end{bmatrix} = \begin{bmatrix} \mathbf{Ar}_i \\ \mathbf{Ur}_i + \mathbf{e}_i \end{bmatrix}$
$\mathsf{\bar{st}} = (\mathbf{E}_i', \mathbf{r}_i, \mathbf{w}_i)$
Return $(\mathbf{Y}_i, \mathbf{w}_i), \mathsf{\bar{st}}$

$\underline{\mathsf{Sign}_2(i, \mathsf{sk}_i, \mathsf{\bar{st}}, \mathrm{SS}, \mu, M):}$

Require: $i \in \mathrm{SS} \subseteq [N]$
$\{(j, \mathbf{Y}_j, \mathbf{w}_j)\}_{j \in \mathrm{SS}} = M$
$(\mathbf{E}_i', \mathbf{r}_i, \hat{\mathbf{w}}_i) = \mathsf{\bar{st}}$
$(\mathbf{S}_i, (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N]}) = \mathsf{sk}_i$
If $\hat{\mathbf{w}}_i \neq \mathbf{w}_i$ then return $\perp$
$\mathbf{Y} = \sum_{j \in \mathrm{SS}} \mathbf{Y}_j$; $\mathbf{w} = \sum_{j \in \mathrm{SS}} \mathbf{w}_j$
$\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w}) \in \mathcal{C}^k \subset \mathcal{R}^k$
$\mathsf{mask}_i \leftarrow \sum_{j \in \mathrm{SS}} H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
$\mathsf{mask}_i' \leftarrow \sum_{j \in \mathrm{SS}} H_4(\mathsf{seed}_{j,i}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
$\mathbf{z}_i = \mathbf{r}_i + L_{\mathrm{SS},i} \cdot \mathbf{S}_i \mathbf{c} + \mathsf{mask}_i - \mathsf{mask}_i'$
Return $\mathbf{z}_i$

$\underline{\mathsf{Combine}(\mathrm{SS}, \mu, \{\mathbf{z}_i\}_{i \in \mathrm{SS}}, \{(\mathbf{Y}_i, \mathbf{w}_i)\}_{i \in \mathrm{SS}}):}$

$\mathbf{Y} = \sum_{j \in \mathrm{SS}} \mathbf{Y}_j$; $\mathbf{w} = \sum_{j \in \mathrm{SS}} \mathbf{w}_j$
$\mathbf{z} = \sum_{i \in \mathrm{SS}} \mathbf{z}_i$
Return $(\mathbf{Y}, \mathbf{w}, \mathbf{z})$

Fig. 6. Lattice-based T-out-of-N threshold signatures Tweed. Also, $H_1 : \{0,1\}^* \to \mathcal{R}_q^{t \times m}$, $H_2 : \{0,1\}^* \to \mathcal{C}^k$, $H_3 : \{0,1\}^* \to \mathcal{R}_q^{t \times k}$, $H_4 : \{0,1\}^* \to \mathcal{R}_q^m$, and $H_5 : \{0,1\}^* \to \mathcal{R}_q^{t \times k}$. In addition, in the ROM, the output of H_5 follows the discrete Gaussian distribution $\mathcal{D}_{\mathcal{R}^{t \times k}, \sigma_1}$. Also, $L_{\mathrm{SS},i}$ denotes the Lagrange coefficient. The public key pk is implicitly given to all algorithms except KeyGen. The verification algorithm Ver is the same as the one defined in Figure 5, except that we enlarge L_1 and L_2 by $\sqrt{N}$ times relative to Figure 5 (see Theorem 3).

– $\mathsf{Adv}_6 \leq \exp\left(\alpha \pi Q_s \left(\frac{a_3}{\sigma_{r_1}}\right)^2\right)^{\frac{\alpha-1}{\alpha}} \cdot \mathsf{Adv}_7^{\frac{\alpha-1}{\alpha}} + k \cdot 2^{-4\kappa}$

If we set $\sigma_{r_1} \geq a_3 \sqrt{Q_s}$, $\sigma_1 \geq a_1 \sqrt{k Q_s}$, and $\sigma_2 \geq a_2 \sqrt{Q_s}$, then we can bound Adv_5 and Adv_6. $\square$

Lemma 15 (Bounding Adv_8). *Assume that $\mathcal{C} := \{c \in \mathcal{R} : \|c\|_\infty = 1, \|c\|_1 = \beta_c\}$. If $d \geq h > 1$ is a power of 2 and $q = 2h + 1 \bmod 4h$ is a prime and $\frac{1}{\sqrt{h}} q^{1/h} > 2$, then we have $\mathsf{Adv}_8 \leq \frac{48\kappa}{\epsilon^2} \cdot 2^{-8\kappa}$, where $\epsilon \leq Q_h^{-2} \cdot \mathsf{Adv}^{\mathsf{adp\text{-}tsuf}}_{\mathsf{Tweed}[1,1]}(\mathcal{A}, \kappa) = \mathsf{Adv}_1$.*

Proof. Let $N = \lceil 12\kappa/\epsilon^2 \rceil$. It suffices to show that

$$\Pr_{\mathbf{V}, (\mathbf{c}_{1,i}, \mathbf{c}_{2,i})_{i \in [N]}} \left[\exists i \in [N] \text{ s.t. } \|\mathbf{V}(\mathbf{c}_{1,i} - \mathbf{c}_{2,i}) \bmod q\| \leq L_2\right] \leq 2 \cdot 2^{-8\kappa}.$$

By Lemma 6, for any $\hat{\mathbf{c}} := \mathbf{c}_1 - \mathbf{c}_2 \neq \mathbf{0}$, there exists $i \in [k]$ such that $\hat{\mathbf{c}}_i$ is invertible in $\mathcal{R}_q$. Hence the distribution of $\mathbf{V}\hat{\mathbf{c}}$ is uniform over $\mathcal{R}_q^t$, and therefore $\Pr_{\mathbf{V}}\left[\|\mathbf{V}\hat{\mathbf{c}} \bmod q\| \leq L_2\right] \leq \left(\frac{L_2}{q}\right)^{dt}$. Consequently, $\Pr_{\mathbf{V}, \mathbf{c}_1, \mathbf{c}_2}\left[\|\mathbf{V}(\mathbf{c}_1 - \mathbf{c}_2) \bmod q\| \leq L_2\right] \leq \Pr_{\mathbf{c}_1, \mathbf{c}_2}\left[\mathbf{c}_1 = \mathbf{c}_2\right] + \Pr_{\mathbf{V}}\left[\|\mathbf{V}\hat{\mathbf{c}} \bmod q\| \leq L_2\right] \leq \frac{1}{|\mathcal{C}|^k} + \left(\frac{L_2}{q}\right)^{dt} \leq 2 \cdot 2^{-8\kappa}$, where the last inequality uses $(q/L_2)^{dt} \geq 2^{8\kappa}$, and $|\mathcal{C}|^k \geq 2^{8\kappa}$. The bound on Adv_8 then follows by a union bound over $i \in [N]$. $\square$

5 Threshold Signature Construction

In this section, we first present our threshold signature scheme, Tweed, for general threshold and then prove the adaptive security of Tweed.

Game $\mathsf{IUF}^{\mathcal{A}}(\kappa)$:

$\mathsf{CS} = 0$; $S = \emptyset$; $H \leftarrow \mathsf{Tweed.HF}$
$\mathbf{A}' \leftarrow \mathcal{R}_q^{\ell \times (m-\ell)}$; $\mathbf{A} := [\mathbf{A}' \mid \mathbf{I}_\ell]$
$\mathbf{S} \leftarrow \mathcal{D}_{\mathcal{R}^{m\times k}, \sigma_S}$; $\mathbf{P} \leftarrow \mathbf{AS}$
For $i \in [N]$ do
  $\mathsf{st}_i.\mathsf{cnt} = 0$; $\mathsf{st}_i.\mathsf{P} = ()$
$(\mu, sig) \leftarrow \mathcal{A}^{\text{oSIGN}_1, \text{oSIGN}_2, \text{RO}, \text{COR}}(\mathbf{A}, \mathbf{P})$
Return $(\mu \notin S \ \wedge \ \mathsf{Ver}(\mathsf{pk}, \mu, sig) = 1)$

Oracle $\text{oSIGN}_1^{(\mathsf{w})}(i, \text{SS}, \mu)$:

Require: $i \in \text{SS} \setminus \mathsf{CS}$, $\text{SS} \subseteq [N]$ and $|\text{SS}| \geq T$
$\mathsf{st}_i.\mathsf{cnt} = \mathsf{st}_i.\mathsf{cnt} + 1$; $S = S \cup \{\mu\}$
If $\mathbf{E}'_{i,\text{SS},\mu} = \perp$ then
  $\mathbf{E}'_{i,\text{SS},\mu} \leftarrow \mathcal{D}_{\mathcal{R}^{t\times k}, \sigma_1}$
$\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}$; $\mathbf{e}_i \leftarrow \mathcal{D}_{\mathcal{R}^t, \sigma_2}$
$\mathbf{U} = H_1(\mu) \in \mathcal{R}_q^{t \times m}$
$\mathbf{w}_i = \begin{bmatrix} \overline{\mathbf{w}}_i \\ \underline{\mathbf{w}}_i \end{bmatrix} = \begin{bmatrix} \mathbf{A}\mathbf{r}_i \\ \mathbf{U}\mathbf{r}_i + \mathbf{e}_i \end{bmatrix}$
$\mathsf{st}_i.\mathsf{P}(\mathsf{st}_i.\mathsf{cnt}) = (0, (\mathbf{E}'_{i,\text{SS},\mu}, \mathbf{r}_i, \mathbf{w}_i))$
Return $\mathbf{w}_i$

Oracle $\mathsf{RO}(x)$:

Return $H(x)$

Oracle $\text{oSIGN}_1^{(\mathsf{Y})}(\text{SS}, \mu)$:

Require: $\text{SS} \subseteq [N]$ and $|\text{SS}| \geq T$
If $\exists\, i \in \text{SS} \setminus \mathsf{CS} : \mathbf{E}'_{i,\text{SS},\mu} = \perp$ then
  Return $\perp$
$\mathbf{U} = H_1(\mu) \in \mathcal{R}_q^{t \times m}$
$\mathbf{Y} = \mathbf{US} + \sum_{i \in \text{SS}\setminus\text{CS}} \mathbf{E}'_{i,\text{SS},\mu}$
Return $\mathbf{Y}$

Oracle $\text{oSIGN}_2(\text{SS}, Q, \mu, \mathbf{Y}, \mathbf{w})$:

Require: $\forall\, i \in \text{SS} \setminus \mathsf{CS} : \text{sid}_i \leq \mathsf{st}_i.\mathsf{cnt}$,
  $\text{SS} \subseteq [N]$ and $|\text{SS}| \geq T$
$Q = \{(i, \text{sid}_i)\}_{i \in \text{SS}\setminus\text{CS}}$
For $i \in \text{SS} \setminus \mathsf{CS}$ do
  $(\mathsf{isUsed}, (\mathbf{E}'_i, \mathbf{r}_i, \mathbf{w}_i)) = \mathsf{st}_i.\mathsf{P}(\text{sid}_i)$
  If isUsed then return $\perp$
  $\mathsf{st}_i.\mathsf{P}(\text{sid}_i) = (1, (\mathbf{E}'_i, \mathbf{r}_i, \mathbf{w}_i))$
$\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$
$\mathbf{z} = \sum_{i \in \text{SS}\setminus\text{CS}} \mathbf{r}_i + \mathbf{Sc}$
Return $\mathbf{z}$

Oracle $\text{COR}(i)$:

Require: $|\mathsf{CS}| < T - 1$
$\mathsf{CS} \leftarrow \mathsf{CS} \cup \{i\}$
Return st_i

Fig. 7. The intermediate game IUF. Here $H_i(x)$ denotes $H(i, x)$.

The scheme is shown in Figure 6. For a set $\text{SS} \subseteq [N]$ and $i \in \text{SS}$, we denote the i-th Lagrange coefficient as $L_{\text{SS},i} = \prod_{j \in \text{SS}\setminus\{i\}} \frac{-j}{i-j}$. Each singer i holds a Shamir secret share $\mathbf{S}_i$ of the secret key $\mathbf{S}$ together with pair-wise seeds $\{\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i}\}_{j \in [N]}$, which are used to generate masks during signing.

The main differences from the single-signer scheme are as follows. In the first signing round, each signer i computes a mask Mask_i based on (SS, μ) and computes $\mathbf{Y}_i = L_{\text{SS},i}\mathbf{US}_i + \mathbf{E}'_i + \mathsf{Mask}_i$. The mask is used here to hide $L_{\text{SS},i}\mathbf{US}_i$. Also, $\mathbf{E}'_i$ is computed from a hash function H_5 instead of sampling by signer i to make sure the same $\mathbf{E}'_i$ is used each time given the same (SS, μ). This is necessary since if signer i samples a new $\mathbf{E}'_i$ on the same (SS, μ) multiple times, the value $L_{\text{SS},i}\mathbf{US}_i + \mathsf{Mask}_i$ would be leaked. In the random oracle model, we model the output H_5 as a sample from the discrete Gaussian distribution $\mathcal{D}_{\mathcal{R}^{t\times k}, \sigma_1}$. This use of the ROM has appeared in prior works, e.g. [LW22, BKL+25].

Similarly, in the second signing round, each signer i computes another mask Mask'_i based on (SS, μ, M). This mask is then applied to the response $\mathbf{z}_i$ in order to hide $L_{\text{SS},i}\mathbf{S}_i\mathbf{c}$. The following theorem establishes the correctness of Figure 6.

Theorem 3 (Correctness of Threshold signature scheme). *If $\sigma_1, \sigma_2 \geq 2\eta_\varepsilon(\mathcal{R}_q^t)$, $\sigma_r, \sigma_S \geq 2\eta_\varepsilon(\mathcal{R}_q^m)$, and $L_1^{(1)} \geq \sqrt{mdN}(\sigma_r + \beta_c k \sigma_S)$, $L_2^{(1)} \geq \sqrt{tdN}(\sigma_2 + \beta_c k \sigma_1)$, then the scheme in Fig 6 is correct with a correctness error $\varepsilon_{\mathsf{cor}} \leq 8(k+1) \cdot 2^{-2\kappa}$.*

Proof. To show correctness, fix any signing interaction on a message μ with a signer set $\text{SS} \subseteq [N]$ such that $|\text{SS}| \geq T$, and let μ be the associated message. For an honestly generated transcript $(\mathbf{Y}, \mathbf{w}, \mathbf{z})$ and $\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$ in Figure 6, it suffices to verify the following:

- $\mathbf{Az} = \mathbf{A}(\mathbf{r} + \mathbf{Sc}) = \overline{\mathbf{w}} + \mathbf{Pc}$.
- $\|\mathbf{Uz} - \underline{\mathbf{w}} - \mathbf{Yc}\| = \| -(\mathbf{e} + \mathbf{E}'\mathbf{c})\| \leq \|\mathbf{e}\| + \|\mathbf{E}'\|_{1,2}\|\mathbf{c}\|_1$.
- $\|\mathbf{z}\| = \|\mathbf{r} + \mathbf{Sc}\| \leq \|\mathbf{r}\| + \|\mathbf{S}\|_{1,2}\|\mathbf{c}\|_1$.

Here,

$$\mathbf{Y} = \sum_{i \in \mathrm{SS}} L_{\mathrm{SS},i} \cdot \mathbf{US}_i + \mathbf{E}'_i + \mathsf{mask}_i - \mathsf{mask}'_i = \mathbf{US} + \mathbf{E}',$$

$$\mathbf{z} = \sum_{i \in \mathrm{SS}} \mathbf{r}_i + L_{\mathrm{SS},i} \cdot \mathbf{S}_i \mathbf{c} + \mathsf{mask}_i - \mathsf{mask}'_i = \mathbf{r} + \mathbf{Sc},$$

where the second equalities use $\mathbf{S} = \sum_{i \in \mathrm{SS}} L_{\mathrm{SS},i} \cdot \mathbf{S}_i$ and $\sum_{i \in \mathrm{SS}} \mathsf{mask}_i = \sum_{i \in \mathrm{SS}} \mathsf{mask}'_i$, and we write $\mathbf{w} = \sum_{j \in \mathrm{SS}} \mathbf{w}_j$, $\mathbf{e} = \sum_{j \in \mathrm{SS}} \mathbf{e}_j$, $\mathbf{r} = \sum_{j \in \mathrm{SS}} \mathbf{r}_j$, and $\mathbf{E}' = \sum_{j \in \mathrm{SS}} \mathbf{E}'_j$.

By Lemma 8 and Lemma 9, if $\sigma_2 \geq 2\eta_\varepsilon(\mathcal{R}^t)$ and $\mathbf{e}_i \leftarrow \mathcal{D}_{\mathcal{R}^t,\sigma_2}$ are sampled i.i.d., then $\mathbf{e} = \sum_{i \in \mathrm{SS}} \mathbf{e}_i \overset{\varepsilon'}{\approx} \mathcal{D}_{\mathcal{R}^t,\sqrt{|\mathrm{SS}|}\cdot\sigma_2}$, where $\varepsilon' = \frac{2\varepsilon}{1-\varepsilon}$. Hence, $\|\mathbf{e}\| \leq \sqrt{|\mathrm{SS}|} \cdot \sigma_2 \cdot \sqrt{td} \leq \sqrt{tdN} \cdot \sigma_2$ except with probability $\frac{\varepsilon}{1-\varepsilon} + \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-td}$. Applying the same argument to $\mathbf{E}'_j$ and $\mathbf{r}_j$, we obtain

- $\|\mathbf{Uz} - \underline{\mathbf{w}} - \mathbf{Yc}\| \leq \sqrt{tdN}(\sigma_2 + \beta_c k \sigma_1) \leq L_2^{(1)}$,
- $\|\mathbf{z}\| \leq \sqrt{mdN}(\sigma_r + \beta_c k \sigma_S) \leq L_1^{(1)}$,

except with correctness error $\varepsilon_{\mathsf{cor}} \leq 8(k+1) \cdot 2^{-2\kappa}$, where we used $\varepsilon \leq 2^{-2\kappa}$ and $d \geq 2\kappa$. □

Security. We show the adaptive security of Tweed under the MLWE and MSIS assumption in the ROM in two steps. First, we show that the adaptive security of Tweed can be reduced to an intermediate game IUF (defined in Figure 7), where each signer no longer stores a secret key in its local state, and the adversary only sees the partial sum $\mathbf{Y}$ and $\mathbf{z}$ of partial signatures $(\mathbf{Y}_i,\mathbf{z}_i)$, as described in Equation (3), instead of the individual partial signatures. Concretely, the adversary is given three signing oracles $\text{oSign}_1^{(\mathsf{w})}$, $\text{oSign}_1^{(\mathsf{Y})}$, and oSign_2. The oracle $\text{oSign}_1^{(\mathsf{w})}$ takes (i, SS, μ) as input and returns a first-round nonce $\mathbf{w}_i$ from signer i, which is computed the same as the $\mathsf{adp\text{-}TSUF}$ game. Also, the oracle samples $\mathbf{E}'_{i,\mathrm{SS},\mu} \leftarrow \mathcal{D}_{\mathcal{R}^{t\times k},\sigma_1}$, which corresponds to the output of the random oracle $H_5(\mathsf{seed}_{i,i}, \mathsf{pk}, \mathrm{SS}, \mu)$ in the $\mathsf{adp\text{-}TSUF}$ game. The oracle $\text{oSign}_1^{(\mathsf{Y})}$ takes (SS, μ) and returns the partial sum $\mathbf{Y} = \mathbf{US} + \sum_{i \in \mathrm{SS}\setminus\mathrm{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu}$ if the adversary made a query (i, SS, μ) to $\text{oSign}_1^{(\mathsf{w})}$ for each $i \in \mathrm{SS} \setminus \mathrm{CS}$. The oracle oSign_2 takes SS, μ, a set of indexes of first-round nonces $\{(i, \mathrm{sid}_i)\}_{i \in \mathrm{SS}\setminus\mathrm{CS}}$, together with aggregated values $(\mathbf{Y}, \mathbf{w})$, and returns the partial sum $\mathbf{z} = \sum_{i \in \mathrm{SS}\setminus\mathrm{CS}} \mathbf{r}_i + \mathbf{Ss}$, where $\mathbf{r}_i$ is the randomness used to generate the first-round nonce with index sid_i from signer i.

Also, IUF is similar to the single-signer security game with the following differences. The first signing round returns $\mathbf{Y} = \mathbf{US} + \sum_{i \in \mathrm{SS}\setminus\mathrm{CS}} \mathbf{E}'_i$ together with $\mathbf{w}_i$ as before, and the second signing rounds returns $\mathbf{z} = \sum_{i \in \mathrm{SS}\setminus\mathrm{CS}} \mathbf{r}_i + \mathbf{Sc}$, where each $\mathbf{E}'_i$ and $\mathbf{r}_i$ are sampled independently as in the single-signer protocol. Moreover, the adversary is allowed to obtain *adaptively* all but one of the values in $\{(\mathbf{E}'_i, \mathbf{r}_i)\}_{i \in \mathrm{SS}\setminus\mathrm{CS}}$.

Then, we show the hardness of the IUF game under the MLWE and MSIS assumption. Thanks to our proof strategy, which largely avoids rewinding, we can prove the hardness of IUF under the standard lattice assumptions similarly to the single-signer case. Formally, the adaptive security of Tweed follows from the following two lemmas.

Lemma 16. *For any $1 \leq T \leq N$ and any $\mathsf{adp\text{-}TSUF}_{\mathsf{Tweed}[N,T]}$ adversary $\mathcal{A}$ making at most Q_s queries to the oSign_1 and Q_h queries to the RO, there exists an IUF adversary $\mathcal{B}$ making at most Q_s queries to the $\text{oSign}_1^{(\mathsf{w})}$ and Q_h queries to the RO, $\mathsf{Adv}^{\mathsf{adp\text{-}tsuf}}_{\mathsf{Tweed}[N,T]}(\mathcal{A}, \kappa) \leq \mathsf{Adv}^{\mathsf{iuf}}(\mathcal{B}, \kappa) + Q_h N^2 \cdot 2^{-2\kappa}$.*

Lemma 17. *For any integers $q = q(\kappa), m = m(\kappa), n = n(\kappa), k = k(\kappa), t = t(\kappa), \ell = \ell(\kappa)$ and any adversary $\mathcal{A}$ making at most Q_s queries to $\text{oSign}_1^{(\mathsf{w})}$ and $\text{oSign}_1^{(\mathsf{Y})}$ oracle and Q_h queries to Random Oracle, then either there exists an $\mathsf{HNF\text{-}MSIS}$ adversary $\mathcal{A}'$ running in time roughly two times that of $\mathcal{A}$ such that,*

$$\mathsf{Adv}^{\mathsf{iuf}}(\mathcal{A}, \kappa) \leq 32^{\frac{1}{3}} Q_h^2 \cdot (\mathsf{Adv}^{\mathsf{hnf\text{-}msis}}_{q,d,\ell,m,4L_1^{(1)}}(\mathcal{A}', \kappa) + 8(k+1) \cdot 2^{-6\kappa})^{\frac{1}{3}}.$$

or there exists an $\mathsf{HNF\text{-}MLWE}$ adversary $\mathcal{B}$ and an MLWE adversary $\mathcal{C}$ running in time roughly of $\mathcal{A}$ plus $\mathsf{poly}(\kappa, 1/\epsilon)$ such that

$$\begin{aligned}\mathsf{Adv}^{\mathsf{iuf}}(\mathcal{A}, \kappa) \leq \max\{2^{-\kappa}, 8Q_h^2 \Big(&\delta_2 \cdot 2^{-2\kappa} + \delta_3 \cdot 2^{-3\kappa} + \delta_4 \cdot 2^{-4\kappa} \\ &+ t \cdot Q_h \cdot \mathsf{Adv}^{\mathsf{hnf\text{-}mlwe}}_{q,d,m,\ell+m,\beta_{\mathsf{hlwe}}}(\mathcal{B}, \kappa) \\ &+ (m - \ell - n) \cdot \mathsf{Adv}^{\mathsf{mlwe}}_{q,d,\ell,n+\ell,\beta_{\mathsf{lwe}}}(\mathcal{C}, \kappa)\Big)\},\end{aligned}$$

where $\delta_2 = 4e^{8\pi}k + 1$, $\delta_3 = 16e^{12\pi}\sqrt{48\kappa} \cdot 2Q_h^2 + e^{4\pi}(k + Q_s)$, $\delta_4 = 16e^{12\pi}\sqrt{2k} + k + Q_s$.

5.1 Proof of Lemma 16

We first show the reduction from the game IUF to the adp-$\mathsf{TSUF}_{\mathsf{Tweed}}$ game, which is formally stated in Lemma 16.

Proof (of Lemma 16). We show the lemma via the following series of games.

$\mathbf{G}_0$: Same as the adp-$\mathsf{TSUF}_{\mathsf{Tweed}}$ game defined in Fig 4.

$\mathbf{G}_1$: Same as $\mathbf{G}_0$ except that we change the way the masks in the oSIGN_1 and oSIGN_2 oracle are computed. All the masks for (SS, μ) (resp. (SS, μ, M)) are sampled during the first oSIGN_1 query (resp. the first oSIGN_2 query), and then we program the random oracle $H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu)$ (resp. $H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M)$) accordingly. In particular, the masks are sampled as follows. For simplicity, we only explain the masks used in oSIGN_1 here. The case of oSIGN_2 is similar. For each (SS, μ), we sample $\mathsf{mask}_{i,\mathrm{SS},\mu}$ uniformly for all $i \in \mathrm{SS} \cap \mathsf{HS}$ except the last index i^* in $\mathrm{SS} \cap \mathsf{HS}$, and we set $\mathsf{mask}_{i^*,\mathrm{SS},\mu} = -\sum_{i \in \mathrm{SS} \setminus \{i^*\}} \mathsf{mask}_{i,\mathrm{SS},\mu}$. Note here for $i \in \mathsf{CS}$, $\mathsf{mask}_{i,\mathrm{SS},\mu}$ is already determined since the seeds $\{\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i}\}_{j \in [N]}$ were already known by the adversary. We use $\mathsf{CompMask}_1$ (resp. $\mathsf{CompMask}_2$) to denote the algorithm that takes (i, SS, μ) (resp. (i, SS, μ, M)) as input and computes the first (resp. second) round mask $\mathsf{mask}_{i,\mathrm{SS},\mu}$ (resp. $\mathsf{mask}_{i,\mathrm{SS},\mu,M}$). The detailed differences are shown in Figure 9.

Denote $\mathsf{BadSeed}$ as the event that $\mathcal{A}$ makes a RO query on $H_3(\mathsf{seed}_{i,j}, x)$, $H_4(\mathsf{seed}_{i,j}, x)$ or $H_5(\mathsf{seed}_{i,j}, x)$ for some $i, j \in [N]$ and x before signer i is corrupted. In addition to the above changes, $\mathbf{G}_1$ aborts if $\mathsf{BadSeed}$ occurs. Since each $\mathsf{seed}_{i,j}$ is uniformly sampled from $\{0,1\}^{2\kappa}$, the probability that $\mathcal{A}$ guesses one of an honest seed is at most $N^2 \cdot 2^{-2\kappa}$ in a single RO query, which implies the probability $\Pr[\mathsf{BadSeed}] \leq Q_h N^2 \cdot 2^{-2\kappa}$ in either $\mathbf{G}_0$ or $\mathbf{G}_1$. Since given $\mathsf{BadSeed}$ does not occur, $\mathbf{G}_1$ provides the same view to $\mathcal{A}$ as $\mathbf{G}_0$, $\mathsf{Adv}^{\mathbf{G}_1}(\mathcal{A}, \kappa) \geq \mathsf{Adv}^{\mathbf{G}_0}(\mathcal{A}, \kappa) - \Pr[\mathsf{BadSeed}] \geq \mathsf{Adv}^{\mathbf{G}_0}(\mathcal{A}, \kappa) - Q_h N^2 \cdot 2^{-2\kappa}$.

$\mathbf{G}_2$: Same as $\mathbf{G}_1$ except that we delay the programming of H_3 and H_4 until the corresponding party is corrupted. In particular, we program the value of $H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu)$ and $H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M)$ when the signer i is corrupted instead of in the algorithm $\mathsf{CompMask}_1$ and $\mathsf{CompMask}_2$. Concretely, the differences are shown in Figure 10. Since $\mathbf{G}_1$ and $\mathbf{G}_2$ aborts given $\mathsf{BadSeed}$ occurs and $\mathbf{G}_2$ provides the same view to $\mathcal{A}$ as $\mathbf{G}_1$ given $\mathsf{BadSeed}$ does not occur, $\mathsf{Adv}^{\mathbf{G}_2}(\mathcal{A}, \kappa) = \mathsf{Adv}^{\mathbf{G}_1}(\mathcal{A}, \kappa)$.

$\mathbf{G}_3$: Same as $\mathbf{G}_2$ except that the computation of each mask is delayed until it is needed either in the signing oracles or the corruption oracle. The differences are shown in Figure 11. Since the masks are only used either in the signing oracles or the COR oracle, the view of $\mathbf{G}_3$ is identical to that of $\mathbf{G}_2$, which means $\mathsf{Adv}^{\mathbf{G}_3}(\mathcal{A}, \kappa) = \mathsf{Adv}^{\mathbf{G}_2}(\mathcal{A}, \kappa)$

$\mathbf{G}_4$: Same as $\mathbf{G}_3$ except that we change the order of the sampling of the masks and the computation of the values to be masked during the signing oracles. For a first-round query (i, SS, μ), since $\mathbf{E}'_i$ and $\mathsf{mask}_{i,\mathrm{SS},\mu}$ are determined by the query, the response $\mathbf{Y}'_i$ is also unique. Therefore, we denote them as $\mathbf{E}'_{i,\mathrm{SS},\mu}$ and $\mathbf{Y}'_{i,\mathrm{SS},\mu}$ respectively. Similarly, for a second-round query (i, SS, μ, M), we denote the response $\mathbf{z}_i$ and the randomness $\mathbf{r}_i$ that is used to compute $\mathbf{z}_i$ as $\mathbf{z}_{i,\mathrm{SS},\mu,M}$ and $\mathbf{r}_{i,\mathrm{SS},\mu,M}$ respectively. For simplicity, we only explain the changes to oSIGN_1 here. The changes to oSIGN_2 are similar. In $\mathrm{oSIGN}_1(i, \mathrm{SS}, \mu)$, we first compute $\mathbf{Y}_{i,\mathrm{SS},\mu}$ and then derive the corresponding mask $\mathsf{mask}_{i,\mathrm{SS},\mu}$ from $\mathbf{Y}_{i,\mathrm{SS},\mu}$. Here, since the $\mathbf{Y}_i$ value is uniquely determined (SS, μ), we denote the value as $\mathbf{Y}_{i,\mathrm{SS},\mu}$. If i is not the last one in $\mathrm{SS} \cap \mathsf{HS}$ such that $\mathbf{Y}_{i,\mathrm{SS},\mu}$ is not computed yet, we sample $\mathbf{Y}_{i,\mathrm{SS},\mu}$ uniformly and compute $\mathsf{mask}_{i,\mathrm{SS},\mu} = \mathbf{Y}_{i,\mathrm{SS},\mu} - L_{\mathrm{SS},i}\mathbf{U}\mathbf{S}_i - \mathbf{E}'_{i,\mathrm{SS},\mu}$, where $\mathbf{E}'_{i,\mathrm{SS},\mu}$ is computed honestly from H_5. Note that here we do not invoke $\mathsf{CompMask}_1$ to compute the mask, but $\mathsf{CompMask}_1$ is still used to compute the mask in the COR oracle.

Otherwise, $\mathbf{Y}_{i,\mathrm{SS},\mu}$ can be determined by $\{\mathbf{Y}_{j,\mathrm{SS},\mu}\}_{j \in (\mathrm{SS} \setminus \mathsf{CS}) \setminus \{i\}}$ and the masks from corrupted signers. In particular, since

$$\begin{aligned}\sum_{j \in \mathrm{SS} \setminus \mathsf{CS}} \mathbf{Y}_{j,\mathrm{SS},\mu} &= \mathbf{U}\left(\sum_{j \in \mathrm{SS} \setminus \mathsf{CS}} L_{\mathrm{SS},j} \cdot \mathbf{S}_j\right) + \sum_{j \in \mathrm{SS} \setminus \mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu} + \sum_{j \in \mathrm{SS} \setminus \mathsf{CS}} \mathsf{mask}_{j,\mathrm{SS},\mu} \\ &= \mathbf{U}\left(\mathbf{S} - \sum_{j \in \mathrm{SS} \cap \mathsf{CS}} L_{\mathrm{SS},j} \cdot \mathbf{S}_j\right) + \sum_{j \in \mathrm{SS} \setminus \mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu} - \sum_{j \in \mathrm{SS} \cap \mathsf{CS}} \mathsf{mask}_{j,\mathrm{SS},\mu}\,,\end{aligned} \tag{5}$$

we compute

$$
\begin{aligned}
\mathbf{Y}_{i,\mathrm{SS},\mu} = \mathbf{U}\left(\mathbf{S} - \sum_{j\in \mathsf{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot \mathbf{S}_j\right) + \sum_{j\in \mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu} \\
- \sum_{j\in \mathsf{SS}\cap\mathsf{CS}} \mathsf{mask}_{j,\mathrm{SS},\mu} - \sum_{j\in (\mathrm{SS}\setminus\mathsf{CS})\setminus\{i\}} \mathbf{Y}_{j,\mathrm{SS},\mu}\ .
\end{aligned} \tag{6}
$$

We use SampY (resp. Compz) to denote the algorithm that takes (i, SS, μ) (resp. (i, SS, μ, M)) as input and computes the value $\mathbf{Y}_{i,\mathrm{SS},\mu}$ (resp. $\mathbf{z}_{i,\mathrm{SS},\mu,M}$).
In addition, we also change the way how $\mathsf{mask}_{i,\mathrm{SS},\mu}$ (resp. $\mathsf{mask}_{i,\mathrm{SS},\mu,M}$) is computed in $\mathsf{CompMask}_1$ (resp. $\mathsf{CompMask}_2$) in case $\mathrm{SS}\setminus\mathsf{CS}\subseteq\mathsf{CS}$. Note that due to the above modification, in $\mathbf{G}_4$, $\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)$ and $\mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)$ are only invoked by the Cor oracle when signer i is corrupted, which implies $i\in\mathrm{SS}\cap\mathsf{CS}$. Therefore, by Equation (5), we compute

$$
\begin{aligned}
\mathsf{mask}_{i,\mathrm{SS},\mu} = \mathbf{U}\left(\mathbf{S} - \sum_{j\in \mathsf{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot \mathbf{S}_j\right) + \sum_{j\in \mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu} \\
- \sum_{j\in \mathsf{SS}\cap\mathsf{CS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu} - \sum_{j\in \mathrm{SS}\setminus\mathsf{CS}} \mathbf{Y}_{j,\mathrm{SS},\mu}\ .
\end{aligned} \tag{7}
$$

We do a similar change to $\mathsf{CompMask}_2$. The detailed changes are shown in Figure 12. The view of $\mathcal{A}$ remains the same after both changes. Therefore, $\mathsf{Adv}^{\mathbf{G}_4}(\mathcal{A},\kappa) = \mathsf{Adv}^{\mathbf{G}_3}(\mathcal{A},\kappa)$.

$\mathbf{G}_5$: Same as $\mathbf{G}_4$ except that we further delay the computation of the masks in oSign_1 and oSign_2 to the Cor oracle. The differences are shown in Figure 15. We can delay the computation since the masks $\mathsf{mask}_{i,\mathrm{SS},\mu}$ and $\mathsf{mask}_{i,\mathrm{SS},\mu,M}$ for $i\in\mathsf{HS}$ are not used until signer i is corrupted. It is clear that the view of $\mathcal{A}$ remains the same. Therefore, $\mathsf{Adv}^{\mathbf{G}_5}(\mathcal{A},\kappa) = \mathsf{Adv}^{\mathbf{G}_4}(\mathcal{A},\kappa)$.

We now construct an adversary $\mathcal{B}$ playing the IUF game that runs $\mathcal{A}$ by simulating the $\mathbf{G}_5$ game.

Construction of $\mathcal{B}$. After receiving $(\mathbf{A},\mathbf{P})$ from the IUF game, $\mathcal{B}$ initializes CS, setQ_1, setQ_2, curMS_1 and curMS_2 following the game $\mathbf{G}_5$. Also, $\mathcal{B}$ initializes st_i as in $\mathbf{G}_5$ except that $\mathcal{B}$ sets $\mathsf{st}_i.\mathsf{sk}=\perp$. Then, $\mathcal{B}$ samples the seed $\mathsf{seed}_{i,j}$ for each $i,j\in[N]$ and runs $\mathcal{A}$ with input $(\mathbf{A},\mathbf{P})$ and access to oracles $\widetilde{\mathrm{oSign}}_1$, $\widetilde{\mathrm{oSign}}_2$, $\widetilde{\mathrm{RO}}$, $\widetilde{\mathrm{Cor}}$, which are simulated as follows.

$\widetilde{\mathbf{RO}}(x)$: If x can be parsed as (s,y) for $s\in\{1,2\}$, $\mathcal{B}$ directly forwards the query to its own oracle RO. Otherwise, $\mathcal{B}$ simulates the random oracle by lazy sampling as follows. If $s=3$ and $H(s,y)=\perp$, $\mathcal{B}$ samples $H(s,y)\leftarrow\mathcal{R}_q^{t\times k}$. If $s=4$ and $H(s,y)=\perp$, $\mathcal{B}$ samples $H(s,y)\leftarrow\mathcal{R}_q^{m}$. If $s=5$ and $H(s,y)=\perp$, $\mathcal{B}$ samples $H(s,y)\leftarrow\mathcal{D}_{\mathcal{R}^{t\times k},\sigma_1}$. Also, $\mathcal{B}$ aborts if BadSeed occurs, i.e., the query is made by the adverary and y can be parsed as $(\mathsf{seed}_{i,j}, x)$ for some $i,j\in[N]$ and $s\in\{3,4,5\}$.

$\widetilde{\mathbf{oSign}}_1(i,\mathrm{SS},\mu)$: Same as oSign_1 in $\mathbf{G}_5$ except that the execution of the algorithm SampY is changed as described below, $\mathcal{B}$ does not compute $\mathbf{E}'_{i,\mathrm{SS},\mu}$, and $\mathcal{B}$ sets $\mathbf{w}_i\leftarrow\mathrm{oSign}_1^{(\mathsf{w})}(i,\mathrm{SS},\mu)$ and $\mathsf{st}_i.\mathsf{P}(\mathsf{st}_i.\mathsf{cnt}) = (0,(\perp,\perp,\mathbf{w}_i))$. In addition, $\mathcal{B}$ sets $\mathrm{sid}_{i,\mathrm{SS},\mu}=\mathsf{st}_i.\mathsf{cnt}$.

$\mathsf{SampY}(i,\mathrm{SS},\mu)$: Same as $\mathbf{G}_5$ except that when $\mathrm{SS}\setminus\mathsf{CS}\subseteq\mathrm{curMS}_1(\mathrm{SS},\mu)$, $\mathcal{B}$ computes $\mathbf{Y}_{i,\mathrm{SS},\mu}$ as follows. $\mathcal{B}$ queries $\bar{\mathbf{Y}}\leftarrow\mathrm{oSign}_1^{(\mathsf{Y})}(\mathrm{SS},\mu)$. Then, $\mathcal{B}$ computes $\mathbf{Y}_{i,\mathrm{SS},\mu} = \bar{\mathbf{Y}} - \sum_{j\in\mathsf{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot\mathbf{U}\mathbf{S}_j - \sum_{j\in(\mathrm{SS}\setminus\mathsf{CS})\setminus\{i\}} \mathbf{Y}_{j,\mathrm{SS},\mu} - \sum_{j\in\mathsf{SS}\cap\mathsf{CS}}\mathsf{mask}_{j,\mathrm{SS},\mu}$.

$\widetilde{\mathbf{oSign}}_2(i,\mathrm{sid},\mathrm{SS},\mu,M)$: Same as oSign_2 in $\mathbf{G}_5$ except that $\mathcal{B}$ records $\mathrm{sid}_{i,\mathrm{SS},\mu,M}=\mathrm{sid}$ given the query is valid (i.e. the oracle does not return $\perp$) and when $\mathrm{SS}\setminus\mathsf{CS}\subseteq\mathrm{curMS}_2(\mathrm{SS},\mu,M)$, $\mathcal{B}$ computes $\mathbf{z}_{i,\mathrm{SS},\mu,M}$ as follows. $\mathcal{B}$ computes $Q=\{(j,\mathrm{sid}_{j,\mathrm{SS},\mu,M})\}_{j\in\mathrm{SS}\setminus\mathsf{CS}}$ and queries $\bar{\mathbf{z}}\leftarrow\mathrm{oSign}_2(\mathrm{SS},Q,\mu,\mathbf{Y},\mathbf{w})$. Then, $\mathcal{B}$ computes $\mathbf{z}_{i,\mathrm{SS},\mu,M} = \bar{\mathbf{z}} - \sum_{j\in\mathsf{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot\mathbf{S}_j\mathbf{c} - \sum_{j\in(\mathrm{SS}\setminus\mathsf{CS})\setminus\{i\}}\mathbf{z}_{j,\mathrm{SS},\mu,M} - \sum_{j\in\mathsf{SS}\cap\mathsf{CS}}\mathsf{mask}_{j,\mathrm{SS},\mu,M}$.

$\widetilde{\mathbf{Cor}}(i)$: $\mathcal{B}$ samples $\mathbf{S}_i\leftarrow\mathcal{R}_q^{m\times k}$ and sets $\mathsf{st}_i.\mathsf{sk}=(\mathbf{S}_i,\{\mathsf{seed}_{i,j},\mathsf{seed}_{j,i}\}_{j\in[N]})$. Then, $\mathcal{B}$ queries $\bar{\mathsf{st}}_i\leftarrow\mathrm{Cor}(i)$ and sets $\mathsf{st}_i.\mathsf{P}=\bar{\mathsf{st}}_i.\mathsf{P}$. For each $(\mathrm{SS},\mu)\in\mathrm{setQ}_1$, $\mathcal{B}$ sets $(\mathbf{E}'_{i,\mathrm{SS},\mu},\mathbf{r}_i,\mathbf{w}_i)=\mathsf{st}_i.\mathsf{P}(\mathrm{sid}_{i,\mathrm{SS},\mu})$ and $H_5(\mathsf{seed}_{i,i},\mathsf{pk},\mathrm{SS},\mu)=\mathbf{E}'_{i,\mathrm{SS},\mu}$. Note that since BadSeed did not occur, $H_5(\mathsf{seed}_{i,i},\mathsf{pk},\mathrm{SS},\mu)$ was not defined prior to the query. For each $(\mathrm{SS},\mu,M)\in\mathrm{setQ}_2$, $\mathcal{B}$ sets $(\mathbf{E}'_i,\mathbf{r}_{i,\mathrm{SS},\mu,M},\mathbf{w}_i)=\mathsf{st}_i.\mathsf{P}(\mathrm{sid}_{i,\mathrm{SS},\mu,M})$. Finally, $\mathcal{B}$ executes as $\mathrm{Cor}(i)$ in $\mathbf{G}_5$ except that $\mathcal{B}$ queries $\widetilde{\mathrm{RO}}(x)$ instead of $\mathrm{RO}(x)$ and the execution of the algorithms $\mathsf{CompMask}_1$ and $\mathsf{CompMask}_2$ is changed as described below.

$\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)$: Same as $\mathbf{G}_5$ except that when $\mathrm{SS} \setminus \mathsf{CS} \subseteq \mathrm{curMS}_1(\mathrm{SS}, \mu)$, $\mathcal{B}$ computes $\mathbf{Y}_{i,\mathrm{SS},\mu}$ as follows. $\mathcal{B}$ queries $\bar{\mathbf{Y}} \leftarrow \mathrm{oSign}_1^{(\mathsf{Y})}(\mathrm{SS}, \mu)$. Then, $\mathcal{B}$ computes $\mathsf{mask}_{i,\mathrm{SS},\mu} = \bar{\mathbf{Y}} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}\setminus\{i\}} L_{\mathrm{SS},j} \cdot \mathbf{US}_j - \sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{Y}_{j,\mathrm{SS},\mu} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu}$.

$\mathsf{CompMask}_2(i, \mathrm{sid}, \mathrm{SS}, \mu, M)$: Same as $\mathbf{G}_5$ except when $\mathrm{SS} \setminus \mathsf{CS} \subseteq \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$, $\mathcal{B}$ computes $\mathbf{z}_{i,\mathrm{SS},\mu,M}$ as follows. $\mathcal{B}$ computes $Q = \{(j, \mathrm{sid}_{j,\mathrm{SS},\mu,M})\}_{j\in\mathrm{SS}\setminus\mathsf{CS}}$ and queries $\bar{\mathbf{z}} \leftarrow \mathrm{oSign}_2(\mathrm{SS}, Q, \mu, \mathbf{Y}, \mathbf{w})$. Then, $\mathcal{B}$ computes the mask $\mathsf{mask}_{i,\mathrm{SS},\mu,M} = \bar{\mathbf{z}} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}\setminus\{i\}} L_{\mathrm{SS},j} \cdot \mathbf{S}_j\mathbf{c} - \sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{z}_{j,\mathrm{SS},\mu,M} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu,M}$.

After $\mathcal{A}$ returns a forgery (μ^*, sig^*), $\mathcal{B}$ outputs (μ^*, sig^*).

Analysis of $\mathcal{B}$. We first show that all the oracle queries made by $\mathcal{B}$ are valid, i.e., the queries do not return $\perp$. By the simulation, in $\mathsf{CompMask}_1$ and SampY, when $\mathcal{B}$ makes a query (SS, μ) to $\mathrm{oSign}_1^{(\mathsf{Y})}$, we have $\mathrm{SS} \setminus \mathsf{CS} \subseteq \mathrm{curMS}_1(\mathrm{SS}, \mu)$, which means $\mathcal{B}$ made a query (j, SS, μ) to $\mathrm{oSign}_1^{(\mathsf{w})}$ for each $j \in \mathrm{SS} \setminus \mathsf{CS}$. Note that the set of corrupted signers in the game simulated by $\mathcal{B}$ is the same as CS defined in the IUF game. Therefore, the query $\mathrm{oSign}_1^{(\mathsf{Y})}(\mathrm{SS}, \mu)$ does not return $\perp$.

When $\mathcal{B}$ makes a query $(\mathrm{SS}, Q, \mu, \mathbf{Y}, \mathbf{w})$ to oSign_2, since $\mathrm{SS} \setminus \mathsf{CS} \subseteq \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$, we know that for each $j \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{sid}_{j,\mathrm{sid}_{j,\mathrm{SS},\mu,M}})$ is defined and is bounded by $\mathsf{st}_j.\mathsf{cnt}$. Also, by the simulation, for any $(\mathrm{SS}', \mu', M') \neq (\mathrm{SS}, \mu, M)$, $\mathrm{sid}_{j,\mathrm{SS}',\mu',M'} \neq \mathrm{sid}_{j,\mathrm{sid}_{j,\mathrm{SS},\mu,M}}$, since only one of the oSign_2 queries with $\mathrm{sid} = \mathrm{sid}_{j,\mathrm{SS},\mu,M}$ does not return $\perp$. Also, $\mathcal{B}$ makes only one query to oSign_2 for each (SS, μ, M). Therefore, $(\mathrm{SS}, Q, \mu, \mathbf{Y}, \mathbf{w})$ is a valid query to oSign_2.

We now show that $\mathcal{B}$ computes the values $\mathbf{Y}_{i,\mathrm{SS},\mu}$, $\mathbf{z}_{i,\mathrm{SS},\mu,M}$ and the masks correctly. From the definition of the IUF game (Figure 7), $\bar{\mathbf{Y}} = \mathbf{US} + \sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu}$. Therefore, by Equations (6) and (7), $\mathbf{Y}_{i,\mathrm{SS},\mu}$ and $\mathsf{mask}_{i,\mathrm{SS},\mu}$ are computed correctly. Similarly, since $\bar{\mathbf{z}} = \sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{r}_{j,\mathrm{SS},\mu,M} + \mathbf{Sc}$, where $\mathbf{r}_{j,\mathrm{SS},\mu,M}$ is the $\mathbf{r}_j$ value stored in $\mathsf{st}_i.\mathsf{P}(\mathrm{sid}_{i,\mathrm{SS},\mu,M})$ in the IUF game, from the description of $\mathbf{G}_5$, $\mathbf{z}_{i,\mathrm{SS},\mu,M}$ and $\mathsf{mask}_{i,\mathrm{SS},\mu,M}$ are computed correctly. Therefore, $\mathcal{B}$ simulates the $\mathbf{G}_5$ game perfectly.

The number of $\mathrm{oSign}_1^{(\mathsf{w})}$ queries made by $\mathcal{B}$ is at most Q_s, and the number of random oracle queries made by $\mathcal{B}$ is at most Q_h. It is left to show that if $\mathcal{A}$ wins the $\mathbf{G}_5$ simulated by $\mathcal{B}$, $\mathcal{B}$ also wins the IUF game. Since $\mathcal{B}$ makes a query to $\mathrm{oSign}_1^{(\mathsf{w})}(\mu)$ if and only if $\mathcal{A}$ made a oSign_1 query on μ, the set S defined in IUF is identical to the set S defined in $\mathbf{G}_5$, which concludes the lemma. □

5.2 Proof of Lemma 17

To reduce IUF to MSIS and MLWE, the main difference from the single-signer setting is that the adversary may learn part of the internal randomness by corrupting signers. Concretely, in $\mathrm{oSign}_1^{(\mathsf{Y})}$ the adversary first observes $\mathbf{Y} = \mathbf{US} + \sum_{i\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_i$, and may then adaptively choose to reveal some $\mathbf{E}'_i$ by corrupting the corresponding signer i. Similarly, in oSign_2 the adversary first observes $\mathbf{z} = \sum_{i\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{r}_i + \mathbf{Sc}$, and may subsequently reveal some $\mathbf{r}_i$.

This adaptivity affects precisely the step in the single-signer proof (cf. Theorem 2) where $\mathbf{E}'_i$ and $\mathbf{r}_i$ are used to noise-flood the information about $\mathbf{S}$, namely Lemma 14 relating $\mathbf{G}_6$ and $\mathbf{G}_7$. Instead, we perform a careful analysis of the relevant conditional distributions, relying on discrete-Gaussian tools developed in [GMPW20, CATZ24]; see Lemma 18.

Game sequence. We begin by describing the sequence of games. We write Adv_x for the adversary's advantage in G_x. Starting from G_2, each game additionally takes as input a parameter $\epsilon \leq \mathsf{Adv}_1 \leq 2\epsilon$. As in Theorem 2, we also adopt the same simplifying assumptions on the adversary $\mathcal{A}$.

$\mathbf{G}_0$: Identical to the original game in Fig. 7. We denote the message–signature pair output by the adversary by $(\mu^*, sig^* = (\mathbf{Y}^*, \mathbf{w}^*, \mathbf{z}^*))$.

$\mathbf{G}_1$-$\mathbf{G}_5$: Identical to the games in the proof of Theorem 2

$\mathbf{G}_6$: Same as $\mathbf{G}_5$, but for each query (i, SS, μ) to $\mathrm{oSign}_1^{(\mathsf{w})}$, we replace $\underline{\mathbf{w}}_i$ with $\big((\mathbf{TA} + \mathbf{E}'')\mathbf{r}_i^{(0)} + \mathbf{e}_i\big)$, where $\mathbf{TA} + \mathbf{E}'' = \mathbf{U} = H_1(\mu)$. We relate Adv_5 to Adv_6 in Lemma 18.

$\mathbf{G}_7$: Same as $\mathbf{G}_6$, but for each valid query (SS, μ) to $\mathrm{oSign}_1^{(\mathsf{Y})}$, we replace $\mathbf{Y} = \mathbf{US} + \sum_{i\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu}$ with $\big((\mathbf{TA}+\mathbf{E}'')\mathbf{S}^{(0)} + \sum_{i\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu}\big)$, where $\mathbf{TA}+\mathbf{E}'' = \mathbf{U} = H_1(\mu)$. We relate Adv_6 to Adv_7 in Lemma 18.

$\mathbf{G}_8$: Same as $\mathbf{G}_7$, but for each valid query $(\mathrm{SS}, \{(i, \mathrm{sid}_i)\}_{i\in\mathrm{SS}\setminus\mathsf{CS}}, \mu, \mathbf{Y}, \mathbf{w})$ to oSign_2, we replace $\mathbf{z} = \sum_{i\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{r}_i + \mathbf{Sc}$ with $\sum_{i\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{r}_i + \mathbf{S}^{(0)}\mathbf{c}$, where $\mathbf{r}_i$ is the randomness used by $\mathrm{oSign}_1^{(\mathsf{w})}$ to generate $\mathbf{w}_i$, and $\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$. We relate Adv_7 to Adv_8 in Lemma 18.

$\mathbf{G}_9$**:** Identical to the $\mathbf{G}_8$ in the proof of Theorem 2

Finally: We bound Adv_9 in Lemma 15. Now we derive the final bounds in Lemma 17. From $\mathbf{G}_1$ to $\mathbf{G}_5$, we have

$$\mathsf{Adv}_0 \leq 32^{\frac{1}{3}} Q_h^2 \cdot \left(\mathsf{Adv}^{\mathsf{hnf\text{-}msis}}_{q,d,\ell,m,4L_1^{(1)}}(\mathcal{A}',\kappa) + 8(k+1)\cdot 2^{-6\kappa}\right)^{\frac{1}{3}},$$

or,

$$\mathsf{Adv}_0 \leq 8Q_h^2\Big(\mathsf{Adv}_5 + t\cdot Q_h \cdot \mathsf{Adv}^{\mathsf{hnf\text{-}mlwe}}_{q,d,m,\ell+m,\beta_{\mathsf{hlwe}}}(\mathcal{B},\kappa) + (m-\ell-n)\cdot \mathsf{Adv}^{\mathsf{mlwe}}_{q,d,\ell,n+\ell,\beta_{\mathsf{lwe}}}(\mathcal{C},\kappa) + 2^{-2\kappa}\Big).$$

Moreover, from $\mathbf{G}_5$ to $\mathbf{G}_8$ we obtain

$$\begin{aligned}
\mathsf{Adv}_5 &\leq e^{(\alpha-1)\pi}\mathsf{Adv}_6^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa},\\
\mathsf{Adv}_6 &\leq 4e^{(\alpha-1)\pi}\mathsf{Adv}_7^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa},\\
\mathsf{Adv}_7 &\leq 4e^{(\alpha-1)\pi}\mathsf{Adv}_8^{\frac{\alpha-1}{\alpha}} + k\cdot 2^{-4\kappa},\\
\mathsf{Adv}_8 &\leq \mathsf{Adv}_9 + 2k\cdot 2^{-8\kappa},\\
\mathsf{Adv}_9 &\leq \frac{48\kappa}{\epsilon^2}\cdot 2^{-8\kappa},
\end{aligned}$$

where $\epsilon \leq Q_h^{-2}\cdot \mathsf{Adv}_0 \leq 2\epsilon$. Setting $\alpha = 5$, we have

$$\mathsf{Adv}_5 \leq +16e^{12\pi}\sqrt{48\kappa}\cdot\frac{2Q_h^2}{\mathsf{Adv}_0}\cdot 2^{-4\kappa} + (16e^{12\pi}\sqrt{2k}+k+Q_s)\cdot 2^{-4\kappa} + e^{4\pi}(k+Q_s)\cdot 2^{-3\kappa} + 4e^{8\pi}\cdot k\cdot 2^{-2\kappa}.$$

Finally, if $\mathsf{Adv}_0 > 2^{-k}$, then $\frac{1}{\mathsf{Adv}_0} \leq 2^{\kappa}$. Substituting this bound into the expression for Adv_5 gives the claimed bound. □

Lemma 18 (Bounding $\mathsf{Adv}_{\{5,6,7\}}$ in IUF). *Assume that $\sigma_{r_1} \geq \max(a_3\sqrt{Q_s}, 2\eta_\varepsilon(\mathcal{R}^{m-\ell-n})), \sigma_1 \geq \max(a_1\sqrt{kQ}, 2\cdot\eta_\varepsilon(\mathcal{R}^t))$, and $\sigma_2 \geq a_2\sqrt{Q_s}$, then we have*

- $\mathsf{Adv}_5 \leq e^{(\alpha-1)\pi}\mathsf{Adv}_6^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa}$.
- $\mathsf{Adv}_6 \leq 4e^{(\alpha-1)\pi}\cdot\mathsf{Adv}_7^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa}$.
- $\mathsf{Adv}_7 \leq 4e^{(\alpha-1)\pi}\cdot\mathsf{Adv}_8^{\frac{\alpha-1}{\alpha}} + k\cdot 2^{-4\kappa}$.

where $a_1 \geq \|\mathbf{E}''\mathbf{D}\mathbf{S}^{(1)}\|_{1,2}, a_2 \geq \|\mathbf{E}''\mathbf{D}\mathbf{r}_j^{(1)}\|, a_3 \geq \|\mathbf{S}^{(1)}\mathbf{c}_j\|$ are defined in the proof and $Q = 2Q_s + 2$.

Proof. In $\mathbf{G}_5$, using $\mathbf{AS} = \mathbf{AS}^{(0)}$ and $\mathbf{Ar}_j = \mathbf{Ar}_j^{(0)}$, we can write

$$\begin{aligned}
\underline{\mathbf{w}}_i &= (\mathbf{TA}+\mathbf{E}'')\mathbf{r}_i^{(0)} + \boxed{\mathbf{e}_i + \mathbf{E}''\mathbf{D}\mathbf{r}_i^{(1)}},\\
\mathbf{Y} &= (\mathbf{TA}+\mathbf{E}'')\mathbf{S}^{(0)} + \boxed{\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i + \mathbf{E}''\mathbf{D}\mathbf{S}^{(1)}},\\
\mathbf{z} &= \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{r}_i^{(0)} + \mathbf{S}^{(0)}\mathbf{c} + \mathbf{D}\cdot\Big(\boxed{\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{r}_i^{(1)} + \mathbf{S}^{(1)}\mathbf{c}}\Big),
\end{aligned}$$

where $\mathbf{TA}+\mathbf{E}'' = H_1(\mu)$ and $\mathbf{c} = H_2(\mu,\mathbf{Y},\mathbf{w})$.

In $\mathbf{G}_8$, the boxed terms are replaced by $\mathbf{e}_i$, $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i$, and $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{r}_i^{(1)}$, respectively. We implement this transition via noise flooding in three steps:

- $\mathbf{G}_6$: Replace $\mathbf{e}_i + \mathbf{E}''\mathbf{D}\mathbf{r}_i^{(1)}$ by $\mathbf{e}_i$, so that $\mathbf{r}_i^{(1)}$ is perfectly hidden.
- $\mathbf{G}_7$: Replace $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i + \mathbf{E}''\mathbf{D}\mathbf{S}^{(1)}$ by $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i$.
- $\mathbf{G}_8$: Replace $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{r}_i^{(1)} + \mathbf{S}^{(1)}\mathbf{c}$ by $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{r}_i^{(1)}$.

As in Lemma 14, except with probability $(k+Q_s)\cdot 2^{-4\kappa}$, here we used $\varepsilon \leq 2^{-4\kappa}$, then we have the following bounds:

– $\|\mathbf{E}''\mathbf{D}\mathbf{S}^{(1)}\|_{1,2} \leq \sigma_{S_1}\beta_{\mathsf{lwe}}\beta_{\mathsf{hlwe}} m(m-\ell-n)d^2\sqrt{td} = a_1$,
– $\|\mathbf{E}''\mathbf{D}\mathbf{r}_i^{(1)}\| \leq \sigma_{r_1}\beta_{\mathsf{lwe}}\beta_{\mathsf{hlwe}} m(m-\ell-n)d^2\sqrt{td} = a_2$,
– $\|\mathbf{S}^{(1)}\mathbf{c}\| \leq \sigma_{S_1}\sqrt{(m-\ell-n)d}\cdot\beta_c\cdot k = a_3$.

In $\mathbf{G}_6$: the proof is identical to Lemma 14. If we set $\sigma_2 \geq a_2\sqrt{Q_s}$, then by Lemma 5 we obtain

$$\mathsf{Adv}_5 \leq e^{(\alpha-1)\pi}\mathsf{Adv}_6^{\frac{\alpha-1}{\alpha}} + (k+Q_s)\cdot 2^{-4\kappa}.$$

In $\mathbf{G}_7$: Here we remove the term $\mathbf{E}''\mathbf{D}\mathbf{S}^{(1)}$ from $\mathbf{Y}$. By the data-processing inequality in Lemma 10, it suffices to bound $R_\alpha(P_1^{(0)},\ldots,P_Q^{(0)}\|P_1^{(1)},\ldots,P_Q^{(1)})$, where $Q = 2Q_s+2$ is an upper bound on the number of queries the adversary can make to the $\textsc{oSign}_1^{(\mathsf{W})}$ and $\textsc{Cor}$ oracles[6] in the IUF game of Fig. 7. Here $P_i^{(0)}$ (resp. $P_i^{(1)}$) denotes the transcript received by the adversary in $\mathbf{G}_6$ (resp. $\mathbf{G}_7$), and $P_i^{(0)}$ (resp. $P_i^{(1)}$) is either

$$\mathbf{Y} = (\mathbf{T}\mathbf{A}+\mathbf{E}'')\mathbf{S}^{(0)} + \mathbf{E}''\mathbf{D}\mathbf{S}^{(1)} + \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i \quad (\text{resp. } \mathbf{Y} = (\mathbf{T}\mathbf{A}+\mathbf{E}'')\mathbf{S}^{(0)} + \sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i)$$

from $\textsc{oSign}_1^{(\mathsf{Y})}(\mathrm{SS},\mu)$, or a value $\mathbf{E}'_i$ returned by $\textsc{Cor}(i)$.

By Lemma 11, we have

$$R_\alpha(P_1^{(0)},\ldots,P_Q^{(0)}\|P_1^{(1)},\ldots,P_Q^{(1)}) \leq \prod_{i\in[Q]}\max_{x_{[i-1]}} R_\alpha(P^{(0)}_{i|x_{[i-1]}}\|P^{(1)}_{i|x_{[i-1]}}).$$

If $P^{(0)}_{i|x_{[i-1]}}$ (resp. $P^{(1)}_{i|x_{[i-1]}}$) corresponds to $\mathbf{Y}$, then by Lemma 9, each column of $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i+\mathbf{E}''\mathbf{D}\mathbf{S}^{(1)}$ (resp. $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i$) is close to the distribution $\overset{\varepsilon'}{\approx}\mathcal{D}_{\mathcal{R}^t,\sqrt{t'}\cdot\sigma_1,\mathbf{c}_0}$ (resp. $\mathcal{D}_{\mathcal{R}^t,\sqrt{t'}\cdot\sigma_1}$), where $1\leq|\mathsf{SS}\setminus\mathsf{CS}| = t' \leq N$, $\|\mathbf{c}_0\|\leq a_1$, and $\varepsilon' = \frac{2\varepsilon}{1-\varepsilon}$. Therefore, by the data-processing and weak triangle inequalities from Lemma 10 and Lemma 5, we have

$$R_\alpha(P^{(0)}_{i|x_{[i-1]}}\|P^{(1)}_{i|x_{[i-1]}}) \leq (1+\varepsilon')^{k\cdot\frac{2\alpha-1}{\alpha-1}}\cdot\exp(\alpha\pi k\cdot\frac{a_1^2}{\sigma_1^2}).$$

If $P^{(0)}_{i|x_{[i-1]}}$ (resp. $P^{(1)}_{i|x_{[i-1]}}$) corresponds to $\mathbf{E}'_i$, where i belongs to a previous $\mathsf{SS}\setminus\mathsf{CS}$ with $|\mathsf{SS}\setminus\mathsf{CS}| = t'\leq N$, and assuming this is the k-th opening $\mathbf{E}'_i$ in $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i+\mathbf{E}''\mathbf{D}\mathbf{S}^{(1)}$ (resp. $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{E}'_i$), then by Lemma 9 the conditional distribution of each column of $\mathbf{E}'_i$ is close to $\overset{\varepsilon''}{\approx}\mathcal{D}_{\mathcal{R}^t,\sigma',\mathbf{c}_0}$ (resp. $\mathcal{D}_{\mathcal{R}^t,\sigma',\mathbf{c}_1}$), where $\sigma' = \sqrt{\frac{\sigma_1^2}{1+\frac{1}{t'-k}}}$, $\|\mathbf{c}_0-\mathbf{c}_1\|\leq\frac{1}{t'-k+1}\cdot a_1$, and $\varepsilon'' = \frac{2((1+\varepsilon)^{t'-k-1}-1)}{2-(1+\varepsilon)^{t'-k-1}}$. Thus, by the data-processing and weak triangle inequalities from Lemma 10 and Lemma 5, we have

$$R_\alpha(P^{(0)}_{i|x_{[i-1]}}\|P^{(1)}_{i|x_{[i-1]}}) \leq (1+\varepsilon'')^{\frac{2\alpha-1}{\alpha-1}}\cdot(\frac{1+\varepsilon}{1-\varepsilon})^{\frac{\alpha}{\alpha-1}}\cdot\exp(\alpha\pi\cdot\frac{a_1^2}{\sigma_1^2}).$$

Consequently,

$$R_\alpha(P_1^{(0)},\ldots,P_Q^{(0)}\|P_1^{(1)},\ldots,P_Q^{(1)}) \leq (1+\varepsilon'')^{2Qk}\cdot(\frac{1+\varepsilon}{1-\varepsilon})^{2kQ}\cdot\exp(\alpha\pi kQ\cdot\frac{a_1^2}{\sigma_1^2}).$$

Setting $\sigma_1\geq\max(a_1\sqrt{kQ},2\cdot\eta_\varepsilon(\mathcal{R}^t))$, and $\varepsilon\leq 2^{-4\kappa}$, $\varepsilon\cdot N^2<1$, we have $(1+\varepsilon)^{t'-k-1}\leq 1+(t'-k)\varepsilon$ and $\varepsilon''\leq 8(t'-k)\cdot 2^{-4\kappa}\leq 8N\cdot 2^{-4\kappa}$, so that $(1+\varepsilon'')^{2kQ}\leq 1+16N\cdot kQ\cdot 2^{-4\kappa}$ and $(\frac{1+\varepsilon}{1-\varepsilon})^{2kQ}\leq 1+4(2kQ+1)\cdot 2^{-4\kappa}$. if we choose κ such that $1+16N\cdot kQ\cdot 2^{-4\kappa}\leq 2$. then we obtain $\mathsf{Adv}_6\leq 4e^{(\alpha-1)\pi}\cdot\mathsf{Adv}_7^{\frac{\alpha-1}{\alpha}}+(k+Q_s)\cdot 2^{-4\kappa}$.

In $\mathbf{G}_8$: Similarly, we analyze $\sum_{i\in\mathsf{SS}\setminus\mathsf{CS}}\mathbf{r}_i^{(1)}+\mathbf{S}^{(1)}\mathbf{c}$ versus $\mathbf{r}_i^{(1)}$ using the same approach. If we set $\sigma_{r_1}\geq\max(a_3\sqrt{Q},2\eta_\varepsilon(\mathcal{R}^{m-\ell-n}))$, then

$$\mathsf{Adv}_7\leq 4e^{(\alpha-1)\pi}\cdot\mathsf{Adv}_8^{\frac{\alpha-1}{\alpha}}+k\cdot 2^{-4\kappa}.$$

where we used $1+16N\cdot kQ\cdot 2^{-4\kappa}\leq 2$, then we obtain. □

[6] Although each time $\textsc{Cor}$ returns st_i of signer i, we count it as $\mathsf{st}_i.\mathsf{cnt}$ queries, since all states in $\mathsf{st}_i.\mathsf{P}(\mathrm{sid})$ for sid $\leq\mathsf{st}_i.\mathsf{cnt}$ are leaked.

κ	$\log_2(q)$	m	ℓ	σ_S	σ_r	σ_1	σ_2	$\lvert\mathsf{pk}\rvert$	$\lvert sig\rvert$	Comm.
128	106	26	10	2^{20}	2^{55}	2^{60}	2^{96}	67.97KB	176.72KB	176.72KB

Fig. 8. The concrete parameters and estimated efficiency of Tweed for $\kappa = 128$ and $N = 1024$. We set $(d, \beta_c) = (512, 64)$. The last column denotes the communication complexity per signer.

5.3 Parameter selection

Denote β_{lwe}, β_{hlwe} as the norm of the underlying MLWE, HNF-MLWE assumption, respectively. We set the parameters as follows.

- We set $d \geq 2\kappa$ to be a power of 2, $t = 1$, and $k = 1$
- We set $\mathcal{C} := \{c \in \mathcal{R} : \|c\|_\infty = 1, \|c\|_1 = \beta_c\}$ and β_c as the smallest integer such that $(2^{\beta_c}\binom{d}{\beta_c})^k \geq 2^{8\kappa}$.
- We set $\sigma_{S_1} \geq 8q^{\frac{t}{m-\ell-n}} \cdot \sqrt{d\log(2(m-\ell-n)d(1+1/\varepsilon'))/\pi}$, where $\varepsilon' = 2^{-8\kappa}$.
- By Lemma 18, we set $\sigma_{r_1} \geq \max(\sigma_{S_1}\beta_c k\sqrt{(m-\ell-n)dQ_s}, \sqrt{2md} \cdot \eta_\varepsilon(\mathcal{R}^m))$, where the first term is dominated, $\sigma_1 \geq \max(\sigma_{S_1}\beta_{\mathsf{lwe}}\beta_{\mathsf{hlwe}}m(m-\ell-n)d^2\sqrt{tdkQ_s}, \sqrt{2md} \cdot \eta_\varepsilon(\mathcal{R}^t))$, where $\varepsilon = 2^{-6\kappa}$ and the first term is dominated.
- By Lemma 18, we set $\sigma_2 \geq \max(\sigma_{r_1}\beta_{\mathsf{lwe}}\beta_{\mathsf{hlwe}}m(m-\ell-n)d^2\sqrt{tdQ_s}, \eta_\varepsilon(\mathcal{R}^t))$, where the first term is dominated.
- We set $\sigma_S \geq \sigma_{S_1}\beta_{\mathsf{lwe}}d\sqrt{2(m-\ell-n)m}$ and $\sigma_r \geq \sigma_{r_1}\beta_{\mathsf{lwe}}d\sqrt{2(m-\ell-n)m}$.
- To guarantee correctness, we set $L_1^{(1)} \geq \sqrt{mdN}(\sigma_r + \beta_c k\sigma_S)$ and $L_2^{(1)} \geq \sqrt{tdN}(\sigma_2 + \beta_c k\sigma_1)$.
- Finally, we find $(q, n, m, \ell, \beta_{\mathsf{lwe}}, \beta_{\mathsf{hlwe}})$ such that $(q/L_2^{(1)})^{dt} \geq 2^{8\kappa}$, and $Q_h^6 \cdot \mathsf{Adv}^{\mathsf{hnf\text{-}msis}}_{q,d,\ell,m,4L_1^{(1)}}(\mathcal{A}', \kappa) \leq 2^{-3\kappa}$, $Q_h^3 \cdot \mathsf{Adv}^{\mathsf{hnf\text{-}mlwe}}_{q,d,m,\ell+m,\beta_{\mathsf{hlwe}}}(\mathcal{B}, \kappa) \leq 2^{-\kappa}$, and $Q_h^2 \cdot \mathsf{Adv}^{\mathsf{mlwe}}_{q,d,\ell,n,\beta_{\mathsf{lwe}}}(\mathcal{C}, \kappa) \leq 2^{-\kappa}$.

Optimizations for $\mathbf{z}$. Denote $\mathbf{z} = \binom{\overline{\mathbf{z}}}{\underline{\mathbf{z}}}$. Then, the signature only needs to contain $\overline{\mathbf{z}}$ instead of all of $\mathbf{z}$. This is because $\mathbf{U}\mathbf{z} = \mathbf{U}'\overline{\mathbf{z}} + \mathbf{U}''\underline{\mathbf{z}}$. And $\mathbf{A}\mathbf{z} = \mathbf{A}'\overline{\mathbf{z}} + \underline{\mathbf{z}}$. So instead of checking $\mathbf{A}\mathbf{z} = \overline{\mathbf{w}} + \mathbf{P}\mathbf{c}$, we simply compute $\underline{\mathbf{z}} := \mathbf{A}'\overline{\mathbf{z}} - (\overline{\mathbf{w}} + \mathbf{P}\mathbf{c})$ and check $\|\underline{\mathbf{z}}\| + \|\overline{\mathbf{z}}\| \leq L_1^{(1)}$, and $\|\mathbf{U}'\overline{\mathbf{z}} + \mathbf{U}''\underline{\mathbf{z}} - \underline{\mathbf{w}} - \mathbf{Y}\mathbf{c}\| \leq L_2^{(1)}$. Given this optimization, both the final signature size and the per-signer communication complexity are reduced to $dm\log_2(q)$ bits, achieving a saving of $d\ell\log_2(q)$ bits.

Concrete parameters. We provide a set of concrete parameters in Figure 8 based on the parameter selection described above except that we are not choosing the MSIS and MLWE parameters according to the concrete bounds from the reduction, but rather we are choosing parameters so that MSIS and MLWE gives κ bits of security. This follows common practice, and it is justified by the fact that our bound is likely not tight. Here we consider $\kappa = 128$, $Q_s = 2^{60}$ and $N = 1024$. The hardness of MLWE and MSIS is estimated using the state-of-art tool, lattice estimator.[7]

Acknowledgments

We thank the reviewers for meticulous and constructive feedback, as well as Divesh Aggarwal for his support. This work was done in part while S.T. and H.W. were visiting the Simons Institute for the Theory of Computing (supported in part by a grant from the UC Noyce Initiative), and while K.J. was visiting the National University of Singapore (supported by NRF investigatorship grant NRF-NRFI09-0005), and in part at Tsinghua University, supported by the Breakthrough Plan of the Ministry of Education, China (No. JYB2025XDXM114). S.T. was also partially supported by NSF grants CNS-2026774, CNS-2154174, CNS-2426905, a gift from Microsoft, and a Stellar Development Foundation Academic Research Award.

References

ACPS09. Benny Applebaum, David Cash, Chris Peikert, and Amit Sahai. Fast cryptographic primitives and circular-secure encryption based on hard learning problems. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 595–618. Springer, Berlin, Heidelberg, August 2009. 9

[7] https://github.com/malb/lattice-estimator

AGHS13. Shweta Agrawal, Craig Gentry, Shai Halevi, and Amit Sahai. Discrete Gaussian leftover hash lemma over infinite domains. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part I*, volume 8269 of *LNCS*, pages 97–116. Springer, Berlin, Heidelberg, December 2013. 7

AMR25. Damiano Abram, Giulio Malavolta, and Lawrence Roy. Succinct oblivious tensor evaluation and applications: Adaptively-secure laconic function evaluation and trapdoor hashing for all circuits. In *STOC*, 2025. 2

ASY22. Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPIcs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022. 1, 2, 9

BCdP+25. Giacomo Borin, Sofia Celi, Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Threshold signatures reloaded: ML-DSA and enhanced raccoon with identifiable aborts. Cryptology ePrint Archive, Paper 2025/1166, 2025. 1

BCK+22. Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022. 3

BdCE+25. Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Efficient, scalable threshold ML-DSA signatures: An MPC approach. Cryptology ePrint Archive, Paper 2025/1163, 2025. 1

BDLR25a. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold schnorr signatures from DDH. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 390–422. Springer, Cham, August 2025. 1, 2

BDLR25b. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 304–334. Springer, Cham, May 2025. 1, 2

BGG+14. Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556. Springer, Berlin, Heidelberg, May 2014. 2

BGG+18. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018. 1, 2

BGGK17. Dan Boneh, Rosario Gennaro, Steven Goldfeder, and Sam Kim. A lattice-based universal thresholdizer for cryptographic systems. Cryptology ePrint Archive, Report 2017/251, 2017. 1, 2

BKL+25. Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical two-round threshold signatures from learning with errors. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 149–164. IEEE Computer Society Press, May 2025. 1, 2, 18

BL22. Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 193–207. ACM Press, November 2022. 1

BLT+24. Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024. 1, 2, 3

BTT22. Cecilia Boschini, Akira Takahashi, and Mehdi Tibouchi. MuSig-L: Lattice-based multi-signature with single-round online phase. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 276–305. Springer, Cham, August 2022. 8

CATZ24. Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 268–302. Springer, Singapore, December 2024. 1, 2, 3, 4, 8, 22

CFGN96. Ran Canetti, Uriel Feige, Oded Goldreich, and Moni Naor. Adaptively secure multi-party computation. In *28th ACM STOC*, pages 639–648. ACM Press, May 1996. 1

Che25a. Yanbo Chen. Dazzle: Improved adaptive threshold signatures from DDH. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part III*, volume 15676 of *LNCS*, pages 233–261. Springer, Cham, May 2025. 1, 2, 3

Che25b. Yanbo Chen. Round-efficient adaptively secure threshold signatures with rewinding. *IACR Communications in Cryptology*, 2(2), 2025. 1, 2

CKGW22. Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. Two-Round Threshold Schnorr Signatures with FROST. Internet-Draft draft-irtf-cfrg-frost-10, Internet Engineering Task Force, September 2022. Work in Progress. 1

CKK+25. Elizabeth C. Crites, Jonathan Katz, Chelsea Komlo, Stefano Tessaro, and Chenzhi Zhu. On the adaptive security of FROST. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 480–511. Springer, Cham, August 2025. 1, 3

CKM23. Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023. 3

CS25. Elizabeth C. Crites and Alistair Stewart. A plausible attack on the adaptive security of threshold schnorr signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 457–479. Springer, Cham, August 2025. 1

Des88. Yvo Desmedt. Society and group oriented cryptography: A new concept. In Carl Pomerance, editor, *CRYPTO'87*, volume 293 of *LNCS*, pages 120–127. Springer, Berlin, Heidelberg, August 1988. 1

DF90. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990. 1

DKM$^+$24. Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024. 1, 2

dPENP25. Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Simple and efficient lattice threshold signatures with identifiable aborts. Cryptology ePrint Archive, Paper 2025/871, 2025. 1, 3

DPS23. Julien Devevey, Alain Passelègue, and Damien Stehlé. G+G: A fiat-shamir lattice signature based on convolved gaussians. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VII*, volume 14444 of *LNCS*, pages 37–64. Springer, Singapore, December 2023. 8

DR24. Sourav Das and Ling Ren. Adaptively secure BLS threshold signatures from DDH and co-CDH. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 251–284. Springer, Cham, August 2024. 1

EKT24. Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024. 1, 2, 3, 5

EY24. Ehsan Ebrahimi and Anshu Yadav. Strongly secure universal thresholdizer. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 207–239. Springer, Singapore, December 2024. 2

FKL18. Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. 3

GCRS25. Paul Gerhart, Davide Li Calsi, Luigi Russo, and Dominique Schröder. Fully-adaptive two-round threshold schnorr signatures from DDH. Cryptology ePrint Archive, Paper 2025/1478, 2025. 1, 3

GKS24. Kamil Doruk Gür, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 266–300. Springer, Cham, June 2024. 1, 2

GMPW20. Nicholas Genise, Daniele Micciancio, Chris Peikert, and Michael Walter. Improved discrete gaussian and subgaussian analysis for lattice cryptography. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part I*, volume 12110 of *LNCS*, pages 623–651. Springer, Cham, May 2020. 8, 22

GPV08. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008. 7

GTJ$^+$26. Liming Gao, Guofeng Tang, Dingding Jia, Yijian Liu, Bingqian Liu, Xianhui Lu, Kunpeng Wang, and Yongjian Yin. TalonG: Bandwidth-efficient two-round threshold signatures from lattices. In *EUROCRYPT*, 2026. 2

KRT24. Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024. 1, 2

LDK$^+$22. Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. 1

LS18. Vadim Lyubashevsky and Gregor Seiler. Short, invertible elements in partially splitting cyclotomic rings and applications to lattice-based zero-knowledge proofs. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 204–224. Springer, Cham, April / May 2018. 8

LW22. George Lu and Brent Waters. How to sample a discrete gaussian (and more) from a random oracle. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part II*, volume 13748 of *LNCS*, pages 653–682. Springer, Cham, November 2022. 18

Lyu12. Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, Berlin, Heidelberg, April 2012. 3, 4, 5

Mak22. Nikolaos Makriyannis. On the classic protocol for MPC schnorr signatures. Cryptology ePrint Archive, Paper 2022/1332, 2022. 1

Natnt. National Institute of Standards and Technology. Multi-Party Threshold Cryptography, 2018–Present. https://csrc.nist.gov/Projects/threshold-cryptography. 1

Nie02. Jesper Buus Nielsen. Separating random oracle proofs from complexity theoretic proofs: The non-committing encryption case. In Moti Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 111–126. Springer, Berlin, Heidelberg, August 2002. 6

Pei15. Chris Peikert. A decade of lattice cryptography. Cryptology ePrint Archive, Report 2015/939, 2015. 9

PFH+22. Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. 1

PKN+25. Rafaël Del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 423–456. Springer, Cham, August 2025. 1, 3

QWW18. Willy Quach, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and applications. In Mikkel Thorup, editor, *59th FOCS*, pages 859–870. IEEE Computer Society Press, October 2018. 2

Ros20. Mélissa Rossi. *Extended security of lattice-based cryptography*. PhD thesis, Université Paris sciences et lettres, 2020. 10

TT15. Katsuyuki Takashima and Atsushi Takayasu. Tighter security for efficient lattice cryptography via the Rényi divergence of optimized orders. In Man Ho Au and Atsuko Miyaji, editors, *ProvSec 2015*, volume 9451 of *LNCS*, pages 412–431. Springer, Cham, November 2015. 8

ZT25. Chenzhi Zhu and Stefano Tessaro. The algebraic one-more MISIS problem and applications to threshold signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part I*, volume 16000 of *LNCS*, pages 548–581. Springer, Cham, August 2025. 1, 2, 4, 6

A Deferred game figures

Game $\mathbf{G}_0^{\mathcal{A}}(\kappa)$, $\boxed{\mathbf{G}_1^{\mathcal{A}}(\kappa)}$:

Oracle $\text{oSign}_1(i, \text{SS}, \mu)$:

Require: $i \in \text{SS} \setminus \mathsf{CS}$, $\text{SS} \subseteq [N]$ and $|\text{SS}| \geq t$

$\mathsf{st}_i.\mathsf{cnt} = \mathsf{st}_i.\mathsf{cnt} + 1$

$\boxed{\begin{array}{l} \text{If } (\text{SS}, \mu) \notin \text{setQ}_1 \text{ then} \\ \quad \text{setQ}_1 \leftarrow \text{setQ}_1 \cup \{(\text{SS}, \mu)\} \\ \quad \text{curMS}_1(\text{SS}, \mu) = \emptyset \\ \quad \text{For } j \in \text{SS} \setminus \mathsf{CS} \text{ do} \\ \quad\quad \text{curMS}_1(\text{SS}, \mu) \leftarrow \text{curMS}_1(\text{SS}, \mu) \cup \{j\} \\ \quad\quad \mathsf{CompMask}_1(j, \text{SS}, \mu) \end{array}}$

$\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}$, $\mathbf{e}_i \leftarrow \mathcal{D}_{\mathcal{R}^t, \sigma_2}$

$\mathbf{U} = \text{RO}(1, \mu) \in \mathcal{R}_q^{t \times m}$

$\mathbf{E}'_i \leftarrow \text{RO}(5, \mathsf{seed}_{i,i}, \mathsf{pk}, \text{SS}, \mu)$

$\mathsf{mask}_{i,\text{SS},\mu} = \sum_{j \in \text{SS}} (\text{RO}(3, \mathsf{seed}_{i,j}, \mathsf{pk}, \text{SS}, \mu)$
$\quad - \text{RO}(3, \mathsf{seed}_{j,i}, \mathsf{pk}, \text{SS}, \mu))$

$\mathbf{Y}_i = L_{\text{SS},i} \cdot \mathbf{U}\mathbf{S}_i + \mathbf{E}'_i + \mathsf{mask}_{i,\text{SS},\mu}$

$$\mathbf{w}_i = \begin{bmatrix} \overline{\mathbf{w}}_i \\ \underline{\mathbf{w}}_i \end{bmatrix} = \begin{bmatrix} \mathbf{A}\mathbf{r}_i \\ \mathbf{U}\mathbf{r}_i + \mathbf{e}_i \end{bmatrix}$$

$\mathsf{st}_i.\mathsf{P}(\mathsf{st}_i.\mathsf{cnt}) = (0, (\mathbf{E}'_i, \mathbf{r}_i, \mathbf{w}_i))$

Return $(\mathbf{Y}_i, \mathbf{w}_i)$

Oracle $\text{oSign}_2(i, \text{sid}, \text{SS}, \mu, M)$:

Require: $i \in \text{SS} \setminus \mathsf{CS}$, $\text{sid} \leq \mathsf{st}_i.\mathsf{cnt}$,
$\quad \text{SS} \subseteq [N]$ and $|\text{SS}| \geq T$

$\{(j, \mathbf{Y}_j, \mathbf{w}_j)\}_{j \in [\text{SS}]} = M$

$(\mathsf{isUsed}, (\mathbf{E}'_i, \mathbf{r}_i, \hat{\mathbf{w}}_i)) = \mathsf{st}_i.\mathsf{P}(\text{sid})$

If isUsed or $\mathbf{w}_i \neq \hat{\mathbf{w}}_i$ then return $\perp$

$\mathsf{st}_i.\mathsf{P}(\text{sid}) = (1, (\mathbf{E}'_i, \mathbf{r}_i, \hat{\mathbf{w}}_i))$

$\boxed{\begin{array}{l} \text{If } (\text{SS}, \mu, M) \notin \text{setQ}_2 \text{ then} \\ \quad \text{setQ}_2 \leftarrow \text{setQ}_2 \cup \{(\text{SS}, \mu, M)\} \\ \quad \text{curMS}_2(\text{SS}, \mu, M) = \emptyset \\ \quad \text{For } j \in \text{SS} \setminus \mathsf{CS} \text{ do} \\ \quad\quad \text{curMS}_2(\text{SS}, \mu, M) \leftarrow \\ \quad\quad\quad \text{curMS}_2(\text{SS}, \mu, M) \cup \{j\} \\ \quad\quad \mathsf{CompMask}_2(j, \text{SS}, \mu, M) \end{array}}$

$(\mathbf{S}_i, (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [n]}) = \mathsf{sk}_i$

$\mathbf{Y} = \sum_{j \in \text{SS}} \mathbf{Y}_j$; $\mathbf{w} = \sum_{j \in \text{SS}} \mathbf{w}_j$

$\mathbf{c} = \text{RO}(2, \mu, \mathbf{Y}, \mathbf{w}) \in \mathcal{C}^k \subset R^k$

$\mathsf{mask}_{i,\text{SS},\mu,M} =$
$\quad \sum_{j \in \text{SS}} (\text{RO}(4, \mathsf{seed}_{i,j}, \mathsf{pk}, \text{SS}, \mu, M)$
$\quad - \text{RO}(4, \mathsf{seed}_{j,i}, \mathsf{pk}, \text{SS}, \mu, M))$

$\mathbf{z}_i = \mathbf{r}_i + L_{\text{SS},i} \cdot \mathbf{S}_i \mathbf{c} + \mathsf{mask}_{i,\text{SS},\mu,M}$

Return $\mathbf{z}_i$'

Fig. 9. The oracles oSign_1 and oSign_2 of the games $\mathbf{G}_0$ and $\mathbf{G}_1$, where $\mathbf{G}_0$ contains all but the solid boxes and $\mathbf{G}_1$ contains all but the dashed boxes. The $\mathsf{CompMask}$ algorithm is defined in Figure 10. Also, $\mathbf{G}_1$ maintains a set setQ_1 (resp. setQ_2) to record all first-round (resp. second-round) messages and maintains a table curMS_1 (resp. curMS_2), where the key of each entry is a first-round (resp. second-round) message and the entry records the set of signers that receive the message. The sets and the tables are initially empty. The rest of each game is the same as $\mathsf{adp\text{-}TSUF}_{\mathsf{TS}[N,T]}$.

Game $\boxed{\mathbf{G}_1^{\mathcal{A}}(\kappa)}$, $\boxed{\mathbf{G}_2^{\mathcal{A}}(\kappa)}$:

Algorithm $\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)$:

If $i \in \mathrm{curMS}_1(\mathrm{SS}, \mu)$ then return $\perp$
If $\mathrm{SS} \setminus \mathsf{CS} \not\subseteq \mathrm{curMS}_1(\mathrm{SS}, \mu)$ then
  $\mathsf{mask}_{i,\mathrm{SS},\mu} \leftarrow \mathcal{R}_q^{t\times k}$
  Pick $j \in (\mathrm{SS} \setminus \mathsf{CS}) \setminus \mathrm{curMS}_1(\mathrm{SS}, \mu)$ arbitrarily
  $H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu) = \mathsf{mask}_{i,\mathrm{SS},\mu} - \sum_{\hat{j}\in\mathrm{SS}\setminus\{j\}} \mathrm{RO}(3, \mathsf{seed}_{i,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu)$
  $+ \sum_{\hat{j}\in\mathrm{SS}} \mathrm{RO}(3, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu)$
Else
  For $j \in \mathsf{CS} \cap \mathrm{SS}$ do
    $\mathsf{mask}_{j,\mathrm{SS},\mu} \leftarrow \sum_{\hat{j}\in\mathrm{SS}}(\mathrm{RO}(3, \mathsf{seed}_{j,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu) - \mathrm{RO}(3, \mathsf{seed}_{\hat{j},j}, \mathsf{pk}, \mathrm{SS}, \mu))$
  $\mathsf{mask}_{i,\mathrm{SS},\mu} = -\sum_{j\in\mathrm{SS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu}$

Algorithm $\mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)$:

If $i \in \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ then return $\perp$
If $\mathrm{SS} \setminus \mathsf{CS} \not\subseteq \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ then
  $\mathsf{mask}_{i,\mathrm{SS},\mu,M} \leftarrow \mathcal{R}_q^{t\times k}$
  Pick $j \in \mathrm{SS} \setminus \mathsf{CS} \setminus \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ arbitrarily
  $H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M) = \mathsf{mask}_{i,\mathrm{SS},\mu,M} - \sum_{\hat{j}\in\mathrm{SS}\setminus\{i\}} \mathrm{RO}(4, \mathsf{seed}_{i,\hat{k}}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
  $+ \sum_{\hat{j}\in\mathrm{SS}} \mathrm{RO}(4, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu, M))$
Else
  For $j \in \mathsf{CS} \cap \mathrm{SS}$ do
    $\mathsf{mask}_{j,\mathrm{SS},\mu,M} \leftarrow \sum_{\hat{j}\in\mathrm{SS}}(\mathrm{RO}(4, \mathsf{seed}_{j,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
    $-\mathrm{RO}(4, \mathsf{seed}_{\hat{j},j}, \mathsf{pk}, \mathrm{SS}, \mu, M))$
  $\mathsf{mask}_{i,\mathrm{SS},\mu,M} = -\sum_{j\in\mathrm{SS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu,M}$

Oracle $\textsc{Cor}(i)$:

$\mathsf{CS} \leftarrow \mathsf{CS} \cup \{i\}$
For $(\mathrm{SS}, \mu) \in \mathrm{setQ}_1$ such that $i \in \mathrm{SS}$ do
  If $\mathrm{SS} \setminus \mathsf{CS} = \emptyset$ then continue
  Pick $j \in \mathrm{SS} \setminus \mathsf{CS}$ arbitrarily
  $H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu) = \mathsf{mask}_{i,\mathrm{SS},\mu} - \sum_{\hat{j}\in\mathrm{SS}\setminus\{i\}} \mathrm{RO}(3, \mathsf{seed}_{i,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu)$
  $+ \sum_{\hat{j}\in\mathrm{SS}} \mathrm{RO}(3, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu))$
For $(\mathrm{SS}, \mu, M) \in \mathrm{setQ}_2$ such that $i \in \mathrm{SS}$ do
  If $\mathrm{SS} \setminus \mathsf{CS} = \emptyset$ then continue
  Pick $j \in \mathrm{SS} \setminus \mathsf{CS}$ arbitrarily
  $H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M) = \mathsf{mask}_{i,\mathrm{SS},\mu,M} - \sum_{\hat{j}\in\mathrm{SS}\setminus\{i\}} \mathrm{RO}(4, \mathsf{seed}_{i,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
  $+ \sum_{\hat{j}\in\mathrm{SS}} \mathrm{RO}(4, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu, M))$
Return st_i

Fig. 10. The $\mathsf{CompMask}_1$, $\mathsf{CompMask}_2$ algorithms and the $\textsc{Cor}$ oracle of the games $\mathbf{G}_1$ and $\mathbf{G}_2$, where $\mathbf{G}_1$ contains all but the solid boxes and $\mathbf{G}_2$ contains all but the dashed boxes. The rest of $\mathbf{G}_2$ is the same as $\mathbf{G}_1$.

Game $\boxed{\mathbf{G}_2^{\mathcal{A}}(\kappa)}$, $\boxed{\mathbf{G}_3^{\mathcal{A}}(\kappa)}$:

Oracle $\textsc{Cor}(i)$:

$\mathsf{CS} \leftarrow \mathsf{CS} \cup \{i\}$; $\mathsf{HS} \leftarrow [N] \setminus \mathsf{CS}$

For $(\mathrm{SS}, \mu) \in \mathrm{setQ}_1$ such that $i \in \mathrm{SS}$ do

$\quad \boxed{\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)}$

$\quad$ If $\mathrm{SS} \setminus \mathrm{CS}$ then continue

$\quad$ Pick $j \in \mathrm{SS} \setminus \mathrm{CS}\setminus$ arbitrarily

$\quad H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu) = \mathsf{mask}_{i,\mathrm{SS},\mu} - \sum_{\hat{j} \in \mathrm{SS}\setminus\{j\}} \mathrm{RO}(3, \mathsf{seed}_{i,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu)$
$\qquad\qquad + \sum_{\hat{j} \in \mathrm{SS}} \mathrm{RO}(3, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu))$

For $(\mathrm{SS}, \mu, M) \in \mathrm{setQ}_2$ such that $i \in \mathrm{SS}$ do

$\quad \boxed{\mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)}$

$\quad$ If $\mathrm{SS} \setminus \mathrm{CS}$ then continue

$\quad$ Pick $j \in \mathrm{SS} \setminus \mathrm{CS}\setminus$ arbitrarily

$\quad H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M) = \mathsf{mask}_{i,\mathrm{SS},\mu,M} - \sum_{\hat{j} \in \mathrm{SS}\setminus\{i\}} \mathrm{RO}(4, \mathsf{seed}_{i,\hat{k}}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
$\qquad\qquad + \sum_{\hat{j} \in \mathrm{SS}} \mathrm{RO}(4, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu, M))$

Return st_i

Oracle $\textsc{oSign}_1(i, \mathrm{SS}, \mu)$:

Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$

$\mathsf{st}_i.\mathsf{cnt} = \mathsf{st}_i.\mathsf{cnt} + 1$

$\boxed{\begin{array}{l} \text{If } (\mathrm{SS}, \mu) \notin \mathrm{setQ}_1 \text{ then} \\ \quad \mathrm{setQ}_1 \leftarrow \mathrm{setQ}_1 \cup \{(\mathrm{SS}, \mu)\} \\ \quad \mathrm{curMS}_1(\mathrm{SS}, \mu) = \emptyset \\ \quad \text{For } j \in \mathrm{SS} \setminus \mathrm{CS} \text{ do} \\ \qquad \mathrm{curMS}_1(\mathrm{SS}, \mu) \leftarrow \\ \qquad\qquad \mathrm{curMS}_1(\mathrm{SS}, \mu) \cup \{i\} \\ \qquad \mathsf{CompMask}_1(j, \mathrm{SS}, \mu) \end{array}}$

$\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}, \mathbf{e}_i \leftarrow \mathcal{D}_{\mathcal{R}^t, \sigma_2}$

$\mathbf{U} = \mathrm{RO}(1, \mu) \in \mathcal{R}_q^{t \times m}$

$\mathbf{E}'_i = \mathrm{RO}(5, \mathsf{seed}_{i,i}, \mathsf{pk}, \mathrm{SS}, \mu)$

$\boxed{\begin{array}{l} \mathrm{curMS}_1(\mathrm{SS}, \mu) \leftarrow \mathrm{curMS}_1(\mathrm{SS}, \mu) \cup \{i\} \\ \mathsf{CompMask}_1(i, \mathrm{SS}, \mu) \end{array}}$

$\mathbf{Y}_i = L_{\mathrm{SS},i} \cdot \mathbf{U}\mathbf{S}_i + \mathbf{E}'_i + \mathsf{mask}_{i,\mathrm{SS},\mu}$

$$\mathbf{w}_i = \begin{bmatrix} \overline{\mathbf{w}}_i \\ \underline{\mathbf{w}}_i \end{bmatrix} = \begin{bmatrix} \mathbf{A}\mathbf{r}_i \\ \mathbf{U}\mathbf{r}_i + \mathbf{e}_i \end{bmatrix}$$

$\mathsf{st}_i.\mathsf{P}(\mathsf{st}_i.\mathsf{cnt}) = (0, (\mathbf{E}'_i, \mathbf{r}_i, \hat{\mathbf{w}}_i))$

Return $(\mathbf{Y}_i, \mathbf{w}_i)$

Oracle $\textsc{oSign}_2(i, \mathrm{sid}, \mathrm{SS}, \mu, M)$:

Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{sid} \leq \mathsf{st}_i.\mathsf{cnt}$,
$\qquad \mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$

$\{(j, \mathbf{Y}_j, \mathbf{w}_j)\}_{j \in \mathrm{SS}} = M$

$(\mathsf{isUsed}, (\mathbf{E}'_i, \mathbf{r}_i, \hat{\mathbf{w}}_i)) = \mathsf{st}_i.\mathsf{P}(\mathrm{sid})$

If isUsed or $\mathbf{w}_i \neq \hat{\mathbf{w}}_i$ then return $\perp$

$\mathsf{st}_i.\mathsf{P}(\mathrm{sid}) = (1, (\mathbf{E}'_i, \mathbf{r}_i, \hat{\mathbf{w}}_i))$

$\boxed{\begin{array}{l} \text{If } (\mathrm{SS}, \mu, M) \notin \mathrm{setQ}_2 \text{ then} \\ \quad \mathrm{setQ}_2 \leftarrow \mathrm{setQ}_2 \cup \{(\mathrm{SS}, \mu, M)\} \\ \quad \mathrm{curMS}_2(\mathrm{SS}, \mu, M) = \emptyset \\ \quad \text{For } j \in \mathrm{SS} \setminus \mathrm{CS} \text{ do} \\ \qquad \mathrm{curMS}_2(\mathrm{SS}, \mu, M) \leftarrow \\ \qquad\qquad \mathrm{curMS}_2(\mathrm{SS}, \mu, M) \cup \{i\} \\ \qquad \mathsf{CompMask}_2(j, \mathrm{SS}, \mu, M) \end{array}}$

$\mathbf{Y} = \sum_{j \in \mathrm{SS}} \mathbf{Y}_j$; $\mathbf{w} = \sum_{j \in \mathrm{SS}} \mathbf{w}_j$

$\mathbf{c} = \mathrm{RO}(2, \mu, \mathbf{Y}, \mathbf{w}) \in \mathcal{C}^k \subset R^k$

$\boxed{\begin{array}{l} \mathrm{curMS}_2(\mathrm{SS}, \mu, M) \leftarrow \\ \qquad \mathrm{curMS}_2(\mathrm{SS}, \mu, M) \cup \{i\} \\ \mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M) \end{array}}$

$\mathbf{z}_i = \mathbf{r}_i + L_{\mathrm{SS},i} \cdot \mathbf{S}_i\mathbf{c} + \mathsf{mask}_{i,\mathrm{SS},\mu,M}$

Return $\mathbf{z}_i$

Fig. 11. The oracles $\textsc{Cor}$, $\textsc{oSign}_1$, $\textsc{oSign}_2$ of the games $\mathbf{G}_2$ and $\mathbf{G}_3$, where $\mathbf{G}_2$ contains all but the solid boxes and $\mathbf{G}_3$ contains all but the dashed boxes. The $\mathsf{CompMask}$ algorithm is defined in Figure 10. The rest of each game is the same as $\mathbf{G}_2$.

Game $\mathbf{G}_3^{\mathcal{A}}(\kappa)$, $\boxed{\mathbf{G}_4^{\mathcal{A}}(\kappa)}$:

Oracle $\text{Cor}(i)$:
$\mathsf{CS} \leftarrow \mathsf{CS} \cup \{i\}$; $\mathsf{HS} \leftarrow [N] \setminus \mathsf{CS}$
For $(\mathrm{SS}, \mu) \in \mathrm{setQ}_1$ such that $i \in \mathrm{SS}$ do
  $\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)$
  $\boxed{\text{If } i \notin \mathrm{curMS}_1(\mathrm{SS}, \mu) \text{ then } \mathsf{CompMask}_1(i, \mathrm{SS}, \mu)}$
  If $\mathrm{SS} \setminus \mathsf{CS} \neq \emptyset$ then continue
  Pick $j \in \mathrm{SS} \setminus \mathsf{CS}$ arbitrarily
  $H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu) = \mathsf{mask}_{i,\mathrm{SS},\mu} - \sum_{\hat{j} \in \mathrm{SS}\setminus\{j\}} \mathrm{RO}(3, \mathsf{seed}_{i,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu)$
    $+ \sum_{\hat{j} \in \mathrm{SS}} \mathrm{RO}(3, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu))$
For $(\mathrm{SS}, \mu, M) \in \mathrm{setQ}_2$ such that $i \in \mathrm{SS}$ do
  $\mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)$
  $\boxed{\text{If } i \notin \mathrm{curMS}_2(\mathrm{SS}, \mu, M) \text{ then } \mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)}$
  If $\mathrm{SS} \setminus \mathsf{CS} \neq \emptyset$ then continue
  Pick $j \in \mathrm{SS} \setminus \mathsf{CS}$ arbitrarily
  $H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M) = \mathsf{mask}_{i,\mathrm{SS},\mu,M} - \sum_{\hat{j} \in \mathrm{SS}\setminus\{i\}} \mathrm{RO}(4, \mathsf{seed}_{i,\hat{k}}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
    $+ \sum_{\hat{j} \in \mathrm{SS}} \mathrm{RO}(4, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu, M))$
Return st_i

Oracle $\text{oSign}_1(i, \mathrm{SS}, \mu)$:
Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$
$\mathsf{st}_i.\mathsf{cnt} = \mathsf{st}_i.\mathsf{cnt} + 1$
$\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m,\sigma_r}$; $\mathbf{e}_i \leftarrow \mathcal{D}_{\mathcal{R}^t,\sigma_2}$
$\mathbf{U} = H_1(\mu) \in \mathcal{R}_q^{t\times m}$
$\mathrm{curMS}_1(\mathrm{SS}, \mu) \leftarrow \mathrm{curMS}_1(\mathrm{SS}, \mu) \cup \{i\}$
$\mathbf{E}'_{i,\mathrm{SS},\mu} = \mathrm{RO}(5, \mathsf{seed}_{i,i}, \mathsf{pk}, \mathrm{SS}, \mu)$
$\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)$
$\mathbf{Y}_{i,\mathrm{SS},\mu} = L_{\mathrm{SS},i} \cdot \mathbf{US}_i + \mathbf{E}'_{i,\mathrm{SS},\mu} + \mathsf{mask}_{i,\mathrm{SS},\mu}$
$\boxed{\text{If } i \notin \mathrm{curMS}_1(\mathrm{SS}, \mu) \text{ then } \mathsf{SampY}(i, \mathrm{SS}, \mu);\ \mathsf{mask}_{i,\mathrm{SS},\mu} = \mathbf{Y}_{i,\mathrm{SS},\mu} - L_{\mathrm{SS},i} \cdot \mathbf{US}_i - \mathbf{E}'_{i,\mathrm{SS},\mu}}$
$\mathbf{w}_i = \begin{bmatrix} \overline{\mathbf{w}}_i \\ \underline{\mathbf{w}}_i \end{bmatrix} = \begin{bmatrix} \mathbf{Ar}_i \\ \mathbf{Ur}_i + \mathbf{e}_i \end{bmatrix}$
$\mathsf{st}_i.\mathsf{P}(\mathsf{st}_i.\mathsf{cnt}) = (0, (\mathbf{E}'_{i,\mathrm{SS},\mu}, \mathbf{r}_i, \hat{\mathbf{w}}_i))$
Return $(\mathbf{Y}_{i,\mathrm{SS},\mu}, \mathbf{w}_i)$

Oracle $\text{oSign}_2(i, \mathrm{sid}, \mathrm{SS}, \mu, M)$:
Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{sid} \leq \mathsf{st}_i.\mathsf{cnt}$,
  $\mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$
$\{(j, \mathbf{Y}_j, \mathbf{w}_j)\}_{j\in \mathrm{SS}} = M$
$(\mathsf{isUsed}, (\mathbf{E}'_i, \mathbf{r}_{i,\mathrm{SS},\mu,M}, \hat{\mathbf{w}}_i)) = \mathsf{st}_i.\mathsf{P}(\mathrm{sid})$
If isUsed or $\mathbf{w}_i \neq \hat{\mathbf{w}}_i$ then return $\perp$
$\mathsf{st}_i.\mathsf{P}(\mathrm{sid}) = (1, (\mathbf{E}'_i, \mathbf{r}_{i,\mathrm{SS},\mu,M}, \hat{\mathbf{w}}_i))$
$\mathbf{Y} = \sum_{j\in \mathrm{SS}} \mathbf{Y}_j$; $\mathbf{w} = \sum_{j \in \mathrm{SS}} \mathbf{w}_j$
$\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$
$\mathrm{curMS}_2(\mathrm{SS}, \mu, M) \leftarrow$
  $\mathrm{curMS}_2(\mathrm{SS}, \mu, M) \cup \{i\}$
$\mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)$
$\mathbf{z}_{i,\mathrm{SS},\mu,M} = \mathbf{r}_{i,\mathrm{SS},\mu,M} + L_{\mathrm{SS},i} \cdot \mathbf{S}_i\mathbf{c} + \mathsf{mask}_{i,\mathrm{SS},\mu,M}$
$\boxed{\mathsf{Compz}(i, \mathrm{SS}, \mu, M);\ \mathsf{mask}_{i,\mathrm{SS},\mu,M} = \mathbf{z}_{i,\mathrm{SS},\mu,M} - L_{\mathrm{SS},i} \cdot \mathbf{S}_i\mathbf{c} - \mathbf{r}_{i,\mathrm{SS},\mu,M}}$
Return $\mathbf{z}_{i,\mathrm{SS},\mu,M}$

Fig. 12. The Cor, oSign$_1$ and oSign$_2$ oracles of the games $\mathbf{G}_3$ and $\mathbf{G}_4$, where $\mathbf{G}_3$ contains all but the solid boxes and $\mathbf{G}_4$ contains all but the dashed boxes. The algorithms SampY, Compz, CompMask$_1$ and CompMask$_2$ of $\mathbf{G}_4$ are defined in Figures 13 and 14. The rest of $\mathbf{G}_4$ is the same as $\mathbf{G}_3$.

Game $\mathbf{G}_3^{\mathcal{A}}(\kappa)$, $\boxed{\mathbf{G}_4^{\mathcal{A}}(\kappa)}$:

Algorithm $\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)$:

If $i \in \mathrm{curMS}_1(\mathrm{SS}, \mu)$ then return $\perp$
If $\mathrm{SS} \setminus \mathsf{CS} \not\subseteq \mathrm{curMS}_1(\mathrm{SS}, \mu)$ then
  $\mathsf{mask}_{i,\mathrm{SS},\mu} \leftarrow \mathcal{R}_q^{t\times k}$
Else
  For $j \in \mathsf{CS} \cap \mathrm{SS}$ do
    $\mathsf{mask}_{j,\mathrm{SS},\mu} \leftarrow \sum_{\hat{j}\in\mathrm{SS}}(\mathrm{RO}(3, \mathsf{seed}_{j,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu) - \mathrm{RO}(3, \mathsf{seed}_{\hat{j},j}, \mathsf{pk}, \mathrm{SS}, \mu))$
  $\mathsf{mask}_{i,\mathrm{SS},\mu} = -\sum_{j\in\mathrm{SS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu}$

$$\boxed{\begin{aligned}\mathsf{mask}_{i,\mathrm{SS},\mu} = \mathbf{U}\left(\mathbf{S} - \textstyle\sum_{j\in\mathrm{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot\mathbf{S}_j\right) + \textstyle\sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu} \\ - \textstyle\sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{Y}_{j,\mathrm{SS},\mu} - \textstyle\sum_{j\in\mathrm{SS}\cap\mathsf{CS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu}\end{aligned}}$$

Algorithm $\mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)$:

If $i \in \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ then return $\perp$
If $\mathrm{SS} \setminus \mathsf{CS} \not\subseteq \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ then
  $\mathsf{mask}_{i,\mathrm{SS},\mu,M} \leftarrow \mathcal{R}_q^{t\times k}$
Else
  For $j \in \mathsf{CS} \cap \mathrm{SS}$ do
    $\mathsf{mask}_{j,\mathrm{SS},\mu,M} \leftarrow \sum_{\hat{j}\in\mathrm{SS}}(\mathrm{RO}(4, \mathsf{seed}_{j,\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
      $-\mathrm{RO}(4, \mathsf{seed}_{\hat{j},j}, \mathsf{pk}, \mathrm{SS}, \mu, M))$
  $\mathsf{mask}_{i,\mathrm{SS},\mu,M} = -\sum_{j\in\mathrm{SS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu,M}$

$$\boxed{\begin{aligned}\mathsf{mask}_{i,\mathrm{SS},\mu,M} = \textstyle\sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{r}_{j,\mathrm{SS},\mu,M} + \left(\mathbf{S} - \textstyle\sum_{j\in\mathrm{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot\mathbf{S}_j\right)\mathbf{c} \\ - \textstyle\sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{z}_{i,\mathrm{SS},\mu,M} - \textstyle\sum_{j\in\mathrm{SS}\cap\mathsf{CS}\setminus\{i\}} \mathsf{mask}_{j,\mathrm{SS},\mu,M}\end{aligned}}$$

Fig. 13. The algorithms $\mathsf{CompMask}_1$ and $\mathsf{CompMask}_2$ of the games $\mathbf{G}_3$ and $\mathbf{G}_4$, where $\mathbf{G}_3$ contains only the dash boxes and $\mathbf{G}_4$ contains only the solid boxes.

Algorithm $\mathsf{SampY}(i, \mathrm{SS}, \mu)$:

$\mathrm{curMS}_1(\mathrm{SS}, \mu) \leftarrow \mathrm{curMS}_1(\mathrm{SS}, \mu) \cup \{i\}$
$\mathbf{U} = H_1(\mu)$
If $\mathrm{SS} \setminus \mathsf{CS} \not\subseteq \mathrm{curMS}_1(\mathrm{SS}, \mu)$ then
  $\mathbf{Y}_{i,\mathrm{SS},\mu} \leftarrow \mathcal{R}_q^{t\times k}$

Else
  $\mathbf{Y}_{i,\mathrm{SS},\mu} = \mathbf{U}\left(\mathbf{S} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot\mathbf{S}_j\right) + \sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{E}'_{i,\mathrm{SS},\mu}$
    $- \sum_{j\in(\mathrm{SS}\setminus\mathsf{CS})\setminus\{i\}} \mathbf{Y}_{j,\mathrm{SS},\mu} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}} \mathsf{mask}_{j,\mathrm{SS},\mu}$

Algorithm $\mathsf{Compz}(i, \mathrm{SS}, \mu, M)$:

If $i \in \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ then return $\mathbf{z}_{i,\mathrm{SS},\mu,M}$
$\mathrm{curMS}_2(\mathrm{SS}, \mu, M) \leftarrow \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$
If $\mathrm{SS} \setminus \mathsf{CS} \not\subseteq \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ then
  $\mathbf{z}_{i,\mathrm{SS},\mu,M} \leftarrow \mathcal{R}_q^{t\times k}$
Else
  $\mathbf{z}_{i,\mathrm{SS},\mu,M} = \sum_{j\in\mathrm{SS}\setminus\mathsf{CS}} \mathbf{r}_{j,\mathrm{SS},\mu,M} + \left(\mathbf{S} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}} L_{\mathrm{SS},j}\cdot\mathbf{S}_j\right)\mathbf{c}$
    $- \sum_{j\in(\mathrm{SS}\setminus\mathsf{CS})\setminus\{i\}} \mathbf{z}_{i,\mathrm{SS},\mu,M} - \sum_{j\in\mathrm{SS}\cap\mathsf{CS}} \mathsf{mask}_{j,\mathrm{SS},\mu,M}$

Fig. 14. The algorithms SampY, Compz of the games $\mathbf{G}_4$.

Game $\mathbf{G}_4^{\mathcal{A}}(\kappa)$, $\boxed{\mathbf{G}_5^{\mathcal{A}}(\kappa)}$:

Oracle $\textsc{Cor}(i)$:

$\mathsf{CS} \leftarrow \mathsf{CS} \cup \{i\}$; $\mathsf{HS} \leftarrow [N] \setminus \mathsf{CS}$
For $(\mathrm{SS}, \mu) \in \mathrm{setQ}_1$ such that $i \in \mathrm{SS}$ do
  If $i \notin \mathrm{curMS}_1(\mathrm{SS}, \mu)$ then
    $\mathsf{CompMask}_1(i, \mathrm{SS}, \mu)$
  $\boxed{\text{Else } \mathsf{mask}_{i,\mathrm{SS},\mu} = \mathbf{Y}_{i,\mathrm{SS},\mu} - L_{\mathrm{SS},i} \cdot \mathbf{U}\mathbf{S}_i - \mathbf{E}'_{i,\mathrm{SS},\mu}}$
  If $\mathrm{SS} \setminus \mathsf{CS} \neq \emptyset$ then continue
  Pick $j \in \mathrm{SS} \setminus \mathsf{CS}$ arbitrarily
  $H_3(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu) = \mathsf{mask}_{i,\mathrm{SS},\mu} - \sum_{\hat{j} \in \mathrm{SS} \setminus \{j\}} \mathrm{RO}(3, \mathsf{seed}_{\hat{i},\hat{j}}, \mathsf{pk}, \mathrm{SS}, \mu)$
    $+ \sum_{\hat{j} \in \mathrm{SS}} \mathrm{RO}(3, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu))$
For $(\mathrm{SS}, \mu, M) \in \mathrm{setQ}_2$ such that $i \in \mathrm{SS}$ do
  If $i \notin \mathrm{curMS}_2(\mathrm{SS}, \mu, M)$ then
    $\mathsf{CompMask}_2(i, \mathrm{SS}, \mu, M)$
  $\boxed{\text{Else } \mathsf{mask}_{i,\mathrm{SS},\mu,M} = \mathbf{z}_{i,\mathrm{SS},\mu,M} - L_{\mathrm{SS},i} \cdot \mathbf{S}_i\mathbf{c} - \mathbf{r}_{i,\mathrm{SS},\mu,M}}$
  If $\mathrm{SS} \setminus \mathsf{CS} \neq \emptyset$ then continue
  Pick $j \in \mathrm{SS} \setminus \mathsf{CS}$ arbitrarily
  $H_4(\mathsf{seed}_{i,j}, \mathsf{pk}, \mathrm{SS}, \mu, M) = \mathsf{mask}_{i,\mathrm{SS},\mu,M} - \sum_{\hat{j} \in \mathrm{SS} \setminus \{i\}} \mathrm{RO}(4, \mathsf{seed}_{i,\hat{k}}, \mathsf{pk}, \mathrm{SS}, \mu, M)$
    $+ \sum_{\hat{j} \in \mathrm{SS}} \mathrm{RO}(4, \mathsf{seed}_{\hat{j},i}, \mathsf{pk}, \mathrm{SS}, \mu, M))$
Return st_i

Oracle $\textsc{oSign}_1(i, \mathrm{SS}, \mu)$:

Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$
$\mathsf{st}_i.\mathsf{cnt} = \mathsf{st}_i.\mathsf{cnt} + 1$
$\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}$; $\mathbf{e}_i \leftarrow \mathcal{D}_{\mathcal{R}^t, \sigma_2}$
$\mathbf{U} = H_1(\mu) \in \mathcal{R}_q^{t \times m}$
$\mathrm{curMS}_1(\mathrm{SS}, \mu) \leftarrow \mathrm{curMS}_1(\mathrm{SS}, \mu) \cup \{i\}$
$\mathbf{E}'_{i,\mathrm{SS},\mu} = \mathrm{RO}(5, \mathsf{seed}_{i,i}, \mathsf{pk}, \mathrm{SS}, \mu)$
$\mathsf{SampY}(i, \mathrm{SS}, \mu)$
$\mathsf{mask}_{i,\mathrm{SS},\mu} = \mathbf{Y}_{i,\mathrm{SS},\mu} - L_{\mathrm{SS},i} \cdot \mathbf{U}\mathbf{S}_i$
  $- \mathbf{E}'_{i,\mathrm{SS},\mu}$
$$\mathbf{w}_i = \begin{bmatrix} \overline{\mathbf{w}}_i \\ \underline{\mathbf{w}}_i \end{bmatrix} = \begin{bmatrix} \mathbf{A}\mathbf{r}_i \\ \mathbf{U}\mathbf{r}_i + \mathbf{e}_i \end{bmatrix}$$
$\mathsf{st}_i.\mathsf{P}(\mathsf{st}_i.\mathsf{cnt}) = (0, (\mathbf{E}'_{i,\mathrm{SS},\mu}, \mathbf{r}_i, \mathbf{w}_i)$
Return $(\mathbf{Y}_{i,\mathrm{SS},\mu}, \mathbf{w}_i)$

Oracle $\textsc{oSign}_2(i, \mathrm{sid}, \mathrm{SS}, \mu, M)$:

Require: $i \in \mathrm{SS} \setminus \mathsf{CS}$, $\mathrm{sid} \leq \mathsf{st}_i.\mathsf{cnt}$,
  $\mathrm{SS} \subseteq [N]$ and $|\mathrm{SS}| \geq T$
$\{(j, \mathbf{Y}_j, \mathbf{w}_j)\}_{j \in \mathrm{SS}} = M$
$(\mathsf{isUsed}, (\mathbf{E}'_i, \mathbf{r}_{i,\mathrm{SS},\mu}, \mathbf{e}_i)) = \mathsf{st}_i.\mathsf{P}(\mathrm{sid})$
If isUsed then return $\perp$
$\mathsf{st}_i.\mathsf{P}(\mathrm{sid}) = (1, (\mathbf{E}'_i, \mathbf{r}_{i,\mathrm{SS},\mu}, \mathbf{e}_i))$
$\mathbf{Y} = \sum_{j \in \mathrm{SS}} \mathbf{Y}_j$; $\mathbf{w} = \sum_{j \in \mathrm{SS}} \mathbf{w}_j$
$\mathbf{c} = H_2(\mu, \mathbf{Y}, \mathbf{w})$
$\mathrm{curMS}_2(\mathrm{SS}, \mu, M) \leftarrow$
  $\mathrm{curMS}_2(\mathrm{SS}, \mu, M) \cup \{i\}$
$\mathsf{Compz}(i, \mathrm{SS}, \mu, M)$
$\mathsf{mask}_{i,\mathrm{SS},\mu,M} = \mathbf{z}_{i,\mathrm{SS},\mu,M} - L_{\mathrm{SS},i} \cdot \mathbf{S}_i\mathbf{c}$
  $- \mathbf{r}_{i,\mathrm{SS},\mu,M}$
Return $\mathbf{z}_{i,\mathrm{SS},\mu,M}$

Fig. 15. The $\textsc{Cor}$, $\textsc{oSign}_1$, $\textsc{oSign}_2$ oracles of the games $\mathbf{G}_4$ and $\mathbf{G}_5$, where $\mathbf{G}_4$ contains all but the solid boxes and $\mathbf{G}_5$ contains all but the dashed boxes. The rest of $\mathbf{G}_5$ is the same as $\mathbf{G}_4$.